BỘ CÔNG THƯƠNG
TRƯỜNG ĐẠI HỌC CÔNG THƯƠNG TP. HCM
KHOA CÔNG NGHỆ THÔNG TIN

KHÓA LUẬN TỐT NGHIỆP

NGHIÊN CỨU PHƯƠNG PHÁP TÍCH HỢP VMAMBA VÀO MÔ HÌNH YOLO26-SEG TRONG PHÂN ĐOẠN POLYP TỪ ẢNH NỘI SOI ĐẠI TRỰC TRÀNG

Ngành: Công nghệ thông tin

SINH VIÊN THỰC HIỆN:

1. 2001230490 – Lê Đức Lương – 14DHTH09
2. 2001230312 – Phùng Tuấn Huy – 14DHTH13
3. 2001230830 – Trần Mạnh Toàn – 14DHTH09

TP. HỒ CHÍ MINH, tháng 11 năm 2026

BỘ CÔNG THƯƠNG
TRƯỜNG ĐẠI HỌC CÔNG THƯƠNG TP. HCM
KHOA CÔNG NGHỆ THÔNG TIN

KHÓA LUẬN TỐT NGHIỆP

NGHIÊN CỨU PHƯƠNG PHÁP TÍCH HỢP VMAMBA VÀO MÔ HÌNH YOLO26-SEG TRONG PHÂN ĐOẠN POLYP TỪ ẢNH NỘI SOI ĐẠI TRỰC TRÀNG

Ngành: Công nghệ thông tin

GIẢNG VIÊN HƯỚNG DẪN:

Phùng Thế Bảo

SINH VIÊN THỰC HIỆN:

1. 2001230490 – Lê Đức Lương – 14DHTH09
2. 2001230312 – Phùng Tuấn Huy – 14DHTH13
3. 2001230830 – Trần Mạnh Toàn – 14DHTH09

TP. HỒ CHÍ MINH, tháng 11 năm 2026

LỜI CẢM ƠN

Nhóm thực hiện xin gửi lời cảm ơn chân thành đến thầy **Phùng Thế Bảo**, giảng viên hướng dẫn. Trong suốt mười hai tuần thực hiện đề tài, thầy đã định hướng nội dung nghiên cứu, góp ý cho từng giai đoạn và nhắc nhóm luôn kiểm chứng kết quả một cách cẩn thận.

Nhóm xin cảm ơn quý thầy cô Khoa Công nghệ Thông tin, Trường Đại học Công Thương TP. Hồ Chí Minh. Những kiến thức thầy cô truyền đạt trong những năm học là nền tảng để nhóm thực hiện khóa luận này.

Nhóm cũng cảm ơn các nhóm tác giả đã công bố miễn phí các bộ dữ liệu Kvasir-SEG, Kvasir, CVC-ClinicDB, CVC-ColonDB, ETIS-Larib và PolypGen cùng các thư viện mã nguồn mở được sử dụng trong đề tài. Nhờ đó nhóm có thể thực nghiệm và đối chiếu kết quả. Nhóm cảm ơn nền tảng Kaggle đã cung cấp tài nguyên GPU cho quá trình huấn luyện.

Cuối cùng, nhóm xin cảm ơn gia đình và bạn bè đã luôn động viên, hỗ trợ trong suốt thời gian học tập và thực hiện đề tài.

Dù đã cố gắng, khóa luận khó tránh khỏi thiếu sót. Nhóm mong nhận được góp ý của quý thầy cô để hoàn thiện hơn.

TP. Hồ Chí Minh, tháng 11 năm 2026 Nhóm sinh viên thực hiện

MỤC LỤC

LỜI CẢM ƠN i
MỤC LỤC ii
DANH MỤC CÁC KÝ HIỆU VÀ CHỮ VIẾT TẮT viii
Chữ viết tắt viii
Ký hiệu x
DANH MỤC HÌNH xi
DANH MỤC BẢNG xiii
MỞ ĐẦU 1
1. Lý do chọn đề tài 1
2. Mục tiêu nghiên cứu 2
3. Đối tượng và phạm vi nghiên cứu 3
4. Phương pháp nghiên cứu 4
5. Đóng góp của đề tài 5
6. Kế hoạch thực hiện và phân công công việc 5
6.1. Kế hoạch theo tuần 5
6.2. Phân công công việc 7
6.3. Cách phối hợp và theo dõi tiến độ 8
7. Bố cục báo cáo 8
CHƯƠNG 1: TỔNG QUAN 9
1.1. Bối cảnh y khoa 9
1.1.1. Ung thư đại trực tràng và polyp 9
1.1.2. Giới hạn của nội soi 9
1.2. Bài toán phân đoạn polyp 9
1.2.1. Phát biểu bài toán 9
1.2.2. Các khó khăn đặc thù 10
1.3. Các bộ dữ liệu công khai 10
1.4. Các hướng nghiên cứu phân đoạn polyp 11
1.4.1. Mạng tích chập dạng mã hóa – giải mã 11
1.4.2. Transformer và mô hình lai 12
1.4.3. Phát hiện và phân đoạn một giai đoạn: họ YOLO 12
1.4.4. Mô hình không gian trạng thái và Mamba 13

1.4.5. Mamba cho ảnh y tế và polyp 13
1.4.6. Mamba trong họ YOLO 14
1.5. Tổng hợp và khoảng trống nghiên cứu 14
1.5.1. Bảng tổng hợp các hướng tiếp cận 14
1.5.2. Vị trí của đề tài 15
1.6. Tình hình nghiên cứu trong nước 16
1.7. Kết chương 17
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT 18
2.1. Các bài toán thị giác máy tính 18
2.2. Mạng nơ-ron tích chập 19
2.2.1. Phép tích chập 19
2.2.2. Các thành phần khác 20
2.3. Họ mô hình YOLO 20
2.3.1. Ý tưởng phát hiện một giai đoạn 20
2.3.2. Gán nhãn và hàm mất mát 21
2.3.3. Bỏ bước lọc trùng NMS 21
2.3.4. Phân đoạn thực thể bằng mặt nạ mẫu 21
2.4. Mô hình YOLO26-seg 22
2.4.1. Kiến trúc 22
2.4.2. Các bước của thuật toán YOLO26-seg 23
2.4.3. Ưu và nhược điểm 25
2.5. Cơ chế chú ý và chi phí tính toán 25
2.5.1. Tự chú ý 25
2.5.2. Chi phí bậc hai 25
2.6. Mô hình không gian trạng thái, Mamba và VMamba 26
2.6.1. Mô hình không gian trạng thái liên tục 26
2.6.2. Rời rạc hóa 26
2.6.3. Mamba: SSM có chọn lọc 27
2.6.4. Đưa Mamba vào ảnh: VMamba và SS2D 28
2.6.5. Ưu và nhược điểm của VMamba 29
2.7. Các độ đo đánh giá 29
2.7.1. Các đại lượng cơ bản 29
2.7.2. IoU và Dice 30

2.7.3. AP, mAP50 và mAP50–95 31
2.7.4. Chi phí tính toán 32
2.8. So sánh hai mô hình qua nhiều lần huấn luyện 32
2.8.1. Vì sao cần nhiều seed 32
2.8.2. Kiểm định t ghép cặp 32
2.8.3. Kiểm định Wilcoxon và cỡ hiệu ứng 33
2.9. Kết chương 34
CHƯƠNG 3: MÔ HÌNH ĐỀ XUẤT 35
3.1. Ý tưởng và động cơ thiết kế 36
3.1.1. Vấn đề cần giải quyết 36
3.1.2. Vì sao chọn tầng 10 36
3.1.3. Phạm vi đóng góp 37
3.2. Kiến trúc tổng thể 38
3.3. Khối C2TSVMamba 39
3.3.1. Khung tách–xử lý–ghép 39
3.3.2. Nhánh SS2D 40
3.3.3. Nhánh hình dạng 43
3.3.4. Cổng 44
3.3.5. Hợp nhất, mạng truyền thẳng và nối tắt có hệ số 45
3.3.6. Ví dụ truy vết kích thước tensor 46
3.3.7. Số tham số 46
3.4. Thuật toán quét kết hợp song song 47
3.4.1. Đưa phép truy hồi về phép quét 47
3.4.2. Thuật toán trong mã 48
3.4.3. Ví dụ minh họa 49
3.4.4. Độ phức tạp 49
3.4.5. Ước tính chi phí của tầng 10 50
3.5. Khởi tạo trọng số và học chuyển giao 51
3.6. Hàm mất mát và chiến lược huấn luyện 52
3.7. Quy trình suy luận và huấn luyện của mô hình đề xuất 54
3.8. Lưu trọng số ở dạng FP32 55
3.9. Ưu điểm và hạn chế của mô hình đề xuất 56
3.10. Các chế độ tách thành phần có sẵn trong mã 57

3.11. Kết chương 58
CHƯƠNG 4: XÂY DỰNG BỘ DỮ LIỆU VÀ TIỀN XỬ LÝ DỮ LIỆU 59
4.1. Nguồn dữ liệu 59
4.1.1. Bộ dữ liệu Kvasir-SEG 59
4.1.2. Ảnh nền normal-cecum 59
4.2. Xây dựng và phân chia bộ dữ liệu BG20 60
4.3. Chuyển mặt nạ phân đoạn thành nhãn đa giác 61
4.3.1. Nhị phân hóa mặt nạ bằng phương pháp Otsu 61
4.3.2. Làm liền vùng polyp bằng phép đóng hình thái học 61
4.3.3. Tìm đường bao và loại bỏ vùng nhiễu nhỏ 62
4.3.4. Đơn giản hóa đường bao 62
4.3.5. Chuẩn hóa tọa độ và ghi tệp nhãn 62
4.4. Đổi kích thước ảnh 63
4.5. Tăng cường dữ liệu 65
4.6. Định dạng và chuẩn hóa dữ liệu đầu vào 67
4.7. Kiểm tra tính toàn vẹn của dữ liệu 67
4.8. Tổng hợp quy trình tiền xử lý 67
CHƯƠNG 5: CÀI ĐẶT VÀ ỨNG DỤNG 69
5.1. Cấu hình hệ thống và môi trường thực nghiệm 69
5.1.1. Môi trường huấn luyện 69
5.1.2. Môi trường đánh giá và đo hiệu năng 70
5.1.3. Ngôn ngữ và công cụ 70
5.2. Cài đặt mô hình gốc YOLO26s-seg 71
5.3. Cài đặt mô hình tích hợp VMamba 72
5.3.1. Tổ chức mã nguồn 72
5.3.2. Khởi tạo và huấn luyện 73
5.3.3. Bản vá lưu checkpoint ở FP32 73
5.3.4. Thí nghiệm tách thành phần 74
5.4. Quy trình huấn luyện và tổ chức lưu trữ kết quả thực nghiệm 74
5.4.1. Notebook Kaggle 74
5.4.2. Cấu trúc lưu trữ kết quả 75
5.5. Ứng dụng PolypVision: tổng quan 76
5.5.1. Yêu cầu chức năng 76

5.5.2. Kiến trúc hệ thống 77
5.6. Dịch vụ suy luận FastAPI 78
5.6.1. Nạp mô hình 78
5.6.2. Các API 79
5.7. Tổ chức lưu trữ kết quả 80
5.8. Giao diện và chức năng 81
5.8.1. Website 81
5.8.2. Ứng dụng di động 83
5.8.3. Kiểm thử phần mềm 84
5.9. Triển khai trên dữ liệu thực tế 85
5.10. Hạn chế của phần cài đặt 85
5.11. Kết chương 86
CHƯƠNG 6: THỰC NGHIỆM VÀ ĐÁNH GIÁ 87
6.1. Thiết lập thực nghiệm 87
6.1.1. Các mô hình và cấu hình huấn luyện 87
6.1.2. Tập đánh giá 88
6.1.3. Chỉ số chính và cách trích số 88
6.1.4. Kiểm định thống kê 88
6.1.5. Đánh giá theo pixel 89
6.2. So sánh mô hình gốc và mô hình đề xuất trên 20 seed 90
6.2.1. Kết quả tổng hợp 90
6.2.2. Độ ổn định giữa các seed 92
6.2.3. Độ nhạy theo quy tắc chọn epoch 93
6.2.4. Quá trình hội tụ 95
6.2.5. Ma trận nhầm lẫn theo đối tượng 96
6.3. Đánh giá theo pixel và so sánh với PraNet, U-Net 97
6.4. Chỉ số topology của mặt nạ 101
6.4.1. Mục đích và cách đo 101
6.4.2. Kết quả 101
6.5. Chi phí tính toán 103
6.5.1. Số tham số và số phép tính 103
6.5.2. Độ trễ suy luận 104
6.5.3. Nhận xét 105

6.6. Thí nghiệm tách thành phần 105
6.6.1. Thiết kế 105
6.6.2. Kết quả 106
6.6.3. Cách đọc kết quả 108
6.7. Kết quả trên dữ liệu thực tế 108
6.7.1. Dữ liệu và giao thức 108
6.7.2. Kết quả 109
6.7.3. Kết quả theo trung tâm của PolypGen 111
6.8. Thảo luận 112
6.8.1. Trả lời câu hỏi nghiên cứu 112
6.8.2. Vì sao mức cải thiện nhỏ 113
6.8.3. Giới hạn của thực nghiệm 113
6.9. Kết chương 114
KẾT LUẬN 115
1. Kết quả đạt được 115
2. Đánh giá kết quả 116
3. Hạn chế 117
4. Hướng phát triển 118
TÀI LIỆU THAM KHẢO 120
PHỤ LỤC 127
Phụ lục A. Kết quả từng seed 127
A.1. Mô hình gốc và mô hình đề xuất (20 seed) 127
A.2. Dice theo pixel của bốn mô hình (20 seed) 127
A.3. Thí nghiệm tách thành phần (5 seed) 128
Phụ lục B. Cấu hình huấn luyện 128
Phụ lục C. Tổ chức mã nguồn và tệp kết quả 129
Phụ lục D. Hướng dẫn chạy lại 130

DANH MỤC CÁC KÝ HIỆU VÀ CHỮ VIẾT TẮT

CHỮ VIẾT TẮT

Viết tắt	Tiếng Anh	Tiếng Việt
AP	Average Precision	Độ chính xác trung bình
API	Application Programming Interface	Giao diện lập trình ứng dụng
BG20	Background 20%	Bộ dữ liệu của đề tài: Kvasir-SEG cộng số ảnh nền bằng 20% số ảnh polyp
BN	Batch Normalization	Chuẩn hóa theo lô
C2PSA	CSP block with Partial Self-Attention	Khối tự chú ý từng phần của YOLO26 (tầng 10 của mô hình gốc)
C2TSVMamba	CSP block with Topology–Shape-aware VMamba	Khối đề xuất thay C2PSA ở tầng 10
CI	Confidence Interval	Khoảng tin cậy
CNN	Convolutional Neural Network	Mạng nơ-ron tích chập
COCO	Common Objects in Context	Bộ dữ liệu ảnh dùng để tiền huấn luyện YOLO
CPU	Central Processing Unit	Bộ xử lý trung tâm
CSP	Cross Stage Partial	Cấu trúc tách–ghép kênh qua các tầng
CUDA	Compute Unified Device Architecture	Nền tảng tính toán song song trên GPU của NVIDIA
CVC	Computer Vision Center	Trung tâm Thị giác máy tính (Barcelona), nơi công bố các bộ CVC-ClinicDB, CVC-ColonDB
DFL	Distribution Focal Loss	Hàm mất mát tiêu điểm theo phân phối
EMA	Exponential Moving Average	Trung bình trượt hàm mũ
FFN	Feed-Forward Network	Mạng truyền thẳng
FLOPs	Floating-Point Operations	Số phép tính dấu phẩy động
FN	False Negative	Âm tính giả (polyp bị bỏ sót)
FP	False Positive	Dương tính giả (báo nhầm)
FP16, FP32	16-bit / 32-bit Floating Point	Số thực dấu phẩy động 16 bit / 32 bit
FPN	Feature Pyramid Network	Mạng kim tự tháp đặc trưng

Viết tắt	Tiếng Anh	Tiếng Việt
GFLOPs	Giga Floating-Point Operations	Tỉ phép tính dấu phẩy động
GPU	Graphics Processing Unit	Bộ xử lý đồ họa
HSV	Hue, Saturation, Value	Không gian màu sắc độ – bão hòa – độ sáng
IEEE	Institute of Electrical and Electronics Engineers	Viện Kỹ sư Điện và Điện tử (kiểu trích dẫn)
IoU	Intersection over Union	Tỉ lệ giao trên hợp
MAC	Multiply–Accumulate	Phép nhân–cộng
mAP	mean Average Precision	Trung bình độ chính xác trung bình
mAP50, mAP50–95	mAP at IoU 0.5 / averaged over IoU 0.5–0.95	mAP ở ngưỡng IoU 0,5 / trung bình trên các ngưỡng IoU 0,5–0,95
NMS	Non-Maximum Suppression	Lọc trùng hộp không cực đại
P, R	Precision, Recall	Độ chính xác, độ phủ
P3, P4, P5	Pyramid levels 3–5	Các mức đặc trưng có bước 8, 16, 32 pixel
PAN	Path Aggregation Network	Mạng tổng hợp đường dẫn
PSA	Partial Self-Attention	Tự chú ý từng phần
SPPF	Spatial Pyramid Pooling – Fast	Gộp kim tự tháp không gian, bản nhanh
SS2D	2D Selective Scan	Quét chọn lọc hai chiều (bốn hướng)
SSM	State Space Model	Mô hình không gian trạng thái
STAL	Small-Target-Aware Label assignment	Gán nhãn có ưu tiên vật nhỏ
TN	True Negative	Âm tính đúng
TP	True Positive	Dương tính đúng
TSVM	Topology–Shape-aware VMamba	Tên gọi tắt của mô hình đề xuất (YOLO26s-seg + C2TSVMamba)
VMamba	Visual Mamba (Visual State Space Model)	Mô hình không gian trạng thái thị giác
YOLO	You Only Look Once	Họ mô hình phát hiện một giai đoạn
ZOH	Zero-Order Hold	Giữ bậc không (cách rời rạc hóa SSM)

KÝ HIỆU

Ký hiệu	Ý nghĩa	Xuất hiện ở
A, B, C, Δ	Ma trận trạng thái, ma trận đầu vào, ma trận đầu ra và bước thời gian của mô hình không gian trạng thái	Mục 2.6, 3.3–3.4
$\bar{A}, \bar{B}$	A, B sau rời rạc hóa	Mục 2.6.2
h_t	Trạng thái ẩn tại bước t	Mục 2.6, 3.4
F, G	Đặc trưng của nhánh SS2D và tín hiệu cổng; cổng có dạng $F \cdot (1 + G)$	Mục 3.3.4
b_0, b_1	Số mảnh và số lỗ của mặt nạ (hai số Betti đầu tiên)	Mục 6.4
Δ (ở Chương 6)	Hiệu số chỉ số giữa hai mô hình cùng seed	Chương 6
d_z	Cỡ hiệu ứng Cohen cho dữ liệu ghép cặp	Mục 2.8.3, 6.2
p	Giá trị p của kiểm định thống kê	Mục 2.8, Chương 6
SD	Độ lệch chuẩn qua các seed (ddof = 1)	Chương 6

DANH MỤC HÌNH

Hình 2.1: Bốn kiểu đầu ra trên cùng một ảnh tập kiểm định có hai polyp: (a) phân loại ảnh, (b) phát hiện đối tượng bằng hộp bao, (c) phân đoạn ngữ nghĩa (hai polyp chung một mặt nạ), (d) phân đoạn thực thể (mỗi polyp một mặt nạ riêng). Hộp bao và mặt nạ vẽ từ nhãn đa giác của ảnh; xác suất ở ô (a) là số minh họa, không phải đầu ra mô hình. 19

Hình 3.1: Kiến trúc YOLO26s-seg và vị trí khối C2TSVMamba (tầng 10). Kích thước tensor ghi theo dạng C×H×W, tính cho ảnh vào 640×640 với hệ số quy mô s. Tầng 22 (C3k2, attn=True) có PSABlock ở cả hai mô hình. 38

Hình 3.2: Cấu trúc khối C2TSVMamba. Khung nét đứt là khối TSVMamba. Đặc trưng bên trong khối có kích thước 256×20×20. Chi tiết nhánh SS2D ở Hình 3.4. 40

Hình 3.3: Bốn thứ tự duyệt của SS2D, minh họa trên lưới 4×4 (thực tế là 20×20, L = 400). Chấm tròn đánh dấu vị trí đầu chuỗi, ô vuông đánh dấu vị trí cuối chuỗi. 41

Hình 3.4: Các bước bên trong nhánh SS2D như được cài đặt. Bước ③ đến ⑤ được thực hiện riêng cho từng hướng. 41

Hình 3.5: Số tham số của từng thành phần trong khối C2TSVMamba. Tiêu đề hình ghi tổng của C2TSVMamba và của C2PSA để so sánh. 47

Hình 4.1: Ví dụ ảnh có polyp, mặt nạ phân đoạn gốc và ảnh nền trong bộ dữ liệu BG20 60

Hình 4.2: Trước và sau khi chuyển mặt nạ phân đoạn thành nhãn đa giác..... 63

Hình 4.3: Trước và sau khi đổi kích thước ảnh bằng thu phóng giữ tỷ lệ và LetterBox..... 64

Hình 4.4: Ảnh huấn luyện trước và sau các phép tăng cường dữ liệu 66

Hình 4.5: Quy trình xây dựng bộ dữ liệu BG20 và tiền xử lý dữ liệu 68

Hình 5.2: Kiến trúc hệ thống PolypVision. Website (Spring Boot, cổng 8080) và ứng dụng di động (Flutter) là hai client độc lập của cùng dịch vụ suy luận FastAPI (cổng 8000). Chỉ nhánh Web có MongoDB và kho tệp ảnh. 77

Hình 6.1: Trung bình các chỉ số mặt nạ của hai mô hình trên 20 seed. Thanh lỗi là ±1 độ lệch chuẩn qua các seed. 90

Hình 6.2: Mask mAP50–95 của từng seed. Mỗi đường nối hai lượt cùng seed của hai mô hình. 91

Hình 6.3: Hiệu số mask mAP50–95 (TSVM − gốc) của từng seed, sắp xếp từ nhỏ đến lớn. 92

Hình 6.4: Phân bố mask mAP50–95 qua 20 seed của hai mô hình. 93

Hình 6.5: Mask mAP50–95 theo ba quy tắc chọn epoch. Điểm là trung bình, thanh là ±1 độ lệch chuẩn..... 94

Hình 6.6: Mask mAP50–95 trên tập kiểm định theo epoch. Đường là trung bình 20 seed, miền tô là ±1 độ lệch chuẩn. Ô nhỏ phóng to đoạn epoch 70–100..... 95

Hình 6.7: Hàm mất mát phân đoạn trên tập kiểm định theo epoch. Dấu ▼ đánh dấu epoch có giá trị trung bình thấp nhất. 96

Hình 6.8: Ma trận nhầm lẫn trung bình 20 seed. Ô nền–nền để trống vì không xác định được ở mức đối tượng. 97

Hình 6.9: Dice, IoU, Precision và Recall theo pixel của bốn mô hình. Thanh lỗi là ±1 độ lệch chuẩn qua 20 seed. 98

Hình 6.10: Dice theo pixel của từng seed. 99

Hình 6.11: Tỉ lệ ảnh nền bị báo nhầm (có ít nhất một pixel dự đoán là polyp) trên 40 ảnh nền. 100

Hình 6.12: Ví dụ dự đoán của bốn mô hình (seed 0). Ba ảnh được chọn tự động: ảnh cả bốn mô hình đều tốt, ảnh TSVM hơn mô hình gốc nhiều nhất và ảnh khó nhất. 100

Hình 6.13: Tỉ lệ ảnh có polyp được dự đoán đúng topology (số mảnh và số lỗ khớp nhãn)..... 102

Hình 6.14: Tỉ lệ ảnh mắc từng kiểu lỗi topology: thừa mảnh, có lỗ và thiếu mảnh. 102

Hình 6.15: Độ trễ trên CPU của hai mô hình ở hai chế độ đo. Thanh lỗi là ±1 độ lệch chuẩn qua 100 lần đo. 104

Hình 6.16: Mask mAP50–95 từng seed (0–4) của năm cấu hình. Đường xám nối kết quả của cùng một seed; vạch ngang đậm là trung bình, thanh dọc là ±1 độ lệch chuẩn qua 5 seed. 107

Hình 6.17: Dice theo pixel của bốn mô hình trên bốn bộ dữ liệu ngoài. Thanh lỗi là ±1 độ lệch chuẩn qua 20 seed..... 110

Hình 6.18: Dice theo pixel trên từng trung tâm của PolypGen.....111

Hình 6.19: Tỉ lệ ảnh không có polyp của PolypGen (125 ảnh) bị báo nhầm..... 112

DANH MỤC BẢNG

Bảng 0.1: Kế hoạch thực hiện theo tuần 6
Bảng 0.2: Phân công nhiệm vụ trong nhóm 7
Bảng 0.3: Phân công theo hạng mục công việc 7
Bảng 1.1: Một số bộ dữ liệu phân đoạn polyp công khai 10
Bảng 1.2: Tổng hợp các nhóm phương pháp phân đoạn polyp 14
Bảng 1.3: So sánh đề tài với các công trình gần nhất 15
Bảng 2.1: So sánh ba bài toán thị giác cơ bản 18
Bảng 2.2: Các khối chính của YOLO26s-seg (ảnh vào 640×640) 22
Bảng 2.3: Ưu và nhược điểm của YOLO26-seg với bài toán phân đoạn polyp 25
Bảng 2.4: Ưu và nhược điểm của VMamba 29
Bảng 3.1: So sánh mô hình đối chứng và mô hình đề xuất 39
Bảng 3.2: Kích thước tensor qua khối C2TSVMamba (một ảnh 640×640) 46
Bảng 3.3: Số tham số của tầng 10 (quy mô s) 46
Bảng 3.4: Các vòng của phép quét song song trên ví dụ 49
Bảng 3.5: Ước tính chi phí phần xử lý chính của tầng 10. Không tính cv1, cv2 vì hai khối dùng chung, khoảng 210 triệu MAC. 51
Bảng 3.6: Các thành phần mất mát của YOLO26-seg (theo đúng nhánh mã đã chạy) 53
Bảng 3.7: Bốn chế độ của khối TSVMamba 57
Bảng 4.1: Phân bố ảnh và nhãn trong bộ dữ liệu BG20 60
Bảng 4.2: Thông số tăng cường dữ liệu trong cấu hình huấn luyện 65
Bảng 5.1: Môi trường huấn luyện 69
Bảng 5.2: Môi trường đánh giá cục bộ 70
Bảng 5.3: Ngôn ngữ, framework và lý do lựa chọn 71
Bảng 5.4: Siêu tham số huấn luyện (giống nhau ở mọi lượt của cả hai mô hình) 72
Bảng 5.5: Các tệp được thêm hoặc sửa trong fork 72
Bảng 5.6: Các cấu hình tách thành phần 74
Bảng 5.7: Các tệp kết quả chính 75
Bảng 5.8: Đối chiếu yêu cầu đề cương với chức năng đã cài đặt 76
Bảng 5.9: Hai mô hình trong dịch vụ suy luận 78
Bảng 5.10: Các API của dịch vụ suy luận 79

Bảng 5.11: Các trường chính trong kết quả suy luận 79
Bảng 5.12: Tập `users` 80
Bảng 5.13: Tập `diagnosis_history` 80
Bảng 5.14: Kết quả kiểm thử tự động **[CẦN KIỂM TRA: chạy lại các lệnh kiểm thử trước khi nộp và cập nhật số]** 84
Bảng 6.1: Các mô hình được đánh giá 87
Bảng 6.2: Chỉ số mặt nạ của hai mô hình (20 seed, epoch có mask mAP50–95 lớn nhất) 90
Bảng 6.3: Mức dao động của mask mAP50–95 qua 20 seed 92
Bảng 6.4: Mask mAP50–95 theo ba quy tắc chọn epoch 94
Bảng 6.5: Mask mAP50–95 tại một số epoch 95
Bảng 6.6: Số vùng polyp phát hiện đúng, bỏ sót và dự đoán thừa (trên 127 vùng polyp) 96
Bảng 6.7: Chỉ số theo pixel của bốn mô hình trên tập kiểm định (20 seed) 97
Bảng 6.8: So sánh ghép cặp TSVM với từng mô hình theo pixel (cùng seed) 98
Bảng 6.9: Chỉ số topology của bốn mô hình (ngưỡng 20 pixel, 20 seed) 101
Bảng 6.10: Số tham số và số phép tính của hai mô hình 103
Bảng 6.11: Độ trễ suy luận của hai mô hình 104
Bảng 6.12: Kết quả thí nghiệm tách thành phần (mask mAP50–95, seed 0–4) 106
Bảng 6.13: Các bộ dữ liệu ngoài 108
Bảng 6.14: Dice và IoU theo pixel trên bốn bộ dữ liệu ngoài (20 seed) 109
Bảng 6.15: Tỉ lệ ảnh có polyp bị bỏ sót hoàn toàn (mô hình không dự đoán pixel nào) 110
Bảng 6.16: Dice theo pixel trên từng trung tâm của PolypGen (20 seed) 111
Bảng PL.1: Chỉ số mặt nạ của từng seed trên tập kiểm định 127
Bảng PL.2: Dice theo pixel trên 120 ảnh có polyp của tập kiểm định 127
Bảng PL.3: Mask mAP50–95 từng seed của năm cấu hình 128
Bảng PL.4: Siêu tham số huấn luyện (theo `args.yaml` của 40 lượt chính) 128
Bảng PL.5: Các thư mục chính của kho mã 129
Bảng PL.6: Script tạo ra số liệu trong báo cáo 130

MỞ ĐẦU

1. LÝ DO CHỌN ĐỀ TÀI

Ung thư đại trực tràng là một trong những bệnh ung thư phổ biến và gây tử vong nhiều nhất. Theo ước tính GLOBOCAN năm 2022, bệnh này chiếm 9,6% số ca ung thư mới, đứng thứ ba, và 9,3% số ca tử vong do ung thư, đứng thứ hai [1]. Phần lớn các ca bệnh phát triển từ polyp ở niêm mạc ruột, nên phát hiện và cắt bỏ polyp sớm qua nội soi là cách phòng bệnh quan trọng [2].

Tuy vậy, kết quả nội soi phụ thuộc nhiều vào người thực hiện, và polyp vẫn bị bỏ sót với tỉ lệ đáng kể [3]. Phát hiện được nhiều u tuyến hơn gắn với nguy cơ ung thư thấp hơn [4]. Chương 1 trình bày chi tiết các số liệu này. Chúng cho thấy một công cụ tự động khoanh vùng polyp trên ảnh có giá trị hỗ trợ thực tế.

Bài toán đặt ra là **phân đoạn polyp**: xác định chính xác những điểm ảnh thuộc polyp, chứ không chỉ đánh dấu một hộp bao quanh. Đây là bài toán khó vì:

- màu và kết cấu của polyp thường gần giống niêm mạc xung quanh, đường biên mờ;
- ảnh có vùng phản quang, nếp gấp, bọt hoặc dịch che khuất;
- kích thước polyp thay đổi lớn, từ rất nhỏ đến chiếm gần hết khung hình.

Các bộ dữ liệu công khai như Kvasir-SEG [5] cho phép nghiên cứu bài toán này theo một quy trình có thể kiểm tra lại.

Về phương pháp, YOLO26-seg [6] (công bố năm 2026, bản tiền ấn phẩm) phát hiện và phân đoạn từng đối tượng trong một lần suy luận, không cần bước lọc trùng NMS, nên phù hợp với yêu cầu xử lý nhanh. Ngữ cảnh toàn cục của mô hình chủ yếu đến từ các khối chú ý (attention) đặt ở mức đặc trưng sâu nhất; phần lớn các khối còn lại là tích chập, chỉ nhìn vùng lân cận nhỏ. Mô hình không gian trạng thái thị giác VMamba [7] là một cách khác để lấy ngữ cảnh toàn cục: mô-đun SS2D quét bản đồ đặc trưng theo bốn hướng, nhờ đó mỗi vị trí nhận thông tin từ cả bốn phía. Đề tài kết hợp cơ chế quét này với một nhánh trích đặc trưng hình dạng và đường biên, vốn là manh mối quan trọng với polyp. Việc thay thế không nhằm làm mô hình nhẹ hơn: ở tầng 10, bản đồ đặc trưng chỉ có 20×20 vị trí, và khối mới thực tế làm tăng số tham số cũng như thời gian suy luận. Chi phí tăng thêm là một câu hỏi cần đo (CH3).

Từ đó, đề tài đặt câu hỏi: **thay khối chú ý ở tầng sâu của YOLO26-seg bằng một khối VMamba có thêm nhánh nhận biết hình dạng thì chất lượng phân đoạn polyp thay đổi thế nào, và phải trả thêm bao nhiêu chi phí tính toán?** Để trả lời một cách tin cậy, đề tài không dựa vào một lần huấn luyện may mắn. Mỗi mô hình được huấn luyện lặp lại với 20 seed khác nhau (seed 0–19), rồi so sánh ghép cặp có kiểm định thống kê.

2. MỤC TIÊU NGHIÊN CỨU

Mục tiêu tổng quát: nghiên cứu và đánh giá phương pháp tích hợp khối VMamba vào YOLO26-seg cho bài toán phân đoạn polyp trên ảnh nội soi đại trực tràng, đồng thời xây dựng ứng dụng minh họa.

Mục tiêu cụ thể (theo đề cương đã duyệt):

1. Khảo sát cơ sở lý thuyết và các công trình liên quan đến phân đoạn polyp, YOLO26-seg, Mamba và VMamba.
2. Xây dựng và tiền xử lý bộ dữ liệu ảnh nội soi có mặt nạ phân đoạn từ các bộ dữ liệu công khai, dùng một cách chia dữ liệu thống nhất cho mọi thực nghiệm.
3. Đề xuất và cài đặt phương án tích hợp khối VMamba vào backbone của YOLO26-seg. Phương án được chọn là khối C2TSVMamba thay khối C2PSA ở tầng 10.
4. So sánh mô hình đề xuất với YOLO26-seg nguyên bản về chất lượng phân đoạn và chi phí tính toán; xây dựng ứng dụng minh họa kết quả.

Câu hỏi nghiên cứu:

- **CH1.** Trên cùng dữ liệu và cùng cấu hình huấn luyện, mô hình tích hợp có mask mAP50–95 trung bình cao hơn mô hình gốc không, và chênh lệch đó có ý nghĩa thống kê không? Cần lưu ý trước: dao động giữa các seed (khoảng 0,01) lớn hơn mức chênh lệch có thể kỳ vọng (vài phần nghìn), nên ngay cả với 20 seed, khả năng phát hiện một chênh lệch nhỏ vẫn còn hạn chế.
- **CH2.** Mỗi thành phần của khối đề xuất (nhánh SS2D, nhánh hình dạng, cổng điều khiển) đóng góp bao nhiêu? Câu hỏi này được trả lời bằng thí nghiệm tách thành phần (ablation).
- **CH3.** Khối đề xuất làm tăng số tham số, chi phí tính toán và thời gian suy luận bao nhiêu?

– **CH4.** Mô hình đã huấn luyện có nạp và chạy đúng trong một ứng dụng thực tế (Web, Mobile) không, và hoạt động ra sao trên ảnh ngoài tập huấn luyện?

3. ĐỐI TƯỢNG VÀ PHẠM VI NGHIÊN CỨU

Đối tượng nghiên cứu:

– Ảnh nội soi đại trực tràng dạng ảnh tĩnh, có hoặc không có polyp.

– Mô hình YOLO26s-seg (cỡ *small*) và mô hình YOLO26s-seg tích hợp khối C2TSVMamba.

– Khối VMamba, cụ thể là mô-đun quét chọn lọc hai chiều SS2D.

Phạm vi dữ liệu:

– Bộ dữ liệu thực nghiệm BG20 gồm 1.200 ảnh: 1.000 ảnh polyp của Kvasir-SEG [5] và 200 ảnh manh tràng bình thường (lớp *normal-cecum*) của Kvasir v2 [8] làm ảnh nền (không có polyp).

– Dữ liệu chia thành 1.040 ảnh huấn luyện và 160 ảnh kiểm định. Phần ảnh polyp dùng đúng danh sách chia 880/120 do nhóm tác giả bộ dữ liệu cung cấp.

– Bộ dữ liệu không có tập kiểm tra (test) riêng. Mọi chỉ số định lượng chính được đo trên tập kiểm định, và epoch tốt nhất cũng được chọn trên chính tập này. Vì vậy số liệu tuyệt đối có thể lạc quan hơn thực tế. Hạn chế này áp dụng như nhau cho cả hai mô hình, nên không làm lệch phép so sánh giữa chúng.

– Phần triển khai trên dữ liệu thực tế dùng bốn bộ dữ liệu công khai khác nguồn: CVC-ClinicDB (612 ảnh), CVC-ColonDB (380 ảnh), ETIS-Larib (196 ảnh) và phần ảnh đơn của PolypGen (1.537 ảnh từ 6 trung tâm, trong đó 125 ảnh không có polyp). Các checkpoint được dùng nguyên, không huấn luyện lại. Đây là dữ liệu công khai, không phải dữ liệu thu tại bệnh viện.

Phạm vi mô hình và thực nghiệm:

– Chỉ có một lớp đối tượng là polyp.

– Hai mô hình chỉ khác nhau ở tầng 10. Mọi siêu tham số khác giữ giống nhau.

– Mỗi mô hình được huấn luyện 100 epoch, ảnh vào 640×640, với 20 seed (seed 0–19).

– PraNet [9] và U-Net [10] được huấn luyện trên cùng dữ liệu, chỉ để đối chiếu chỉ số theo điểm ảnh.

Phạm vi ứng dụng:

– Ứng dụng PolypVision gồm dịch vụ suy luận (FastAPI), website (Spring Boot) và ứng dụng di động (Flutter).

– Ứng dụng xử lý ảnh tĩnh do người dùng tải lên, chưa xử lý video trực tiếp từ máy nội soi.

– Ứng dụng phục vụ nghiên cứu và minh họa, **không phải thiết bị y tế** và không thay thế chẩn đoán của bác sĩ.

4. PHƯƠNG PHÁP NGHIÊN CỨU

Nghiên cứu lý thuyết: đọc và tổng hợp tài liệu về phân đoạn polyp, họ YOLO, cơ chế chú ý, mô hình không gian trạng thái, Mamba và VMamba. Kết quả được trình bày ở Chương 1 và Chương 2.

Nghiên cứu thực nghiệm có đối chứng:

– Mô hình gốc YOLO26s-seg là nhóm đối chứng. Mô hình tích hợp là nhóm thử nghiệm.

– Hai mô hình dùng chung dữ liệu, cách tiền xử lý, siêu tham số và danh sách seed. Vì vậy, khác biệt có hệ thống giữa hai mô hình chỉ đến từ tầng 10.

– Kết quả được ghép cặp theo seed. Chênh lệch được kiểm định bằng kiểm định t ghép cặp và kiểm định Wilcoxon, kèm khoảng tin cậy 95% và cỡ hiệu ứng.

– Thí nghiệm tách thành phần (ablation) tắt lần lượt từng nhánh của khối đề xuất để đo đóng góp của từng nhánh.

Các độ đo sử dụng (định nghĩa ở mục 2.7):

Nhóm	**Độ đo**
Chất lượng phân đoạn theo đối tượng	mask mAP50–95 (độ đo chính), mask mAP50, Precision, Recall, F1
Chất lượng phân đoạn theo điểm ảnh	Dice, IoU, Precision, Recall theo điểm ảnh
Chi phí	Số tham số, chi phí tính toán (FLOPs), thời gian suy luận trên CPU và GPU, thời gian huấn luyện
Lỗi phát hiện	Số dương tính đúng (TP), dương tính giả (FP), âm tính giả (FN)

Nghiên cứu ứng dụng: xây dựng PolypVision, tích hợp hai checkpoint đã huấn luyện, cho phép tải ảnh, xem mặt nạ, độ tin cậy và so sánh hai mô hình trên cùng một ảnh.

5. ĐÓNG GÓP CỦA ĐỀ TÀI

- **Về mô hình:** thiết kế và cài đặt khối C2TSVMamba thay khối C2PSA ở tầng 10 của YOLO26s-seg. Khối này kết hợp nhánh SS2D quét bốn hướng với nhánh trích đặc trưng hình dạng đa tỉ lệ và có hướng, nối với nhau qua một cổng điều khiển. Bản cài đặt chỉ dùng PyTorch, không cần biên dịch kernel CUDA riêng.
- **Về kỹ thuật:** khi lưu checkpoint ở dạng FP16 (mặc định của Ultralytics), một thống kê của lớp chuẩn hóa lô trong khối mới vượt giới hạn của FP16 và bị tràn số. Đề tài phát hiện vấn đề này và chuyển sang lưu FP32, nhờ đó trọng số của khối mới được lưu và nạp lại đúng.
- **Về đánh giá:**
  + So sánh hai mô hình trên nhiều seed ghép cặp, có kiểm định thống kê, khoảng tin cậy và cỡ hiệu ứng, thay vì chỉ báo cáo một lần chạy tốt nhất.
  + Bổ sung thí nghiệm tách thành phần để xem đóng góp của từng nhánh trong khối đề xuất.
  + Đánh giá trên bốn bộ dữ liệu ngoài để kiểm tra khả năng tổng quát hóa.
  + Mọi con số đều truy được về tệp kết quả gốc và script tính lại.
- **Về ứng dụng:** xây dựng PolypVision gồm dịch vụ suy luận dùng chung cho Web và Mobile, có chức năng so sánh hai mô hình trên cùng ảnh.

Đề tài không khẳng định mô hình đề xuất tốt hơn mô hình gốc khi chưa có bằng chứng thống kê. Kết quả cụ thể và mức độ tin cậy được trình bày ở Chương 6.

6. KẾ HOẠCH THỰC HIỆN VÀ PHÂN CÔNG CÔNG VIỆC

6.1. Kế hoạch theo tuần

Đề tài thực hiện trong 12 tuần, từ 17/08/2026 đến 08/11/2026. Nội dung từng tuần lấy theo đề cương đã duyệt. Cột “Người phụ trách”, “Sản phẩm” và “Trạng thái” do nhóm điền.

Bảng 0.1: Kế hoạch thực hiện theo tuần

Tuần	Thời gian	Nội dung công việc	Người phụ trách	Sản phẩm	Trạng thái
1	17/08 – 23/08	Xác định lý do chọn đề tài, khảo sát các công trình liên quan. Lập kế hoạch và phân công	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
2	24/08 – 30/08	Thu thập, phân tích bộ dữ liệu ảnh polyp đại tràng (Kaggle và nguồn công khai)	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
3	31/08 – 06/09	Làm sạch, chuyển nhãn, tiền xử lý dữ liệu cho 1.000 ảnh polyp và ảnh nền	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
4	07/09 – 13/09	Mô tả bài toán, thiết lập môi trường huấn luyện, chọn ngôn ngữ và framework	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
5	14/09 – 20/09	Cài đặt và huấn luyện mô hình gốc YOLO26-seg	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
6	21/09 – 27/09	Cài đặt và huấn luyện các mô hình so sánh	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
7	28/09 – 04/10	Cài đặt thành phần cải tiến: tích hợp VMamba vào YOLO26-seg	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
8	05/10 – 11/10	Thực nghiệm và đánh giá bằng Dice, IoU, Precision, Recall, F1, mAP50, mAP50–95, tốc độ suy luận, số tham số, chi phí tính toán	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
9	12/10 – 18/10	Đánh giá trên dữ liệu thực tế. So sánh và chọn mô hình	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
10	19/10 – 25/10	Xây dựng backend, tổ chức lưu trữ kết quả dự đoán	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
11	26/10 – 01/11	Cài đặt chức năng, thiết kế giao diện Web/App: tải ảnh, hiển thị mặt nạ polyp và độ tin cậy	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
12	02/11 – 08/11	Hoàn chỉnh báo cáo, thiết kế slide, nộp báo cáo	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]

6.2. Phân công công việc

Bảng 0.2: Phân công nhiệm vụ trong nhóm

Thành viên	MSSV	Lớp	Vai trò	Nhiệm vụ chính	Chương phụ trách viết	Mức đóng góp (%)
Lê Đức Lương	2001230490	14DHTH09	[ĐIỀN, ví dụ: Nhóm trưởng]	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
Phùng Tuấn Huy	2001230312	14DHTH13	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
Trần Mạnh Toàn	2001230830	14DHTH09	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]

Ghi **C** (chính) hoặc **H** (hỗ trợ) vào từng ô.

Bảng 0.3: Phân công theo hạng mục công việc

Hạng mục	Lê Đức Lương	Phùng Tuấn Huy	Trần Mạnh Toàn
Khảo sát tài liệu, viết Chương 1	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
Cơ sở lý thuyết, viết Chương 2	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
Thu thập và tiền xử lý dữ liệu, viết Chương 4	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
Thiết kế và cài đặt khối C2TSVMamba, viết Chương 3	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
Huấn luyện Baseline và TSVM trên Kaggle	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
Huấn luyện PraNet, U-Net	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
Thí nghiệm tách thành phần (ablation)	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
Phân tích thống kê, vẽ hình, viết Chương 6	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
Dịch vụ suy luận FastAPI	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
Website Spring Boot, MongoDB	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
Ứng dụng di động Flutter	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
Viết Chương 5	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
Mở đầu, Kết luận, định dạng báo cáo	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
Slide và chuẩn bị bảo vệ	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]

6.3. Cách phối hợp và theo dõi tiến độ

- Nhóm gặp giảng viên hướng dẫn ít nhất một lần mỗi tuần, theo yêu cầu của đề cương. Trước mỗi buổi gặp, các phần việc được gộp vào cùng một tài liệu và một kho mã.
- Mã nguồn, cấu hình và kết quả huấn luyện được quản lý bằng Git **[ĐIỀN: tên kho/nền tảng nếu muốn nêu]**.
- Các lượt huấn luyện được chia cho nhiều tài khoản Kaggle để chạy song song, mỗi lượt lưu riêng tệp `args.yaml`, `results.csv` và trọng số.
- [ĐIỀN: công cụ trao đổi trong nhóm, ví dụ Zalo, Google Drive; tần suất họp nhóm.]

7. BỐ CỤC BÁO CÁO

Ngoài phần Mở đầu và Kết luận, báo cáo gồm sáu chương:

- **Chương 1. Tổng quan:** bối cảnh bài toán, các bộ dữ liệu công khai, các hướng nghiên cứu phân đoạn polyp từ CNN, Transformer đến Mamba, và khoảng trống mà đề tài hướng tới.
- **Chương 2. Cơ sở lý thuyết:** các bài toán thị giác máy tính, mạng tích chập, họ YOLO và YOLO26-seg, cơ chế chú ý, mô hình không gian trạng thái, Mamba và VMamba, các độ đo và phương pháp kiểm định thống kê.
- **Chương 3. Mô hình đề xuất:** kiến trúc khối C2TSVMamba, thuật toán quét song song, cách khởi tạo, quy trình huấn luyện và suy luận.
- **Chương 4. Dữ liệu và tiền xử lý:** nguồn dữ liệu, cách xây dựng bộ BG20, chuyển mặt nạ sang nhãn đa giác và các bước tiền xử lý.
- **Chương 5. Cài đặt và ứng dụng:** môi trường thực nghiệm, cài đặt hai mô hình, quy trình huấn luyện trên Kaggle và ứng dụng PolypVision.
- **Chương 6. Thực nghiệm và đánh giá:** kết quả so sánh nhiều seed, thí nghiệm tách thành phần, chi phí tính toán, so sánh với PraNet và U-Net, kết quả trên dữ liệu ngoài.

Phần **Kết luận** trả lời bốn câu hỏi nghiên cứu, nêu hạn chế và hướng phát triển. **Phụ lục** gồm kết quả từng seed, cấu hình huấn luyện, tổ chức mã nguồn và hướng dẫn chạy lại.

CHƯƠNG 1: TỔNG QUAN

Chương này trình bày bối cảnh của bài toán phân đoạn polyp, các bộ dữ liệu công khai và các hướng nghiên cứu chính. Thứ tự trình bày đi từ mạng tích chập, qua Transformer, đến mô hình không gian trạng thái Mamba. Cuối chương, đề tài được đặt cạnh các công trình gần nhất để chỉ ra khoảng trống mà khóa luận hướng tới.

1.1. BỐI CẢNH Y KHOA

1.1.1. Ung thư đại trực tràng và polyp

Ung thư đại trực tràng đứng thứ ba về số ca mắc mới và thứ hai về số ca tử vong trong các bệnh ung thư năm 2022 [1]. Phần lớn các ca phát triển qua nhiều năm từ polyp, tức những khối mô nhô lên từ niêm mạc ruột. Không phải polyp nào cũng hóa ác, nhưng u tuyến (adenoma) là loại có nguy cơ cao. Phát hiện và cắt bỏ polyp sớm khi nội soi giúp giảm nguy cơ tiến triển thành ung thư [2].

1.1.2. Giới hạn của nội soi

Nội soi đại tràng là phương pháp chuẩn để tìm polyp, nhưng kết quả phụ thuộc vào kinh nghiệm, sự tập trung của bác sĩ và chất lượng chuẩn bị ruột. Hai nhóm bằng chứng cho thấy giới hạn này:

- **Tỉ lệ bỏ sót cao.** Khi bệnh nhân được nội soi hai lần liên tiếp, lần đầu bỏ sót khoảng 26% u tuyến; với u tuyến phẳng, tỉ lệ này khoảng 34% [3].
- **Phát hiện nhiều hơn thì giảm nguy cơ.** Theo dõi hơn 300 nghìn ca nội soi cho thấy tỉ lệ phát hiện u tuyến của bác sĩ tăng 1% thì nguy cơ ung thư xuất hiện giữa hai lần nội soi giảm khoảng 3% [4].

Vì vậy, các hệ thống máy tính hỗ trợ phát hiện và khoanh vùng polyp được xem là công cụ có ích. Những hệ thống như vậy không thay bác sĩ ra quyết định. Vai trò của chúng là nhắc bác sĩ chú ý những vùng nghi ngờ có thể bị bỏ qua.

1.2. BÀI TOÁN PHÂN ĐOẠN POLYP

1.2.1. Phát biểu bài toán

Đầu vào là một ảnh nội soi màu. Đầu ra là mặt nạ nhị phân cùng kích thước ảnh: điểm ảnh thuộc polyp mang giá trị 1, còn lại mang giá trị 0. Đề tài giải bài toán ở dạng **phân đoạn thực thể** (instance segmentation): mỗi polyp có hộp bao,

độ tin cậy và mặt nạ riêng. Các mặt nạ có thể gộp lại thành một mặt nạ chung khi cần so sánh với các phương pháp phân đoạn ngữ nghĩa. Mục 2.1 phân biệt rõ các dạng bài toán này.

So với chỉ vẽ hộp bao, mặt nạ cho biết chính xác hình dạng và đường biên của tổn thương. Thông tin này có ích khi bác sĩ cần ước lượng kích thước hoặc quan sát ranh giới polyp với niêm mạc lành.

1.2.2. Các khó khăn đặc thù

Khảo sát về phân đoạn polyp [11] nêu một số khó khăn chung:

- **Tương phản thấp.** Màu và kết cấu của polyp gần giống niêm mạc xung quanh, nhất là với polyp phẳng. Đường biên vì thế mờ.
- **Nhiễu do điều kiện chụp.** Ảnh có vùng phản quang của đèn nội soi, bọt, dịch, nếp gấp ruột và chuyển động gây nhòe.
- **Kích thước và hình dạng đa dạng.** Polyp có thể chỉ chiếm vài phần trăm khung hình hoặc gần như toàn bộ ảnh, có dạng tròn, dạng cuống hoặc dạng phẳng.
- **Khác biệt giữa các trung tâm.** Mỗi bệnh viện dùng thiết bị và quy trình khác nhau. Mô hình học tốt trên dữ liệu của một nơi có thể kém đi khi gặp ảnh từ nơi khác [12].
- **Dữ liệu có nhãn ít.** Vẽ mặt nạ cho từng điểm ảnh cần chuyên gia và tốn nhiều thời gian, nên các bộ dữ liệu thường chỉ có vài trăm đến vài nghìn ảnh.

Những khó khăn này giải thích vì sao một mô hình phân đoạn polyp cần hai năng lực cùng lúc: nhìn chi tiết cục bộ để bám sát đường biên, và nắm ngữ cảnh toàn ảnh để phân biệt polyp với các vùng trông giống polyp.

1.3. CÁC BỘ DỮ LIỆU CÔNG KHAI

Bảng 1.1: Một số bộ dữ liệu phân đoạn polyp công khai

Bộ dữ liệu	**Số ảnh**	**Đặc điểm**	**Vai trò trong đề tài**
Kvasir-SEG [5]	1.000	Ảnh polyp kèm mặt nạ do bác sĩ vẽ và chuyên gia duyệt; độ phân giải khác nhau	Nguồn ảnh polyp chính

Bộ dữ liệu	Số ảnh	Đặc điểm	Vai trò trong đề tài
Kvasir [8]	Bản v2: 8 lớp, mỗi lớp 1.000 ảnh	Ảnh nội soi tiêu hóa phân loại theo mốc giải phẫu, tổn thương, thủ thuật	Lấy 200 ảnh lớp *normal-cecum* làm ảnh nền
CVC-ClinicDB [13]	612	Khung hình cắt từ video nội soi, kèm mặt nạ	Dữ liệu ngoài, kiểm tra trên dữ liệu thực tế
CVC-ColonDB [14]	380 (bản dùng trong các benchmark phân đoạn từ PraNet [9]; bài gốc mô tả 300 ảnh)	Khung hình cắt từ 15 đoạn video nội soi	Dữ liệu ngoài
ETIS-Larib [15]	196	Nhiều polyp nhỏ, khó phát hiện	Dữ liệu ngoài
PolypGen [12]	1.537 ảnh đơn từ 6 trung tâm (1.412 có polyp, 125 không polyp)	Thu ở nhiều bệnh viện và thiết bị, dùng để đánh giá khả năng tổng quát hóa	Dữ liệu ngoài, đánh giá theo từng trung tâm

Trong đề tài, Kvasir-SEG được chọn làm dữ liệu huấn luyện vì có mặt nạ chất lượng, được dùng phổ biến và có sẵn danh sách chia huấn luyện/kiểm định do nhóm tác giả cung cấp. Ba bộ CVC-ClinicDB, CVC-ColonDB và ETIS-Larib là ảnh cắt từ video, khác nguồn với Kvasir-SEG. Cùng với PolypGen, chúng được dùng để kiểm tra mô hình trên dữ liệu chưa thấy khi huấn luyện. Kết quả trình bày ở mục 6.7.

1.4. CÁC HƯỚNG NGHIÊN CỨU PHÂN ĐOẠN POLYP

1.4.1. Mạng tích chập dạng mã hóa – giải mã

U-Net [10] là kiến trúc nền của phân đoạn ảnh y tế. Mạng gồm hai nửa đối xứng:

- nửa mã hóa giảm dần kích thước và tăng dần số kênh để trích đặc trưng ngữ nghĩa;
- nửa giải mã phóng to dần để khôi phục kích thước ảnh;
- các kết nối tắt (skip connection) chuyển chi tiết không gian từ nửa mã hóa sang nửa giải mã.

Nhờ cấu trúc này, U-Net huấn luyện được với lượng ảnh nhỏ.

PraNet [9] là mạng thiết kế riêng cho polyp. Mạng trước hết dự đoán một bản đồ vị trí thô, sau đó dùng *chú ý ngược* (reverse attention): xóa vùng đã chắc chắn là polyp để tập trung tinh chỉnh vùng biên còn mơ hồ. Cách làm này đặt trọng tâm vào đường biên, đúng với khó khăn chính của bài toán.

Điểm mạnh của nhóm này là bám tốt chi tiết cục bộ. Điểm yếu là mỗi lớp tích chập chỉ nhìn một vùng nhỏ. Muốn có ngữ cảnh toàn ảnh, mạng phải xếp chồng nhiều lớp và giảm kích thước nhiều lần, đổi lại mất bớt chi tiết.

1.4.2. Transformer và mô hình lai

Transformer dùng cơ chế tự chú ý (self-attention): mỗi vị trí so sánh trực tiếp với mọi vị trí khác. Nhờ vậy mô hình có ngữ cảnh toàn cục ngay trong một lớp. Một số công trình tiêu biểu cho polyp:

- **Polyp-PVT** [16] dùng Pyramid Vision Transformer làm bộ mã hóa, thêm các mô-đun hợp nhất đặc trưng và tách polyp lẫn vào nền.
- **ColonFormer** [17] dùng bộ mã hóa Transformer phân cấp, bộ giải mã nhiều mức và mô-đun tinh chỉnh biên. Đây là công trình của nhóm tác giả Việt Nam.

Hạn chế chung là chi phí. Với N vị trí, attention phải tính N^2 cặp tương quan. Chi phí này tăng rất nhanh khi ảnh lớn hoặc khi áp dụng ở bản đồ đặc trưng có độ phân giải cao (mục 2.5).

1.4.3. Phát hiện và phân đoạn một giai đoạn: họ YOLO

YOLO [18] xem phát hiện vật thể là một phép hồi quy: ảnh đi qua mạng một lần và cho ra trực tiếp hộp bao và lớp. Cách làm này nhanh hơn nhiều so với các mô hình hai giai đoạn, nên YOLO được dùng rộng rãi khi cần xử lý thời gian thực. Các phiên bản sau bổ sung nhánh dự đoán mặt nạ theo cách của YOLACT [19] để làm phân đoạn thực thể. YOLOv10 [20] đưa ra cách huấn luyện hai đầu để bỏ bước lọc trùng NMS, cùng khối chú ý một phần PSA.

YOLO26 [6] là phiên bản được Ultralytics công bố năm 2026 (bản tiền ấn phẩm) và được dùng trong đề tài. Mô hình kế thừa hai cơ chế trên của YOLOv10: suy luận end-to-end không cần NMS và khối C2PSA ở cuối backbone, đồng thời có đầu riêng cho phân đoạn. Chi tiết kiến trúc ở mục 2.4.

Theo khảo sát của nhóm, các công trình phân đoạn polyp phần lớn dùng mạng dạng U (mục 1.4.1 và 1.4.5). Họ YOLO đã được dùng cho polyp, nhưng chủ yếu

ở bài toán phát hiện bằng hộp bao, ví dụ YOLOv4 cải tiến cho phát hiện polyp thời gian thực [21]. Điều này tạo một khoảng trống: dùng mô hình phân đoạn thực thể một giai đoạn, vốn nhanh và cho cả hộp bao lẫn độ tin cậy, cho bài toán polyp.

1.4.4. Mô hình không gian trạng thái và Mamba

Mô hình không gian trạng thái (State Space Model, SSM) xuất phát từ lý thuyết điều khiển. Mô hình duy trì một trạng thái ẩn và cập nhật trạng thái đó theo từng phần tử của chuỗi.

- **S4** [22] đưa ra cách tham số hóa giúp SSM xử lý hiệu quả chuỗi rất dài.
- **Mamba** [23] cho các tham số của SSM phụ thuộc vào đầu vào. Nhờ cơ chế chọn lọc này, mô hình tự quyết định giữ hay quên thông tin ở từng bước. Mamba có chi phí tăng gần tuyến tính theo độ dài chuỗi khi dùng thuật toán quét tối ưu cho GPU.

Ảnh là dữ liệu hai chiều, không có thứ tự tự nhiên như chuỗi. Để đưa Mamba vào ảnh, **VMamba** [7] đề xuất mô-đun SS2D, quét bản đồ đặc trưng theo bốn hướng (trái sang phải, phải sang trái, trên xuống dưới, dưới lên trên) rồi gộp kết quả. Nhờ vậy, mỗi vị trí nhận được thông tin từ cả bốn phía. Đề tài dùng SS2D làm nhánh ngữ cảnh toàn cục.

1.4.5. Mamba cho ảnh y tế và polyp

Từ năm 2024, nhiều công trình đưa Mamba vào phân đoạn ảnh y tế:

- **U-Mamba** [24] chèn khối lai CNN–SSM vào khung nnU-Net.
- **Swin-UMamba** [25] cho thấy bộ mã hóa Mamba được khởi tạo từ trọng số tiền huấn luyện trên ImageNet cải thiện rõ kết quả phân đoạn y tế. Điểm này liên quan trực tiếp tới đề tài, vì nhánh SS2D trong khối đề xuất được khởi tạo mới, không có trọng số tiền huấn luyện.

Riêng với polyp:

- **Polyp-Mamba** [26] kết hợp Visual Mamba với mô-đun ngữ nghĩa theo tỉ lệ.
- **PolyMamba** [27] thêm thông tin vị trí có sẵn để bù cho kiểu quét cố định của Mamba, và dùng lọc tần số cao có cổng để làm rõ biên polyp.
- **Topo-VM-UNetV2** [28] đưa thông tin tô-pô vào mạng VM-UNetV2: bản đồ chú ý tô-pô được tính bằng phương pháp đồng điều bền vững (persistent homology), rồi đưa vào mô-đun hợp nhất đặc trưng SDI qua quy trình huấn luyện hai giai đoạn.

Hai nhận xét rút ra từ nhóm công trình này:

1. **Hình dạng và đường biên là manh mối quan trọng.** Các công trình [27], [28] bổ sung thông tin biên, tần số cao hoặc tô-pô vào các mạng dùng Mamba. Đây là cơ sở để khối đề xuất có thêm nhánh hình dạng và hướng.
2. **Gần như tất cả đều là mạng dạng U** cho phân đoạn ngữ nghĩa. Rất ít công trình đưa Mamba vào mô hình phân đoạn thực thể một giai đoạn như YOLO-seg.

1.4.6. Mamba trong họ YOLO

Hai công trình gần nhất với đề tài là:

– **Mamba YOLO** [29] thay các khối của YOLO bằng khối SSM (ODSS) để thêm ngữ cảnh toàn cục. Công trình thử trên COCO và PASCAL VOC cho bài toán phát hiện vật thể. Phần lớn backbone được thay thế.

– **MambaPSA** [30] thay đúng khối C2PSA ở cuối backbone YOLO26 bằng một khối Mamba nhẹ, đồng thời thêm mô-đun Mamba hai chiều (BiViM) ở neck. Công trình thử trên PASCAL VOC cho bài toán phát hiện vật thể, với YOLO26 cỡ *nano*. Theo bài báo, so với YOLO26-nano gốc, toàn mô hình giảm 2,9% số tham số, giảm 12,1% FLOPs và tăng tốc độ trên CPU, trong khi mAP50:95 gần như giữ nguyên (giảm 0,1 điểm). Bài báo là bản tiền ấn phẩm. Mọi kết quả chỉ từ **một seed huấn luyện**, và chính các tác giả lưu ý rằng dao động giữa các seed có thể ngang mức chênh lệch được báo cáo.

1.5. TỔNG HỢP VÀ KHOẢNG TRỐNG NGHIÊN CỨU

1.5.1. Bảng tổng hợp các hướng tiếp cận

Bảng 1.2: Tổng hợp các nhóm phương pháp phân đoạn polyp

Nhóm	**Ví dụ**	**Điểm mạnh**	**Hạn chế**
CNN dạng U	U-Net [10], PraNet [9]	Bám chi tiết cục bộ và đường biên tốt; ít dữ liệu vẫn học được	Vùng nhìn của mỗi lớp nhỏ, ngữ cảnh toàn cục hạn chế
Transformer, mô hình lai	Polyp-PVT [16], ColonFormer [17]	Có ngữ cảnh toàn cục ngay trong một lớp	Chi phí tăng theo bình phương số vị trí; cần nhiều dữ liệu hoặc tiền huấn luyện

Nhóm	Ví dụ	Điểm mạnh	Hạn chế
YOLO một giai đoạn	YOLO [18], YOLO26 [6]	Nhanh; cho cả hộp bao, độ tin cậy và mặt nạ	Ít được nghiên cứu cho phân đoạn polyp; mặt nạ có độ phân giải thấp hơn ảnh
Mamba dạng U	U-Mamba [24], Polyp-Mamba [26], PolyMamba [27], Topo-VM-UNetV2 [28]	Ngữ cảnh toàn cục với chi phí thấp hơn attention; một số có thêm thông tin biên, tô-pô	Chủ yếu là phân đoạn ngữ nghĩa; quét theo thứ tự cố định
Mamba trong YOLO	Mamba YOLO [29], MambaPSA [30]	Thêm ngữ cảnh toàn cục vào mô hình nhanh	Đến nay chỉ thử cho phát hiện vật thể trên dữ liệu tổng quát

1.5.2. Vị trí của đề tài

Bảng 1.3: So sánh đề tài với các công trình gần nhất

Tiêu chí	Mamba YOLO [29]	MambaPSA [30]	Topo-VM-UNetV2 [28]	Đề tài
Mô hình nền	YOLO	YOLO26 cỡ *nano*	VM-UNetV2 (dạng U)	**YOLO26s-seg**
Bài toán	Phát hiện vật thể	Phát hiện vật thể	Phân đoạn ngữ nghĩa	**Phân đoạn thực thể**
Dữ liệu	COCO, VOC	PASCAL VOC	Ảnh polyp	**Ảnh polyp (Kvasir-SEG + ảnh nền)**
Phạm vi thay đổi	Thay phần lớn backbone	Thay C2PSA, thêm khối ở neck	Thêm bản đồ chú ý tô-pô	**Chỉ thay C2PSA ở tầng 10**
Khối Mamba	Khối SSM riêng	Mamba nhẹ, BiViM hai chiều	VMamba	**SS2D bốn hướng + nhánh hình dạng có hướng + cổng**

Tiêu chí	Mamba YOLO [29]	MambaPSA [30]	Topo-VM-UNetV2 [28]	Đề tài
Cách đánh giá	Báo một kết quả, không nêu số lần chạy hay độ lệch chuẩn	Một seed (tác giả tự nêu là hạn chế)	5 seed, lấy trung bình	**20 seed ghép cặp, có kiểm định thống kê**

Từ các bảng trên, khoảng trống mà đề tài hướng tới gồm ba điểm:

1. Theo khảo sát của nhóm đến tháng 10/2026, chưa thấy công trình nào đưa khối VMamba vào **YOLO26-seg** cho **phân đoạn thực thể polyp**.
2. Các công trình đưa Mamba vào YOLO chưa kết hợp thông tin **hình dạng và đường biên**, trong khi nhóm công trình Mamba cho polyp cho thấy thông tin này quan trọng.
3. Các công trình gần nhất hoặc chỉ báo cáo một lần huấn luyện [30], hoặc lấy trung bình vài seed nhưng không kiểm định thống kê ghép cặp [28]. Với dữ liệu nhỏ như Kvasir-SEG, dao động giữa các lần huấn luyện có thể lớn hơn mức chênh lệch giữa hai mô hình [31]. Cần đánh giá lặp lại nhiều seed, ghép cặp và có kiểm định để kết luận đáng tin.

Đề tài không nhắm tới việc vượt các mô hình tốt nhất hiện nay. Mục tiêu là trả lời câu hỏi có kiểm soát: **chỉ thay một tầng của YOLO26s-seg bằng khối VMamba có nhận biết hình dạng thì kết quả thay đổi thế nào, với độ tin cậy thống kê ra sao, và tốn thêm bao nhiêu chi phí**.

Cũng cần nói rõ một khác biệt với MambaPSA. Khối đề xuất trong đề tài **tăng** số tham số của tầng 10 (khoảng 1,81 triệu so với 0,99 triệu của C2PSA) và làm suy luận trên CPU **chậm hơn**. Đề tài không nhắm tới làm mô hình nhẹ hơn như MambaPSA.

1.6. TÌNH HÌNH NGHIÊN CỨU TRONG NƯỚC

Ở trong nước, nhóm nghiên cứu của Trường Đại học Bách khoa Hà Nội phối hợp với Viện Nghiên cứu và Đào tạo Tiêu hóa, Gan mật có nhiều công trình về polyp:

– **ColonFormer** [17], đã nêu ở mục 1.4.2.

– **NeoUNet** [32] mở rộng bài toán thành vừa phân đoạn polyp vừa phân biệt polyp tân sinh với polyp không tân sinh, kèm bộ dữ liệu NeoPolyp. Phần công

khai của bộ này, BKAI-IGH NeoPolyp-Small, có 1.200 ảnh nội soi chụp bằng ánh sáng trắng và FICE.

Về họ YOLO, Đào Việt Hằng và cộng sự [33] đánh giá YOLOv8 phát hiện polyp trên 30 video nội soi thu tại Hà Nội. Mô hình phát hiện được 65/68 polyp, nhưng còn nhiều báo nhầm khi xét theo từng khung hình (Precision 0,48 ở ngưỡng IoU 0,5). Công trình này dừng ở mức phát hiện bằng hộp bao, không phân đoạn.

Trong phạm vi khảo sát của nhóm (arXiv, Crossref và kho tài liệu của một số trường), chưa thấy công trình trong nước nào đưa khối Mamba vào mô hình họ YOLO cho bài toán polyp.

1.7. KẾT CHƯƠNG

Chương 1 đã trình bày:

- bối cảnh y khoa: ung thư đại trực tràng phổ biến, polyp vẫn bị bỏ sót khi nội soi, nên công cụ hỗ trợ khoanh vùng có giá trị;
- bài toán phân đoạn polyp và các khó khăn đặc thù: biên mờ, nhiễu, kích thước đa dạng, ít dữ liệu;
- các bộ dữ liệu công khai, trong đó Kvasir-SEG là nguồn chính của đề tài;
- các hướng nghiên cứu từ CNN, Transformer đến Mamba, và các công trình đưa Mamba vào YOLO.

Khoảng trống được xác định là: chưa có công trình đưa VMamba kèm thông tin hình dạng vào YOLO26-seg cho phân đoạn polyp, với cách đánh giá lặp lại nhiều seed. Chương 2 trình bày cơ sở lý thuyết cần thiết, gồm YOLO26-seg, cơ chế chú ý, mô hình không gian trạng thái, VMamba và các độ đo.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

Chương này trình bày các kiến thức nền mà Chương 3 (mô hình đề xuất) và Chương 6 (đánh giá) sử dụng. Nội dung đi theo thứ tự:

1. các bài toán thị giác máy tính: phân loại, phát hiện, phân đoạn;
2. mạng nơ-ron tích chập;
3. họ mô hình YOLO và cách YOLO tạo mặt nạ;
4. YOLO26-seg: kiến trúc, các bước của thuật toán, ưu và nhược điểm;
5. cơ chế chú ý và chi phí của nó;
6. mô hình không gian trạng thái, Mamba và VMamba: nguyên lý, các bước, ưu và nhược điểm;
7. các độ đo đánh giá;
8. phương pháp so sánh hai mô hình qua nhiều lần huấn luyện.

2.1. CÁC BÀI TOÁN THỊ GIÁC MÁY TÍNH

Ba bài toán cơ bản khác nhau ở **mức chi tiết của đầu ra**:

Bảng 2.1: So sánh ba bài toán thị giác cơ bản

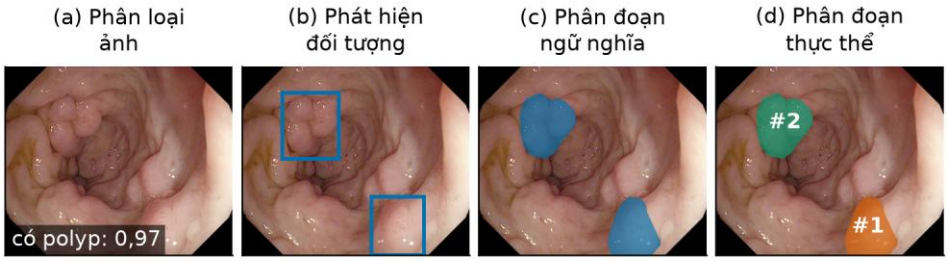
Bài toán	**Câu hỏi trả lời**	**Đầu ra**	**Ví dụ với ảnh nội soi**
Phân loại ảnh	Ảnh có gì?	Một nhãn (hoặc xác suất) cho cả ảnh	"Ảnh có polyp" với xác suất 0,97
Phát hiện đối tượng	Đối tượng ở đâu?	Danh sách hộp bao, mỗi hộp có lớp và độ tin cậy	Hộp (x_1, y_1, x_2, y_2) quanh polyp, độ tin cậy 0,91
Phân đoạn ảnh	Điểm ảnh nào thuộc đối tượng?	Mặt nạ cùng kích thước ảnh	Các điểm ảnh thuộc polyp mang giá trị 1

Phân đoạn ảnh lại chia thành hai dạng:

– **Phân đoạn ngữ nghĩa** gán mỗi điểm ảnh vào một lớp, không phân biệt các đối tượng cùng lớp. Nếu ảnh có hai polyp, đầu ra chỉ là một mặt nạ chung. U-Net [10] và PraNet [9] thuộc dạng này.

– **Phân đoạn thực thể** tách riêng từng đối tượng: mỗi polyp có hộp bao, độ tin cậy và mặt nạ riêng. YOLO26-seg thuộc dạng này.

Ví dụ minh họa. Hình 2.1 đặt bốn kiểu đầu ra cạnh nhau cho cùng một ảnh Kvasir-SEG.



Hình 2.1: Bốn kiểu đầu ra trên cùng một ảnh tập kiểm định có hai polyp: (a) phân loại ảnh, (b) phát hiện đối tượng bằng hộp bao, (c) phân đoạn ngữ nghĩa (hai polyp chung một mặt nạ), (d) phân đoạn thực thể (mỗi polyp một mặt nạ riêng). Hộp bao và mặt nạ vẽ từ nhãn đa giác của ảnh; xác suất ở ô (a) là số minh họa, không phải đầu ra mô hình.

Nguồn: ảnh từ Kvasir-SEG [5]; hình do nhóm vẽ.

Ba bài toán xếp theo độ chi tiết tăng dần, đồng thời chi phí gán nhãn cũng tăng dần: nhãn phân loại chỉ cần một từ, nhãn hộp cần bốn tọa độ, còn nhãn mặt nạ cần vẽ đường biên từng điểm. Đề tài chọn phân đoạn thực thể vì đầu ra vừa có mặt nạ để quan sát đường biên, vừa có hộp bao và độ tin cậy để hiển thị trong ứng dụng.

2.2. MẠNG NƠ-RON TÍCH CHẬP

2.2.1. Phép tích chập

Mạng nơ-ron tích chập (CNN) là nền của hầu hết mô hình thị giác. Phép tích chập trượt một bộ lọc nhỏ trên ảnh. Tại mỗi vị trí, giá trị đầu ra là tổng các tích giữa bộ lọc và vùng ảnh bên dưới:

$$y(i,j) = \sum_u \sum_v w\,(u,v)\, x(i+u,\ j+v) + b \qquad (2.1)$$

Ví dụ. Vùng ảnh 3×3 và bộ lọc phát hiện biên dọc:

$$x = \begin{bmatrix} 10 & 10 & 80 \\ 10 & 10 & 80 \\ 10 & 10 & 80 \end{bmatrix}, \qquad w = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

Đầu ra tại tâm là $3 \times (-10 + 80) = 210$. (Nói chặt chẽ, công thức (2.1) là phép tương quan chéo; các thư viện học sâu vẫn gọi là tích chập.) Giá trị lớn này báo hiệu có biên dọc, chỗ độ sáng đổi đột ngột từ 10 sang 80. Ở vùng đồng nhất, đầu ra bằng 0. Trong CNN, các bộ lọc không được thiết kế bằng tay như ví dụ mà được học từ dữ liệu.

Ba tính chất làm tích chập phù hợp với ảnh:

– **Kết nối cục bộ:** mỗi đầu ra chỉ phụ thuộc một vùng nhỏ.

– **Chia sẻ trọng số:** cùng một bộ lọc dùng cho mọi vị trí, nên số tham số ít. Một lớp tích chập $k \times k$ từ C_{in} sang C_{out} kênh có $k^2 C_{\text{in}} C_{\text{out}}$ trọng số, không phụ thuộc kích thước ảnh.

– **Đồng biến tịnh tiến:** đối tượng dịch chuyển trong ảnh thì đáp ứng của bộ lọc dịch chuyển theo, nên đối tượng ở đâu cũng được nhận ra bằng cùng bộ lọc.

Hai biến thể được dùng nhiều trong đề tài:

– **Tích chập 1×1** chỉ trộn các kênh tại cùng một vị trí, dùng để đổi số kênh.

– **Tích chập theo chiều sâu (depthwise)** dùng một bộ lọc riêng cho từng kênh, không trộn kênh, nên rất ít tham số. Các bộ lọc 1×5 và 5×1 trong nhánh hình dạng của khối đề xuất (Chương 3) là tích chập theo chiều sâu.

2.2.2. Các thành phần khác

– **Chuẩn hóa theo lô (Batch Normalization, BN)** đưa đầu ra mỗi kênh về trung bình 0, phương sai 1 trên một lô, rồi co giãn bằng hai tham số học được. BN giúp huấn luyện ổn định hơn.

– **Hàm kích hoạt SiLU**, $\text{SiLU}(x) = x \cdot \sigma(x)$, tạo tính phi tuyến. Trong YOLO, khối cơ bản `Conv` gồm tích chập, BN và SiLU.

– **Kết nối tắt dạng cộng (residual)** cộng đầu vào vào đầu ra của một khối: $y = x + F(x)$. Nhờ đó gradient đi qua mạng sâu dễ hơn. Khối đề xuất dùng biến thể $y = x + \gamma F(x)$ với γ nhỏ (Chương 3).

– **Trường tiếp nhận (receptive field)** là vùng ảnh đầu vào ảnh hưởng tới một điểm đầu ra. Xếp chồng tích chập 3×3 chỉ mở rộng vùng này dần dần. Đây là lý do CNN thuần khó nắm ngữ cảnh toàn ảnh, và là động cơ để thêm attention hoặc SSM vào mạng.

2.3. HỌ MÔ HÌNH YOLO

2.3.1. Ý tưởng phát hiện một giai đoạn

Các mô hình hai giai đoạn như Mask R-CNN [34] trước hết đề xuất các vùng có thể chứa đối tượng, sau đó phân loại và tinh chỉnh từng vùng. YOLO [18] gộp hai bước thành một: ảnh đi qua mạng một lần, và mỗi ô trên bản đồ đặc trưng dự đoán trực tiếp hộp bao, lớp và độ tin cậy. Nhờ vậy YOLO nhanh, phù hợp xử lý thời gian thực.

Một mô hình YOLO hiện đại gồm ba phần:

- **Backbone:** trích đặc trưng, giảm dần kích thước qua các mức P1 đến P5. Mức Pk có kích thước bằng $1/2^k$ ảnh vào; với ảnh 640×640, P5 có kích thước 20×20.
- **Neck:** trộn đặc trưng giữa các mức. FPN [35] truyền thông tin ngữ nghĩa từ mức sâu xuống mức nông; PAN [36] truyền chi tiết vị trí từ mức nông lên mức sâu.
- **Head:** dự đoán đầu ra tại ba mức P3, P4, P5, tương ứng với đối tượng nhỏ, vừa và lớn.

2.3.2. Gán nhãn và hàm mất mát

Khi huấn luyện, mô hình phải biết điểm dự đoán nào chịu trách nhiệm cho đối tượng thật nào. YOLO26 dùng bộ gán nhãn căn chỉnh nhiệm vụ của TOOD [37]: một điểm được chọn khi vừa có điểm lớp cao, vừa có hộp dự đoán trùng tốt với hộp thật.

Hộp bao được học bằng mất mát CIoU [38]. CIoU mở rộng IoU (mục 2.7) bằng hai hạng tử phạt: khoảng cách giữa hai tâm hộp và độ lệch tỉ lệ khung. Nhờ vậy, ngay cả khi hai hộp chưa chồng nhau, mô hình vẫn có tín hiệu để kéo hộp dự đoán về đúng chỗ.

2.3.3. Bỏ bước lọc trùng NMS

Các YOLO trước đây dự đoán nhiều hộp cho cùng một đối tượng, rồi dùng thuật toán loại bỏ không cực đại (NMS) để giữ hộp tốt nhất. NMS chạy sau mạng, có ngưỡng phải chỉnh tay và làm thời gian suy luận khó dự đoán. YOLOv10 [20] đề xuất huấn luyện hai đầu song song:

- **đầu một-nhiều** gán mỗi đối tượng cho nhiều điểm dự đoán, cho nhiều tín hiệu học;
- **đầu một-một** gán mỗi đối tượng cho đúng một điểm dự đoán, nên đầu ra không bị trùng và không cần NMS.

Khi suy luận chỉ dùng đầu một-một. YOLO26 kế thừa cơ chế này.

2.3.4. Phân đoạn thực thể bằng mặt nạ mẫu

YOLACT [19] đề xuất cách tạo mặt nạ nhanh, được YOLO-seg dùng lại:

1. Một nhánh riêng (Proto) sinh K **mặt nạ mẫu** $P_1, \dots, P_K$ dùng chung cho cả ảnh.
2. Với mỗi đối tượng, đầu dự đoán sinh thêm K **hệ số** $c_1, \dots, c_K$.
3. Mặt nạ của đối tượng là tổ hợp tuyến tính, đưa qua hàm sigmoid rồi cắt theo hộp bao:

$$\widehat{M} = \sigma\left(\sum_{k=1}^{K} c_k P_k\right) \quad (2.2)$$

Ví dụ với $K = 2$ trên lưới 2×2. Giả sử:

$$P_1 = \begin{bmatrix} 4 & 4 \\ -4 & -4 \end{bmatrix} \text{ (nửa trên)}, \qquad P_2 = \begin{bmatrix} 4 & -4 \\ 4 & -4 \end{bmatrix} \text{ (nửa trái)}$$

– Đối tượng A có hệ số $(1,\ 1)$: tổng là $\begin{bmatrix} 8 & 0 \\ 0 & -8 \end{bmatrix}$. Qua sigmoid và quy tắc "thuộc đối tượng khi giá trị lớn hơn 0,5", chỉ ô trên–trái thuộc A (hai ô giá trị 0 cho sigmoid đúng bằng 0,5 nên không được tính).

– Đối tượng B có hệ số $(-1,\ 0)$: tổng là $\begin{bmatrix} -4 & -4 \\ 4 & 4 \end{bmatrix}$, tức nửa dưới thuộc B.

Như vậy, chỉ với vài hệ số, mỗi đối tượng có mặt nạ riêng mà không cần chạy một mạng phân đoạn cho từng đối tượng. YOLO26-seg dùng $K = 32$ mặt nạ mẫu kích thước 160×160 (bằng 1/4 ảnh vào 640×640).

2.4. MÔ HÌNH YOLO26-SEG

2.4.1. Kiến trúc

YOLO26 [6] là phiên bản YOLO được Ultralytics công bố năm 2026 (bản tiền ấn phẩm). Đề tài dùng cỡ *small* (YOLO26s-seg) với hệ số độ sâu 0,5 và độ rộng 0,5. Bảng 2.2 liệt kê các khối chính; sơ đồ đầy đủ ở Hình 3.1 (Chương 3) **[CÓ THỂ ĐƯA BẢNG 24 TẦNG VÀO PHỤ LỤC]**.

Bảng 2.2: Các khối chính của YOLO26s-seg (ảnh vào 640×640)

Phần	**Tầng**	**Khối**	**Vai trò**	**Đầu ra**
Backbone	0–1	Conv 3×3, bước 2	Giảm kích thước nhanh	64×160×160
	2–8	C3k2 xen kẽ Conv bước 2	Trích đặc trưng ở P3, P4, P5	512×20×20
	9	SPPF	Gộp đặc trưng nhiều tỉ lệ bằng max-pooling lặp	512×20×20

Phần	Tầng	Khối	Vai trò	Đầu ra
	10	**C2PSA**	**Chú ý toàn cục trên P5. Đây là tầng mà đề tài thay thế**	512×20×20
Neck	11–22	Upsample, Concat, C3k2	FPN đi xuống rồi PAN đi lên; riêng C3k2 ở tầng 22 có thêm khối PSA	P3 128×80×80, P4 256×40×40, P5 512×20×20
Head	23	Segment26	Hộp, lớp, 32 hệ số mặt nạ, Proto 32×160×160	Tối đa 300 đối tượng

Một số khối cần giải thích thêm:

– **C3k2** là khối tích chập có kết nối tắt kiểu CSP: chia kênh làm hai phần, chỉ một phần đi qua các khối nút cổ chai, rồi ghép lại. Cách làm này giảm tính toán mà vẫn giữ luồng gradient.

– **SPPF** áp dụng max-pooling 5×5 ba lần nối tiếp và ghép các kết quả, tương đương nhìn đặc trưng ở ba kích thước vùng khác nhau.

– **C2PSA** cũng có khuôn tách–xử lý–ghép. Phần xử lý là khối PSA (partial self-attention) [20] gồm attention đa đầu và mạng truyền thẳng. Ở cỡ *s*, C2PSA có 990.976 tham số. Mục 2.5 giải thích attention.

2.4.2. Các bước của thuật toán YOLO26-seg

Suy luận trên một ảnh gồm các bước sau:

1. **Tiền xử lý.** Co ảnh sao cho cạnh dài bằng 640 và giữ tỉ lệ, thêm viền tối thiểu để mỗi cạnh là bội số của bước 32 (letterbox), chia giá trị điểm ảnh cho 255.
2. **Backbone.** Trích đặc trưng qua các tầng 0–10, tạo ba mức P3, P4, P5.
3. **Neck.** Trộn đặc trưng giữa ba mức theo FPN rồi PAN.
4. **Head.** Tại mỗi điểm của ba mức, đầu một-một dự đoán bốn khoảng cách tới các cạnh hộp, điểm lớp và 32 hệ số mặt nạ. Nhánh Proto26 gộp đặc trưng của cả P3, P4, P5 để sinh 32 mặt nạ mẫu. (Proto26 còn có một nhánh phân đoạn ngữ nghĩa phụ, chỉ chạy khi huấn luyện.)
5. **Chọn đối tượng.** Lấy tối đa 300 dự đoán có điểm cao nhất, bỏ những dự đoán dưới ngưỡng tin cậy. Không có bước NMS.

6. **Tạo mặt nạ.** Với mỗi dự đoán còn lại, tính mặt nạ theo (2.2), phóng song tuyến về kích thước ảnh vào mạng, nhị phân hóa (sigmoid lớn hơn 0,5), cắt theo hộp, rồi đưa về kích thước ảnh gốc.
7. **Trả kết quả.** Danh sách gồm hộp bao, độ tin cậy và mặt nạ của từng đối tượng.

Huấn luyện mỗi lô ảnh gồm các bước:

1. **Tăng cường dữ liệu**, gồm ghép bốn ảnh (Mosaic) [39], lật, co giãn và đổi màu. Mosaic được tắt ở 10 epoch cuối.
2. **Lan truyền xuôi** qua cả hai đầu một-nhiều và một-một. Đặc trưng đi vào đầu một-một được tách khỏi đồ thị gradient (detach), nên đầu này không cập nhật ngược vào backbone.
3. **Gán nhãn** cho từng đầu bằng bộ gán nhãn căn chỉnh nhiệm vụ: đầu một-nhiều lấy 10 điểm tốt nhất cho mỗi đối tượng, đầu một-một giữ 1 điểm. YOLO26 thêm quy tắc STAL nới kích thước vật rất nhỏ khi chọn ứng viên [6].
4. **Tính mất mát** của từng đầu, gồm mất mát hộp (CIoU và L1), mất mát lớp, mất mát mặt nạ và mất mát ngữ nghĩa phụ. Tổng mất mát là tổ hợp có trọng số của hai đầu; trọng số của đầu một-nhiều giảm tuyến tính từ 0,8 xuống 0,1 trong quá trình huấn luyện. Chi tiết hệ số ở Bảng 3.6 và công thức (3.16).
5. **Lan truyền ngược và cập nhật trọng số** bằng bộ tối ưu. Đề tài dùng AdamW [40], là biến thể của Adam tách phần suy giảm trọng số khỏi bước cập nhật theo gradient.
6. **Cập nhật bản trung bình động (EMA)** của trọng số sau mỗi bước. Cuối mỗi epoch, đánh giá trên tập kiểm định bằng trọng số EMA và lưu trọng số tốt nhất.

YOLO26 bỏ thành phần DFL vốn dùng ở các phiên bản trước để biểu diễn tọa độ hộp dưới dạng phân phối [6]. Thay vào đó, mô hình hồi quy trực tiếp bốn khoảng cách, kèm mất mát L1.

Ví dụ giải mã hộp bao. Xét một điểm dự đoán ở ô (cột 10, hàng 12) của mức P3. Mức P3 có bước (stride) 8 pixel, nên tâm ô trên ảnh vào là $((10 + 0{,}5) \times 8,\ (12 + 0{,}5) \times 8) = (84,\ 100)$. Giả sử đầu dự đoán trả về bốn khoảng cách tới cạnh trái, trên, phải, dưới là $(2,\ 1,\ 3,\ 4)$ đơn vị ô. Nhân với bước 8 được $(16,\ 8,\ 24,\ 32)$ pixel. Hộp bao là:

$$(x_1, y_1, x_2, y_2) = (84 - 16,\ 100 - 8,\ 84 + 24,\ 100 + 32)$$
$$= (68,\ 92,\ 108,\ 132)$$

Hộp này rộng 40 và cao 40 pixel. Nếu điểm lớp sau sigmoid là 0,91 và vượt ngưỡng tin cậy, hộp được giữ lại; mặt nạ của nó tính theo (2.2) từ 32 hệ số đi kèm.

2.4.3. Ưu và nhược điểm

Bảng 2.3: Ưu và nhược điểm của YOLO26-seg với bài toán phân đoạn polyp

Ưu điểm	**Nhược điểm**
Một lần suy luận cho cả hộp bao, độ tin cậy và mặt nạ	Mặt nạ mẫu chỉ có độ phân giải 160×160, có thể làm mất chi tiết biên của polyp nhỏ
Không cần NMS nên thời gian suy luận ổn định, dễ triển khai	Thiết kế cho dữ liệu tổng quát (COCO), chưa có thành phần riêng cho biên mờ
Có trọng số tiền huấn luyện trên COCO [41], giúp học nhanh với dữ liệu nhỏ	Attention chỉ có ở mức P5 (C2PSA ở tầng 10 và khối PSA trong tầng 22); các mức P3, P4 chỉ có tích chập
Thư viện Ultralytics [42] hoàn chỉnh: huấn luyện, đánh giá, xuất mô hình	Bài báo YOLO26 là bản tiền ấn phẩm, một số chi tiết chỉ có trong mã nguồn
Có nhiều cỡ mô hình (n, s, m, l, x) để cân đối tốc độ và độ chính xác	Mặt nạ bị cắt theo hộp, nên hộp sai thì mặt nạ cũng sai

2.5. CƠ CHẾ CHÚ Ý VÀ CHI PHÍ TÍNH TOÁN

2.5.1. Tự chú ý

Cơ chế tự chú ý (self-attention) của Transformer [43] cho mỗi vị trí "hỏi" mọi vị trí khác. Với chuỗi đầu vào X gồm N vị trí, mô hình tính ba ma trận truy vấn Q, khóa K và giá trị V bằng các phép chiếu tuyến tính, rồi:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d}}\right)V \tag{2.3}$$

Ma trận $QK^{\top}$ có kích thước $N \times N$: phần tử (i, j) cho biết vị trí i chú ý đến vị trí j bao nhiêu. Đầu ra của mỗi vị trí là trung bình có trọng số của mọi giá trị.

2.5.2. Chi phí bậc hai

Vì phải tính N^2 cặp, chi phí tính toán và bộ nhớ của attention tăng theo bình phương số vị trí:

– ở mức P5 (20×20, $N = 400$): $N^2 = 160.000$ cặp, vẫn chấp nhận được;

– ở mức P3 (80×80, $N = 6.400$): $N^2 \approx 41$ triệu cặp, gấp 256 lần.

Đây là lý do YOLO chỉ đặt attention ở mức P5. Mô hình không gian trạng thái là một hướng giảm chi phí: giữ tính toàn cục nhưng chi phí tăng chậm hơn.

2.6. MÔ HÌNH KHÔNG GIAN TRẠNG THÁI, MAMBA VÀ VMAMBA

2.6.1. Mô hình không gian trạng thái liên tục

Mô hình không gian trạng thái (SSM) mô tả một hệ có trạng thái ẩn $h(t)$, nhận tín hiệu vào $x(t)$ và cho tín hiệu ra $y(t)$:

$$h'(t) = A\,h(t) + B\,x(t), \qquad y(t) = C\,h(t) \tag{2.4}$$

Trong đó A quyết định trạng thái "nhớ" và "quên" thế nào, B quyết định đầu vào ghi vào trạng thái ra sao, C quyết định đọc đầu ra từ trạng thái thế nào. S4 [22] dùng ma trận A dạng chéo cộng hạng thấp; biến thể S4D [44] đơn giản hóa thành ma trận chéo với các phần tử âm. Khi đó trạng thái tự suy giảm theo thời gian nếu không có đầu vào mới. Mamba và VMamba dùng dạng chéo này.

2.6.2. Rời rạc hóa

Dữ liệu thực là chuỗi rời rạc $x_1, x_2, \dots, x_L$. Với bước thời gian Δ, phép rời rạc hóa giữ nguyên trong mỗi bước (zero-order hold) cho:

$$\bar{A} = \exp(\Delta A), \qquad \bar{B} = (\Delta A)^{-1}(\exp(\Delta A) - I)\,\Delta B \approx \Delta B \tag{2.5}$$

$$h_t = \bar{A}\,h_{t-1} + \bar{B}\,x_t, \qquad y_t = C\,h_t \tag{2.6}$$

Xấp xỉ $\bar{B} \approx \Delta B$ (phương pháp Euler) chỉ chính xác khi $|\Delta A|$ nhỏ; Mamba [23] và bản cài đặt trong đề tài (Chương 3) đều dùng xấp xỉ này. Công thức (2.6) là một phép truy hồi: trạng thái mới bằng trạng thái cũ nhân hệ số suy giảm $\bar{A}$ cộng phần đầu vào mới. Vì A âm nên $0 < \bar{A} < 1$. Δ lớn thì $\bar{A}$ nhỏ, nghĩa là quên nhanh quá khứ và chú ý nhiều hơn đến đầu vào hiện tại.

Ví dụ (một chiều). Lấy $A = -1$, $B = C = 1$, $\Delta = 0{,}5$, chuỗi vào $x = (1,\ 0,\ 2)$, $h_0 = 0$. Khi đó $\bar{A} = e^{-0{,}5} \approx 0{,}607$ và, theo xấp xỉ Euler như trong mã, $\bar{B} \approx \Delta B = 0{,}5$. (Nếu dùng công thức ZOH đầy đủ, $\bar{B} = 1 - e^{-0{,}5} \approx 0{,}393$; các con số dưới đây đổi theo nhưng xu hướng giữ nguyên.)

Bước	x_t	h = +	y_t
1	1	0 + =	0,500
2	0	0 × =	0,303
3	2	0 × + =	1,184

Ở bước 2, không có đầu vào mới nên trạng thái giảm dần: mô hình "nhớ" giá trị cũ nhưng mờ đi. Ở bước 3, đầu ra là tổng của thông tin cũ đã suy giảm và đầu vào mới.

2.6.3. Mamba: SSM có chọn lọc

Ở S4, Δ, B, C cố định cho mọi vị trí. Mamba [23] cho ba đại lượng này **phụ thuộc vào đầu vào**: chúng được tính từ x_t qua các phép chiếu tuyến tính, với $\Delta_t = \text{softplus}(\cdot) > 0$. Nhờ vậy mô hình tự quyết định ở từng vị trí nên giữ hay bỏ thông tin.

Tiếp ví dụ trên: nếu ở bước 3 mô hình chọn $\Delta_3 = 2$, thì $\bar{A}_3 = e^{-2} \approx 0{,}135$ và $\bar{B}_3 \approx 2$. Khi đó $h_3 = 0{,}135 \times 0{,}303 + 2 \times 2 \approx 4{,}04$. Phần thông tin cũ gần như bị xóa, còn đầu vào mới được ghi mạnh hơn. Với $\Delta = 2$, xấp xỉ Euler đã khá thô: công thức ZOH cho $\bar{B}_3 = 1 - e^{-2} \approx 0{,}865$ và $h_3 \approx 1{,}76$, nhưng kết luận không đổi: trọng số của đầu vào mới (0,865) lớn hơn hẳn trường hợp $\Delta = 0{,}5$ (0,393), còn phần quá khứ bị giảm mạnh. Mô hình đã "chọn" tập trung vào đầu vào hiện tại.

Sau khi Δ, B, C phụ thuộc đầu vào, không thể tính SSM bằng một phép tích chập cố định như S4 nữa. Phép truy hồi (2.6) phải tính tuần tự hoặc bằng **thuật toán quét song song** [45], [46]. Ý tưởng là mỗi bước có dạng $h_t = a_t h_{t-1} + b_t$, và hai bước liên tiếp có thể gộp thành một bước cùng dạng:

$$(a_1, b_1) \circ (a_2, b_2) = (a_2 a_1,\ a_2 b_1 + b_2) \tag{2.7}$$

Phép gộp (2.7) có tính kết hợp, nên có thể gộp từng cặp song song theo cây, cần khoảng $\log_2 L$ vòng thay vì L bước. Bản Mamba gốc có kernel CUDA tối ưu, chi phí tăng tuyến tính theo độ dài chuỗi. Bản cài đặt trong đề tài dùng biến thể Hillis–Steele [46] viết bằng PyTorch thuần, tổng công việc là $O(L\log L)$ (Chương 3).

2.6.4. Đưa Mamba vào ảnh: VMamba và SS2D

Ảnh hai chiều không có thứ tự tự nhiên. Nếu trải ảnh thành chuỗi theo một hướng, mỗi vị trí chỉ "thấy" được các vị trí đứng trước nó. VMamba [7] đề xuất mô-đun **SS2D** quét theo bốn hướng.

Bốn thứ tự quét này được minh họa ở Hình 3.3 (Chương 3), trên lưới 4×4.

Các bước của SS2D (thuật toán VMamba) trên bản đồ đặc trưng $C \times H \times W$:

1. **Chiếu vào.** Phép chiếu tuyến tính (tích chập 1×1) đưa C kênh lên $2EC$ kênh rồi tách thành nhánh chính U và nhánh cổng Z, mỗi nhánh EC kênh; E là hệ số mở rộng (VMamba thường dùng $E = 2$).
2. **Trộn cục bộ.** Cho U qua tích chập theo chiều sâu 3×3 và SiLU.
3. **Trải thành bốn chuỗi.** Trải U thành chuỗi dài $L = H \times W$ theo bốn thứ tự: theo hàng từ đầu đến cuối; theo hàng từ cuối về đầu (đảo ngược chuỗi thứ nhất); theo cột từ đầu đến cuối; theo cột từ cuối về đầu.
4. **Quét chọn lọc.** Với mỗi chuỗi, tính Δ_t, B_t, C_t từ đầu vào, rời rạc hóa theo (2.5), chạy phép truy hồi (2.6), rồi cộng thêm hạng tử nhảy $D \cdot u_t$ (đưa thẳng đầu vào ra đầu ra).
5. **Gộp.** Đưa bốn chuỗi đầu ra về lưới $H \times W$, cộng lại và chuẩn hóa bằng LayerNorm.
6. **Cổng và chiếu ra.** Nhân với $\mathrm{SiLU}(Z)$ rồi chiếu 1×1 về C kênh.

Bản SS2D cài đặt trong đề tài là bản rút gọn: $E = 1$, không có hạng tử D và không có LayerNorm ở đầu ra. Chương 3 (mục 3.3.2) trình bày các điểm khác này.

Ví dụ thứ tự quét trên lưới 2×2. Đặt tên bốn ô là $\begin{bmatrix} a & b \\ c & d \end{bmatrix}$. Bốn chuỗi là:

– theo hàng: a, b, c, d;

– theo hàng, đảo ngược: d, c, b, a;

– theo cột: a, c, b, d;

– theo cột, đảo ngược: d, b, c, a.

Xét ô b. Ở chuỗi thứ nhất, b chỉ nhận thông tin từ a; ở chuỗi thứ hai, từ d và c; ở chuỗi thứ ba, từ a và c; ở chuỗi thứ tư, từ d. Sau khi cộng bốn chuỗi, b đã nhận thông tin từ cả ba ô còn lại. Với một lần quét duy nhất, điều này không xảy ra.

Sau bước 5, mỗi vị trí đã nhận thông tin từ cả bốn phía, tức là từ toàn bộ ảnh.

2.6.5. Ưu và nhược điểm của VMamba

Bảng 2.4: Ưu và nhược điểm của VMamba

Ưu điểm	**Nhược điểm**
Mỗi vị trí nhận thông tin toàn ảnh sau một khối	Thứ tự quét cố định, không tự nhiên với ảnh; hai điểm gần nhau theo chiều dọc có thể cách xa nhau trong chuỗi quét hàng
Chi phí tăng gần tuyến tính theo số vị trí (với kernel tối ưu), rẻ hơn attention khi ảnh lớn	Ở bản đồ nhỏ như P5 ($L = 400$), lợi thế chi phí so với attention chưa rõ
Cơ chế chọn lọc cho phép giữ hay quên thông tin theo nội dung	Kernel tối ưu cần CUDA; bản PyTorch thuần chậm hơn nhiều
Có nhiều kết quả tốt trên ảnh y tế [24]–[26]	Huấn luyện từ đầu khó hơn; trọng số tiền huấn luyện ảnh hưởng lớn [25]
Không có ma trận $N \times N$ nên tiết kiệm bộ nhớ	Một số chi tiết (khởi tạo Δ, chuẩn hóa đầu ra) nhạy với độ ổn định số

2.7. CÁC ĐỘ ĐO ĐÁNH GIÁ

2.7.1. Các đại lượng cơ bản

So sánh dự đoán với nhãn thật cho bốn loại kết quả:

– **TP (dương tính đúng):** dự đoán là polyp và đúng là polyp.

– **FP (dương tính giả):** dự đoán là polyp nhưng thực tế không phải.

– **FN (âm tính giả):** thực tế là polyp nhưng không được dự đoán.

– **TN (âm tính đúng):** dự đoán là nền và đúng là nền.

Từ đó:

$$\text{Precision} = \frac{TP}{TP+FP}, \qquad \text{Recall} = \frac{TP}{TP+FN}, \qquad F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision}+\text{Recall}} \tag{2.8}$$

Precision cho biết trong những gì mô hình báo là polyp, bao nhiêu phần đúng. Recall cho biết trong các polyp thật, mô hình tìm được bao nhiêu. Trong nội soi, Recall thấp nghĩa là bỏ sót polyp, nên thường được coi trọng hơn.

Các đại lượng này tính được ở hai mức:

– **Mức điểm ảnh:** mỗi điểm ảnh là một mẫu. Dùng khi so sánh mặt nạ.

– **Mức đối tượng:** mỗi polyp là một mẫu. Một dự đoán được tính là TP khi trùng với một polyp thật đủ nhiều (IoU vượt ngưỡng). Ở mức này, TN không xác định được, vì không thể đếm "số đối tượng không phải polyp" trong ảnh.

2.7.2. IoU và Dice

Với G là tập điểm ảnh của nhãn thật và P là tập điểm ảnh dự đoán:

$$\text{IoU} = \frac{|P \cap G|}{|P \cup G|} = \frac{TP}{TP+FP+FN}, \qquad \text{Dice} = \frac{2|P \cap G|}{|P|+|G|} = \frac{2\,TP}{2\,TP+FP+FN} \tag{2.9}$$

IoU còn gọi là chỉ số Jaccard; Dice còn gọi là hệ số Sørensen–Dice. Hai chỉ số liên hệ với nhau qua $\text{Dice} = 2\,\text{IoU}/(1+\text{IoU})$, nên Dice luôn lớn hơn hoặc bằng IoU. Ở mức điểm ảnh, Dice trùng với F1.

Ví dụ trên lưới 4×4 (1 là polyp):

$$G = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}, \qquad P = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

– Nhãn thật có 9 điểm ảnh polyp, dự đoán có 8.

– Phần trùng: $TP = 7$. Dự đoán thừa: $FP = 1$ (ô hàng 1, cột 4). Bỏ sót: $FN = 2$ (ô hàng 2, cột 1 và ô hàng 4, cột 2). Còn lại $TN = 6$.

Độ đo	**Cách tính**	**Giá trị**
Precision	7 / (7 + 1)	0,875
Recall	7 / (7 + 2)	0,778
IoU	7 / (7 + 1 + 2)	0,700
Dice = F1	2·7 / (2·7 + 1 + 2)	0,824

Kiểm tra lại: $2 \times 0{,}7/(1+0{,}7) = 0{,}824$, khớp với Dice.

Khi đánh giá cả tập, có hai cách lấy trung bình:

– **trung bình theo ảnh:** tính Dice cho từng ảnh rồi lấy trung bình;

– **gộp toàn tập:** cộng TP, FP, FN của mọi ảnh rồi mới tính.

Hai cách cho kết quả khác nhau khi kích thước polyp chênh lệch lớn. Chương 6 ghi rõ cách nào được dùng.

2.7.3. AP, mAP50 và mAP50–95

Ở mức đối tượng, mỗi dự đoán có một độ tin cậy. Sắp xếp các dự đoán theo độ tin cậy giảm dần, rồi tính Precision và Recall tích lũy sau mỗi dự đoán, ta được đường Precision–Recall. **AP** (Average Precision) là diện tích dưới đường này, sau khi làm đường đơn điệu bằng cách lấy Precision lớn nhất về phía Recall cao hơn [47].

Ví dụ. Tập ảnh có 3 polyp. Mô hình cho 4 dự đoán; ở ngưỡng IoU 0,5, kết quả như sau:

Thứ tự	Độ tin cậy	Đúng/sai	TP tích lũy	Precision	Recall
1	0,95	TP	1	1/1 = 1,000	1/3 = 0,333
2	0,85	FP	1	1/2 = 0,500	0,333
3	0,70	TP	2	2/3 = 0,667	2/3 = 0,667
4	0,40	TP	3	3/4 = 0,750	3/3 = 1,000

Sau khi làm đơn điệu, Precision bằng 1,0 trên đoạn Recall từ 0 đến 1/3, và bằng 0,75 trên đoạn từ 1/3 đến 1. Do đó:

$$AP = 1/3 \times 1{,}0 + 2/3 \times 0{,}75 \approx 0{,}833$$

Đây là cách nội suy mọi điểm. COCO [41] và Ultralytics lấy mẫu đường đơn điệu tại 101 mức Recall cách đều (0; 0,01; …; 1); với ví dụ này cách đó cũng cho AP khoảng 0,83.

Từ AP có các chỉ số:

- **mAP50:** AP tại ngưỡng IoU 0,5, lấy trung bình qua các lớp. Đề tài chỉ có một lớp nên mAP50 bằng AP50.
- **mAP50–95:** trung bình AP tại 10 ngưỡng IoU từ 0,50 đến 0,95, bước 0,05 [41]. Ở ngưỡng cao như 0,9, chỉ những mặt nạ bám rất sát đường biên mới được tính là đúng. Vì vậy mAP50–95 phản ánh độ chính xác của đường biên tốt hơn mAP50.

Với phân đoạn thực thể, IoU trong các chỉ số trên được tính giữa **mặt nạ** dự đoán và mặt nạ thật, ký hiệu *mask* mAP. Nếu tính giữa hai hộp bao thì gọi là *box* mAP. Đề tài lấy **mask mAP50–95** làm độ đo chính.

2.7.4. Chi phí tính toán

– **Số tham số:** tổng số trọng số học được. Tích chập $k \times k$ từ C_{in} sang C_{out} kênh có $k^2 C_{\text{in}} C_{\text{out}}$ trọng số (cộng thêm hệ số tự do nếu có). Ví dụ tầng đầu của YOLO26s-seg (3×3, từ 3 sang 32 kênh) có $9 \times 3 \times 32 = 864$ trọng số, cộng 64 tham số của lớp BN.

– **FLOPs:** số phép tính dấu phẩy động cho một ảnh. Một phép nhân–cộng tính là 2 FLOPs. Tích chập trên bản đồ $H \times W$ cần khoảng $2k^2 C_{\text{in}} C_{\text{out}} HW$ FLOPs. Ví dụ tích chập 1×1 từ 256 sang 512 kênh trên lưới 20×20 cần $2 \times 256 \times 512 \times 400 \approx 105$ triệu FLOPs.

– **Thời gian suy luận:** thời gian xử lý một ảnh, đo bằng mili giây, phụ thuộc phần cứng. Cần phân biệt thời gian chỉ chạy mạng (forward) với thời gian toàn quy trình (tiền xử lý, chạy mạng, hậu xử lý).

FLOPs và số tham số không phản ánh đầy đủ tốc độ thực tế. Một khối có ít FLOPs nhưng gồm nhiều phép tính tuần tự, như phép quét trong SS2D, vẫn có thể chạy chậm.

2.8. SO SÁNH HAI MÔ HÌNH QUA NHIỀU LẦN HUẤN LUYỆN

2.8.1. Vì sao cần nhiều seed

Huấn luyện mạng nơ-ron có nhiều yếu tố ngẫu nhiên: khởi tạo trọng số, thứ tự đưa ảnh vào, các phép tăng cường dữ liệu. Các yếu tố này được điều khiển bằng một số nguyên gọi là **seed**. Cùng một mô hình, đổi seed có thể cho kết quả khác nhau đáng kể [31]. Nếu chỉ chạy mỗi mô hình một lần, chênh lệch quan sát được có thể chỉ do may rủi.

Vì vậy, đề tài huấn luyện mỗi mô hình với 20 seed (seed 0–19). Hai mô hình dùng **cùng danh sách seed**, nên kết quả được **ghép cặp** theo seed. Với mỗi seed i, tính chênh lệch $d_i = \text{TSVM}_i - \text{Baseline}_i$. Phép so sánh ghép cặp loại bỏ được phần dao động chung của hai mô hình ở cùng seed.

2.8.2. Kiểm định t ghép cặp

Với n chênh lệch $d_1, \ldots, d_n$, có trung bình $\bar{d}$ và độ lệch chuẩn s_d:

$$t = \frac{\bar{d}}{s_d / \sqrt{n}} \tag{2.10}$$

Dưới giả thuyết không (H_0: hai mô hình như nhau, kỳ vọng của d bằng 0), t có phân phối Student với $n - 1$ bậc tự do [48]. Giá trị p là xác suất gặp $|t|$ lớn như

vậy hoặc hơn khi H_0 đúng. Nếu $p < 0{,}05$, ta nói chênh lệch có ý nghĩa thống kê ở mức 5%.

Khoảng tin cậy 95% của chênh lệch trung bình là:

$$\bar{d} \pm t_{0{,}975;\, n-1} \cdot \frac{s_d}{\sqrt{n}} \tag{2.11}$$

Nếu khoảng này chứa 0, dữ liệu chưa đủ để khẳng định mô hình nào tốt hơn.

2.8.3. Kiểm định Wilcoxon và cỡ hiệu ứng

Kiểm định t giả định chênh lệch có phân phối gần chuẩn. Với ít seed, giả định này khó kiểm tra. **Kiểm định Wilcoxon dấu–hạng** [49] chỉ dùng thứ hạng của $|d_i|$ và dấu của chúng, không cần giả định phân phối. Demšar [50] khuyến nghị dùng kiểm định này khi so sánh hai mô hình học máy. Đề tài báo cáo cả hai kiểm định.

Giá trị p chỉ cho biết chênh lệch có đáng tin hay không, không cho biết chênh lệch lớn hay nhỏ. **Cỡ hiệu ứng** Cohen d_z cho dữ liệu ghép cặp [51] đo độ lớn này:

$$d_z = \frac{\bar{d}}{s_d} \tag{2.12}$$

Theo quy ước của Cohen, được Lakens [51] trình bày lại, d_z khoảng 0,2 là nhỏ, 0,5 là vừa, 0,8 là lớn.

Ví dụ minh họa (số tự đặt). Năm seed cho chênh lệch mAP lần lượt $0{,}010;\ -0{,}004;\ 0{,}006;\ 0{,}002;\ 0{,}008$.

- $\bar{d} = 0{,}0044$; $s_d \approx 0{,}0055$; sai số chuẩn $s_d/\sqrt{5} \approx 0{,}0025$.
- $t \approx 0{,}0044/0{,}0025 \approx 1{,}77$, với 4 bậc tự do cho $p \approx 0{,}15$ (hai phía).
- Kiểm định Wilcoxon: chỉ một chênh lệch âm, có hạng 2, nên thống kê $W = 2$ và $p \approx 0{,}19$ (hai phía).
- Khoảng tin cậy 95%: $0{,}0044 \pm 2{,}776 \times 0{,}0025 = [-0{,}0025;\ 0{,}0113]$, chứa 0.
- $d_z \approx 0{,}79$, tức cỡ hiệu ứng khá lớn.

Ví dụ này cho thấy một điểm quan trọng: dù trung bình dương và cỡ hiệu ứng không nhỏ, **với chỉ 5 cặp và dao động lớn giữa các seed, kết quả vẫn chưa có ý nghĩa thống kê**. Muốn kết luận chắc chắn hơn cần thêm seed hoặc chênh lệch thật phải lớn hơn. Chương 6 dùng đúng các công cụ này để đọc kết quả thực nghiệm.

2.9. KẾT CHƯƠNG

Chương 2 đã trình bày:

- ba bài toán thị giác cơ bản và lý do chọn phân đoạn thực thể;
- các thành phần của mạng tích chập và giới hạn về trường tiếp nhận;
- họ YOLO, cơ chế bỏ NMS bằng hai đầu dự đoán, và cách tạo mặt nạ bằng mặt nạ mẫu;
- kiến trúc, các bước suy luận và huấn luyện, ưu và nhược điểm của YOLO26-seg;
- cơ chế chú ý và chi phí bậc hai;
- mô hình không gian trạng thái, cơ chế chọn lọc của Mamba, mô-đun SS2D của VMamba, cùng ưu và nhược điểm;
- các độ đo Precision, Recall, F1, IoU, Dice, mAP50, mAP50–95 và chi phí tính toán, có ví dụ tính tay;
- phương pháp so sánh ghép cặp qua nhiều seed bằng kiểm định t, Wilcoxon và cỡ hiệu ứng.

Chương 3 dựa trên các kiến thức này để trình bày khối C2TSVMamba: cách đặt nhánh SS2D của VMamba vào vị trí khối C2PSA của YOLO26s-seg, kèm nhánh hình dạng và cổng điều khiển.

CHƯƠNG 3: MÔ HÌNH ĐỀ XUẤT

Chương 2 đã trình bày các khối nền của YOLO26-seg và nguyên lý của mô hình không gian trạng thái. Chương này mô tả cách nhóm kết hợp hai phần đó. Mô hình đề xuất giữ nguyên gần như toàn bộ YOLO26s-seg, chỉ thay khối C2PSA ở tầng 10 bằng một khối mới tên là C2TSVMamba. Bên trong khối mới có hai nhánh. Nhánh SS2D quét đặc trưng theo kiểu VMamba. Nhánh hình dạng dùng tích chập có hướng và điều khiển nhánh SS2D thông qua một cổng.

Chương lần lượt trình bày các nội dung sau:

– ý tưởng thiết kế;

– kiến trúc tổng thể;

– cấu tạo từng nhánh;

– thuật toán quét song song;

– cách khởi tạo;

– quy trình suy luận và huấn luyện;

– ưu điểm và hạn chế của thiết kế.

Thông số trong chương được đọc từ mã nguồn đã dùng để huấn luyện: bản fork Ultralytics 8.4.127 [42] cùng mô-đun `topology_shape_vmamba.py`. Mọi thông số đều được kiểm tra lại bằng cách dựng mô hình rồi đếm tham số. Con số nào là ước tính tính tay thì có ghi rõ ngay tại chỗ.

Quy ước thuật ngữ trong chương:

Thuật ngữ	**Nghĩa trong chương này**
Khối C2TSVMamba	Toàn bộ tầng 10.
Khối TSVMamba	Phần xử lý nằm bên trong C2TSVMamba.
Nhánh SS2D	Phần quét chọn lọc theo kiểu VMamba.
Nhánh hình dạng	Hai mô-đun ShapeAwareBranch và DirectionalShapeExtractor.
Cổng	Mô-đun TopologyShapeGate.
Attention	Cơ chế tự chú ý trong khối PSA.
Lượt huấn luyện	Một lần huấn luyện một mô hình với một seed.

3.1. Ý TƯỞNG VÀ ĐỘNG CƠ THIẾT KẾ

3.1.1. Vấn đề cần giải quyết

Polyp trong ảnh nội soi thường có ba đặc điểm gây khó cho phân đoạn:

– biên mờ;

– màu gần với niêm mạc xung quanh;

– kích thước rất khác nhau giữa các ảnh.

Vì vậy, để khoanh đúng vùng polyp, mô hình cần hai loại thông tin.

– **Ngữ cảnh rộng:** cho biết vùng đang xét là một khối lồi lên khỏi thành ruột, hay chỉ là vệt phản quang hoặc nếp gấp niêm mạc.

– **Hình dạng của vùng:** vùng đó trải theo hướng nào và có đường bao ra sao.

Trong YOLO26-seg, ngữ cảnh rộng ở cuối backbone do khối C2PSA đảm nhận. Khối này dùng attention [43] trên bản đồ đặc trưng P5 kích thước 20×20. Attention so khớp mọi cặp vị trí với nhau, nên nắm được quan hệ xa. Thông tin không gian cục bộ của khối này chỉ đến từ một tích chập theo chiều sâu 3×3 trên nhánh giá trị, đóng vai trò mã hóa vị trí. Ngoài tích chập đó, khối không có thành phần nào thiết kế riêng cho hình dạng.

Các công trình gần đây về phân đoạn polyp cho thấy thông tin biên và hình dạng là manh mối có ích [27], [28]. Cùng thời gian đó, Mamba [23] và VMamba [7] đưa ra một cách khác để thu ngữ cảnh toàn cục:

– trải bản đồ đặc trưng thành chuỗi;

– cập nhật một trạng thái ẩn có chọn lọc theo nội dung của từng vị trí.

Từ hai hướng này, đề tài đặt câu hỏi: nếu thay C2PSA bằng một khối kết hợp quét chọn lọc kiểu VMamba với một nhánh tích chập có hướng, thì

– chất lượng mặt nạ polyp của YOLO26s-seg thay đổi thế nào;

– chi phí tính toán phải trả thêm là bao nhiêu.

3.1.2. Vì sao chọn tầng 10

Khối mới được đặt đúng vào vị trí của C2PSA ở tầng 10, vì bốn lý do sau.

1. **Đây là khối attention duy nhất trong backbone của YOLO26s-seg.** Thay đúng khối này thì hai mô hình chỉ khác nhau ở cách tổng hợp ngữ cảnh tại một vị trí. Các tầng còn lại giống hệt nhau, nên phép so sánh rõ ràng.

2. **Chuỗi cần quét ngắn.** Tầng 10 nằm ở mức P5 (bước nhảy 32). Với ảnh vào 640×640, bản đồ đặc trưng chỉ có 20×20 = 400 vị trí. Ở mức sâu này, mỗi vị trí đã mang thông tin ngữ nghĩa cao. Dù vậy, như sẽ thấy ở mục 3.4.5 và Chương 6, cách cài đặt phép quét vẫn làm độ trễ tăng đáng kể.
3. **Đầu ra của tầng 10 lan tới cả ba mức của đầu Segment26.** Đầu ra này được dùng ở hai nơi trong neck:

+ tầng 11, nơi đặc trưng được phóng to rồi đi xuống nhánh P4 và P3;
+ tầng 21, nơi đặc trưng được ghép vào nhánh P5 đi lên.

– Vì vậy đặc trưng do khối mới tạo ra có mặt ở cả P3, P4 và P5.

4. **Giao diện không đổi.** C2PSA có vào/ra là 512×20×20, và khối mới giữ đúng giao diện này. Nhờ đó không phải sửa tầng nào khác, cũng không phải đổi neck, head hay hàm mất mát.

3.1.3. Phạm vi đóng góp

Các thành phần dùng trong khối mới đều đã có từ trước:

– SS2D là ý tưởng của VMamba [7];
– cơ chế quét chọn lọc đến từ Mamba [23];
– tích chập theo chiều sâu, cổng sigmoid và nối tắt có hệ số là các kỹ thuật quen thuộc.

Đề tài không nhận mình là tác giả của các thành phần trên. Đóng góp của đề tài gồm bốn phần:

1. Thiết kế khối C2TSVMamba: ghép các thành phần trên theo một cách cụ thể, trong đó có cổng để nhánh hình dạng điều biến nhánh SS2D.
2. Cài đặt lại SS2D bằng PyTorch thuần, không cần kernel CUDA riêng, để gắn được vào Ultralytics.
3. Tích hợp khối mới vào YOLO26s-seg cho bài toán phân đoạn polyp.
4. Đánh giá lặp lại trên 20 seed (seed 0–19), có kiểm định thống kê (Chương 6).

Hai công trình gần nhất, đã so sánh chi tiết ở Chương 1:

– **MambaPSA [30]** cũng thay C2PSA của YOLO26 bằng một khối Mamba. Tuy vậy, bài này giải bài toán phát hiện vật thể trên PASCAL VOC, nhắm tới giảm chi phí, và thêm khối BiViM vào neck. Đề tài này giải bài toán phân đoạn polyp và bổ sung nhánh hình dạng.

– **Mamba YOLO [29]** thay nhiều khối của YOLO bằng khối SSM, chứ không chỉ thay ở một vị trí.

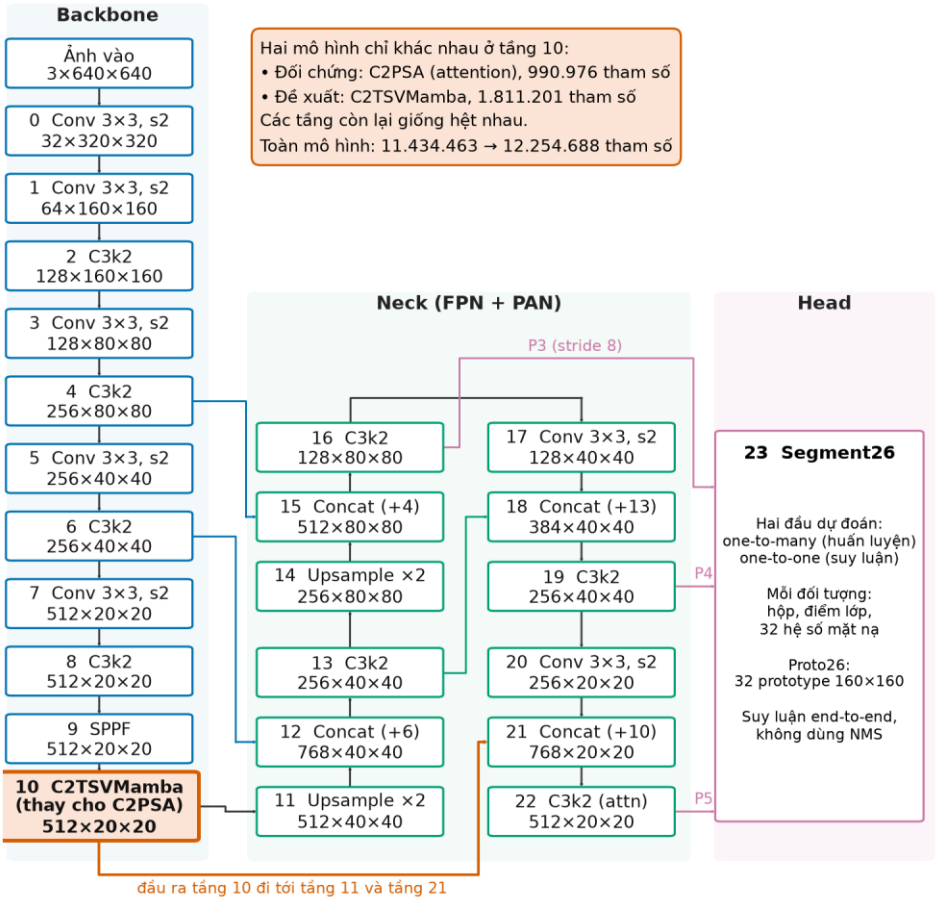
3.2. KIẾN TRÚC TỔNG THỂ

Hình 3.1 vẽ toàn bộ mô hình, gồm ba phần.

– **Backbone (tầng 0 đến 10):** năm lần tích chập bước 2, xen giữa là các khối C3k2, giảm độ phân giải từ 640×640 xuống 20×20. Tầng 9 là SPPF. Tầng 10 là vị trí thay đổi.

– **Neck (tầng 11 đến 22):** một nhánh đi từ trên xuống theo kiểu FPN [35] và một nhánh đi từ dưới lên theo kiểu PAN [36]. Neck cho ra ba bản đồ đặc trưng: P3 (80×80), P4 (40×40) và P5 (20×20).

– **Đầu Segment26 (tầng 23):** nhận cả ba mức để dự đoán hộp, điểm lớp và hệ số mặt nạ. Đầu này cũng sinh 32 mặt nạ mẫu (prototype) kích thước 160×160, theo cơ chế của YOLACT [19].



Hình 3.1: Kiến trúc YOLO26s-seg và vị trí khối C2TSVMamba (tầng 10). Kích thước tensor ghi theo dạng C×H×W, tính cho ảnh vào 640×640 với hệ số quy mô s. Tầng 22 (C3k2, attn=True) có PSABlock ở cả hai mô hình.

Tệp cấu hình của mô hình đề xuất được sao từ `yolo26-seg.yaml`. Ngoài phần chú thích, hai tệp chỉ khác nhau ở dòng khai báo tầng 10. Bảng 3.1 tóm tắt điểm khác nhau giữa hai mô hình.

Bảng 3.1: So sánh mô hình đối chứng và mô hình đề xuất

Tiêu chí	**YOLO26s-seg (đối chứng)**	**YOLO26s-seg + C2TSVMamba (đề xuất)**
Khối ở tầng 10	C2PSA (1 PSABlock, 4 đầu attention)	C2TSVMamba (1 khối TSVMamba)
Vào/ra tầng 10	512×20×20	512×20×20
Tham số tầng 10	990.976	1.811.201
Tham số toàn mô hình	11.434.463	12.254.688 (+7,17%)
Tầng 0–9 và 11–23	Giống nhau	Giống nhau
Hàm mất mát, dữ liệu, siêu tham số	Giống nhau	Giống nhau

Tầng 22 của cả hai mô hình là khối C3k2 với tham số `attn=True`. Khi tham số này bật, mỗi khối con bên trong gồm một Bottleneck nối tiếp một PSABlock. Vì vậy, mô hình đề xuất không bỏ hết attention: nó chỉ thay attention ở tầng 10, còn PSABlock ở tầng 22 vẫn giữ nguyên trong cả hai mô hình.

Khi suy luận, YOLO26 chạy ở chế độ end-to-end:

– chỉ dùng đầu dự đoán một-một;

– lấy tối đa 300 đối tượng có điểm cao nhất rồi lọc theo ngưỡng tin cậy;

– không qua bước loại bỏ trùng lặp NMS.

Cơ chế này kế thừa từ YOLOv10 [20] và giữ nguyên ở cả hai mô hình.

3.3. KHỐI C2TSVMAMBA

3.3.1. Khung tách–xử lý–ghép

C2TSVMamba giữ nguyên khuôn ngoài của C2PSA và xử lý theo bốn bước:

1. Đặc trưng vào $x \in \mathbb{R}^{512\times20\times20}$ đi qua một tích chập 1×1, kèm chuẩn hóa lô và hàm SiLU.
2. Kết quả được chia đôi theo kênh thành a và b, mỗi phần 256 kênh.
3. Phần a đi thẳng; phần b đi qua khối TSVMamba.
4. Hai phần được ghép lại rồi đi qua một tích chập 1×1 nữa.

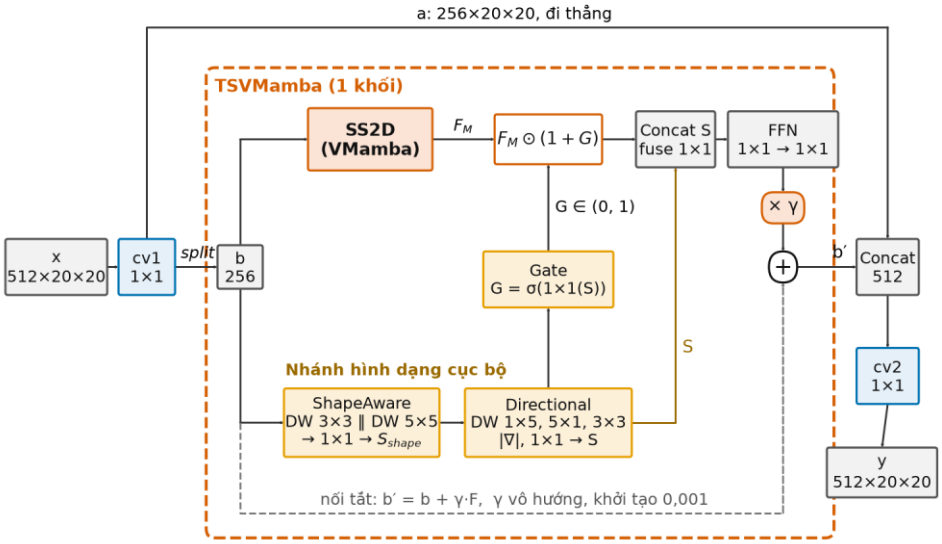
$$[a,\ b] = \text{split}(\text{Conv}_{1\times1}^{cv1}(x)), \qquad y = \text{Conv}_{1\times1}^{cv2}([\,a \parallel \text{TSVMamba}(b)\,]) \tag{3.1}$$

Trong đó $[\cdot \parallel \cdot]$ là phép ghép theo kênh. Tệp cấu hình ghi 2 khối lặp, nhưng với quy mô s, hệ số độ sâu 0,5 làm số khối thực tế còn $\max(\text{round}(2 \times 0{,}5),1) = 1$.

Cách chia đôi này giống thiết kế CSP dùng trong họ YOLO từ YOLOv4 [39]. Một nửa số kênh không đi qua phần tính toán nặng, nhờ đó giảm chi phí và giữ đường truyền gradient ngắn.

Hình 3.2 vẽ cấu trúc bên trong khối TSVMamba:

1. Đặc trưng b được đưa đồng thời vào nhánh SS2D và nhánh hình dạng.
2. Nhánh hình dạng sinh ra bản đồ cổng G để điều biến đầu ra của SS2D.
3. Hai nhánh được hợp nhất, rồi tinh chỉnh qua một mạng truyền thẳng (FFN).
4. Kết quả được nhân với hệ số γ rồi cộng trở lại vào b.



Hình 3.2: Cấu trúc khối C2TSVMamba. Khung nét đứt là khối TSVMamba. Đặc trưng bên trong khối có kích thước 256×20×20. Chi tiết nhánh SS2D ở Hình 3.4.

3.3.2. Nhánh SS2D

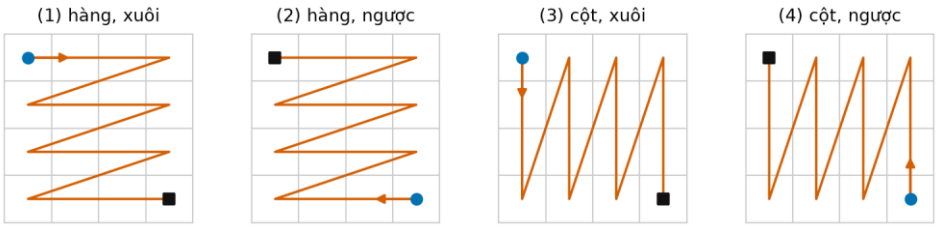
Nhánh này cài đặt lại mô-đun quét chọn lọc hai chiều (SS2D) của VMamba [7]. Mô-đun xử lý theo ba ý chính:

– biến bản đồ đặc trưng hai chiều thành các chuỗi một chiều;

– cho mỗi chuỗi đi qua một mô hình không gian trạng thái chọn lọc;

– gộp kết quả trở lại lưới.

Nếu chỉ duyệt một lần theo hàng, mỗi vị trí chỉ nhận được thông tin từ các vị trí đứng trước nó trong thứ tự duyệt. Vì vậy SS2D duyệt theo bốn thứ tự khác nhau, để mỗi vị trí nhận thông tin từ mọi phía (Hình 3.3).



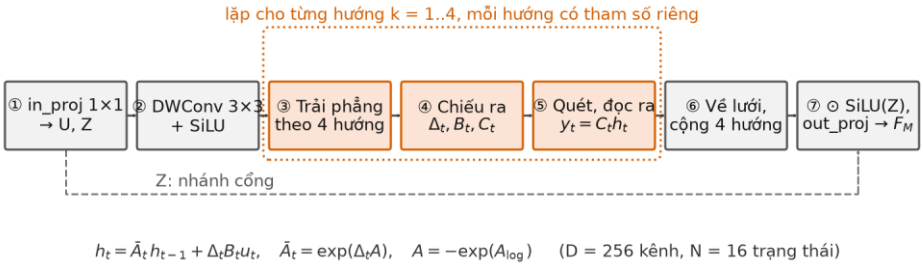
Hình 3.3: Bốn thứ tự duyệt của SS2D, minh họa trên lưới 4×4 (thực tế là 20×20, L = 400). Chấm tròn đánh dấu vị trí đầu chuỗi, ô vuông đánh dấu vị trí cuối chuỗi.

Hình 3.4 tóm tắt các bước tính của SS2D trong mã. Các ký hiệu dùng trong mục này:

– $D = 256$: số kênh;

– $N = 16$: số chiều trạng thái;

– $L = H \times W = 400$: độ dài chuỗi.



Hình 3.4: Các bước bên trong nhánh SS2D như được cài đặt. Bước ③ đến ⑤ được thực hiện riêng cho từng hướng.

Bước 1. Chiếu vào và tách nhánh cổng. Một tích chập 1×1 không có độ lệch (bias) nâng b từ 256 lên 512 kênh, rồi chia thành hai phần U và Z, mỗi phần 256 kênh:

– U là nội dung sẽ được quét;

– Z được giữ lại để làm cổng ở bước cuối.

Bước 2. Trộn cục bộ. U đi qua một tích chập theo chiều sâu 3×3 và hàm SiLU, cho ra $\tilde{U}$. Bước này cho mỗi vị trí nhận một ít thông tin từ lân cận trước khi quét. Mamba cũng đặt một tích chập ngắn trước SSM theo cách tương tự [23].

Bước 3. Trải phẳng theo bốn hướng. Lưới 20×20 được trải thành bốn chuỗi, mỗi chuỗi dài 400:

1. theo hàng, từ đầu đến cuối;
2. theo hàng, từ cuối về đầu;
3. theo cột, từ đầu đến cuối;
4. theo cột, từ cuối về đầu.

Gọi Π_k là phép trải phẳng thứ k ($k = 1..4$) và $u^{(k)} = \Pi_k(\tilde{U})$.

Bước 4. Sinh tham số phụ thuộc đầu vào. Với mỗi hướng k, một phép chiếu tuyến tính riêng biến mỗi vectơ $u_t^{(k)} \in \mathbb{R}^D$ thành ba đại lượng: bước thời gian Δ_t, ma trận vào B_t và ma trận ra C_t.

$$\Delta_t^{(k)} = \text{softplus}\left(W_\Delta^{(k)} W_{x,\Delta}^{(k)} u_t^{(k)} + b_\Delta^{(k)}\right) \in \mathbb{R}^D, \quad B_t^{(k)} = W_{x,B}^{(k)} u_t^{(k)} \in \mathbb{R}^N, \quad C_t^{(k)} = W_{x,C}^{(k)} u_t^{(k)} \in \mathbb{R}^N \qquad (3.2)$$

Một vài điểm về công thức (3.2):

– Δ được sinh qua một phép chiếu hạng thấp (hạng 16), rồi mở rộng lên 256 kênh.

– Hàm softplus bảo đảm $\Delta_t > 0$.

– Đây là cơ chế chọn lọc của Mamba [23]. Vì Δ, B, C phụ thuộc nội dung tại từng vị trí, mô hình tự quyết định giữ lại thông tin cũ hay ghi đè bằng thông tin mới.

Bước 5. Rời rạc hóa, quét và đọc ra. Ma trận trạng thái được tham số hóa dạng chéo và luôn âm, $A^{(k)} = -\exp(A_{\log}^{(k)}) \in \mathbb{R}^{D\times N}$, theo cách của S4D [44]. Hệ số chuyển trạng thái, phương trình truy hồi và đầu ra lần lượt là:

$$\bar{A}_{t,d,n}^{(k)} = \exp\left(\text{clip}\left(\Delta_{t,d}^{(k)} A_{d,n}^{(k)}, -50, 0\right)\right) \qquad (3.3)$$

$$h_{t,d,n}^{(k)} = \bar{A}_{t,d,n}^{(k)} h_{t-1,d,n}^{(k)} + \Delta_{t,d}^{(k)} B_{t,n}^{(k)} u_{t,d}^{(k)}, \qquad h_0^{(k)} = 0 \qquad (3.4)$$

$$y_{t,d}^{(k)} = \sum_{n=1}^{N} C_{t,n}^{(k)} \; h_{t,d,n}^{(k)} \qquad (3.5)$$

Giải thích các công thức (3.3)–(3.5):

– Vì $\Delta A \leq 0$ nên $\bar{A} \in (0,1]$. Trạng thái cũ luôn mờ dần, và tốc độ mờ do Δ quyết định.

– Số hạng đầu vào dùng xấp xỉ bậc một $\bar{B} \approx \Delta B$, thay cho công thức rời rạc hóa giữ bậc không (ZOH) đầy đủ. Mamba cũng dùng xấp xỉ này khi cài đặt [23].

– Phép cắt clip ở (3.3) chỉ có trong bản cài đặt này, nhằm tránh lấy exp của số âm quá lớn.

– Phương trình (3.4) không được giải bằng vòng lặp qua từng vị trí, mà bằng thuật toán quét song song ở mục 3.4.

Bước 6. Gộp bốn hướng. Mỗi chuỗi kết quả $y^{(k)}$ được đảo ngược phép trải phẳng để trở về lưới 20×20. Sau đó bốn lưới được cộng lại.

Bước 7. Cổng và chiếu ra. Tổng bốn hướng được nhân từng phần tử với $\mathrm{SiLU}(Z)$, rồi đi qua một tích chập 1×1 để cho ra đặc trưng toàn cục F_M:

$$F_M = W_{\text{out}} * \left(\left(\sum_{k=1}^{4} \Pi_k^{-1} \left(y^{(k)} \right) \right) \odot \mathrm{SiLU}(Z) \right) \in \mathbb{R}^{256 \times 20 \times 20} \qquad (3.6)$$

Mỗi hướng có bộ tham số chiếu và $A_{\log}$ riêng, nên bốn hướng có thể học bốn cách ghi nhớ khác nhau.

So với VMamba gốc [7], bản cài đặt này có năm điểm khác:

– Không có hạng tử nhảy $D \cdot u$ bên trong SSM. Đường tắt được thay bằng nối tắt có hệ số γ ở ngoài khối (mục 3.3.5).

– Không có lớp chuẩn hóa LayerNorm sau khi gộp bốn hướng.

– Vẫn giữ cổng nhân $\mathrm{SiLU}(Z)$, trong khi khối VSS cải tiến của VMamba đã bỏ nhánh này.

– Phép chiếu Δ dùng khởi tạo mặc định của PyTorch. Mamba thì khởi tạo sao cho Δ nằm trong khoảng [0,001; 0,1] [23].

– Phép quét chạy bằng các phép toán tensor của PyTorch, không dùng kernel CUDA tối ưu.

3.3.3. Nhánh hình dạng

Nhánh hình dạng chỉ gồm các tích chập và có hai mô-đun nối tiếp. Theo ý đồ thiết kế, nhánh này cung cấp cho khối thông tin hình học của vùng đặc trưng: đặc trưng trải theo hướng nào và rộng hẹp ra sao.

Cần lưu ý rằng nhánh chạy ở mức P5, tức lưới 20×20 với bước nhảy 32. Một nhân 1×5 ở đây đã phủ khoảng 160 pixel trên ảnh gốc. Vì vậy nhánh này nắm hình dạng ở mức vùng lớn, không phải chi tiết đường biên ở mức pixel. Đề tài cũng

chưa có thí nghiệm riêng để kiểm chứng nhánh thực sự học được thông tin hình dạng.

ShapeAwareBranch xử lý như sau:

1. Chạy song song hai tích chập theo chiều sâu kích thước 3×3 và 5×5. Mỗi tích chập kèm chuẩn hóa lô và SiLU.
2. Ghép hai kết quả lại.
3. Trộn kênh bằng tích chập 1×1.

$$S_{\text{shape}} = \text{Conv}_{1\times1}([\text{DW}_{3\times3}(b) \parallel \text{DW}_{5\times5}(b)]) \quad (3.7)$$

Hai kích thước nhân cho hai vùng tiếp nhận có độ rộng khác nhau.

DirectionalShapeExtractor tách thông tin theo hướng:

1. Chạy song song ba tích chập theo chiều sâu trên S_{shape}: nhân dải ngang 1×5, nhân dải dọc 5×1 và nhân 3×3.
2. Từ hai đáp ứng ngang và dọc, tính thêm một đại lượng độ lớn S_{grad}.
3. Ghép bốn kết quả, trộn bằng tích chập 1×1 được S_{topo}.
4. Ghép S_{topo} với S_{shape}, trộn thêm một lần 1×1 được đầu ra S.

$$S_h = \text{DW}_{1\times5}(S_{\text{shape}}), \quad S_v = \text{DW}_{5\times1}(S_{\text{shape}}), \quad S_{loc} = \text{DW}_{3\times3}(S_{\text{shape}}), \quad S_{grad} = \sqrt{S_h^2 + S_v^2 + 10^{-6}} \quad (3.8)$$

$$S_{topo} = \text{Conv}_{1\times1}([S_h \parallel S_v \parallel S_{loc} \parallel S_{grad}]), \qquad S = \text{Conv}_{1\times1}([S_{\text{shape}} \parallel S_{topo}]) \quad (3.9)$$

Công thức của S_{grad} có dạng giống độ lớn gradient trong xử lý ảnh [52]. Tuy vậy, S_h và S_v là đáp ứng của các nhân học được, đã qua chuẩn hóa lô và SiLU. Vì thế S_{grad} không phải gradient ảnh như bộ lọc Sobel. Nó là độ lớn tổng hợp của hai đáp ứng ngang và dọc, cho mạng thêm một đặc trưng ít phụ thuộc vào hướng cụ thể. Đầu ra S có kích thước 256×20×20.

3.3.4. Cổng

Cổng nối nhánh hình dạng với nhánh SS2D. Từ S, một tích chập 1×1 có độ lệch, theo sau là hàm sigmoid, sinh ra bản đồ cổng G cùng kích thước với F_M. Đầu ra SS2D được điều biến như sau:

$$G = \sigma(W_g * S + b_g) \in (0,1)^{256\times20\times20}, \qquad F'_M = F_M \odot (1 + G) \quad (3.10)$$

Đặc điểm của cổng:

- **Chỉ khuếch đại.** Hệ số $(1 + G)$ luôn nằm trong khoảng (1, 2), nên độ lớn của mỗi phần tử F_M được giữ nguyên hoặc khuếch đại tối đa gấp đôi, không bao giờ bị giảm và không thể tắt hẳn một vị trí.
- **Vì sao dùng $(1 + G)$ thay cho G:** để tránh trường hợp cổng gần 0 lúc mới huấn luyện làm mất tín hiệu của SS2D.
- **Tác dụng thực tế:** sau bước hợp nhất còn một lớp chuẩn hóa lô. Vì vậy tác dụng của cổng là thay đổi tầm quan trọng tương đối giữa các vị trí và các kênh, chứ không phải tăng biên độ tuyệt đối.
- **Thông tin đi một chiều:** nhánh hình dạng điều khiển nhánh SS2D, còn nhánh SS2D không tác động ngược lại.

Tên mô-đun trong mã là `TopologyShapeGate`. Chữ "topology" chỉ thể hiện ý đồ thiết kế: dùng đặc trưng hình dạng và hướng để dẫn dắt đặc trưng toàn cục. Đề tài không dùng hàm mất mát hay ràng buộc tô-pô nào. Vì vậy không có căn cứ để nói mô hình bảo toàn tô-pô của mặt nạ.

3.3.5. Hợp nhất, mạng truyền thẳng và nối tắt có hệ số

Phần cuối của khối TSVMamba xử lý theo bốn bước:

1. Ghép F'_M với S, rồi dùng tích chập 1×1 (kèm chuẩn hóa lô và SiLU) đưa 512 kênh về 256 kênh.
2. Cho kết quả đi qua FFN gồm hai tích chập 1×1: tích chập đầu nâng 256 lên 512 kênh, có kích hoạt; tích chập sau hạ 512 về 256 kênh, không kích hoạt.
3. Nhân đầu ra với hệ số học được γ.
4. Cộng vào đặc trưng vào b.

$$F = \mathrm{FFN}\big(\mathrm{Conv}_{1\times1}^{fuse}([F'_M \parallel S])\big), \qquad b' = b + \gamma\, F, \qquad \gamma_0 = 10^{-3} \quad (3.11)$$

γ là một số vô hướng, dùng chung cho mọi kênh, khởi tạo bằng 0,001. Vì giá trị khởi tạo nhỏ, ở đầu quá trình huấn luyện khối TSVMamba gần như không làm thay đổi đầu vào ($b' \approx b$). Đóng góp của khối mới chỉ tăng dần trong lúc huấn luyện. Nhờ vậy, các tầng phía sau đã được tiền huấn luyện không bị xáo trộn đột ngột.

Gộp các công thức trên, khối TSVMamba viết gọn thành:

$$\mathrm{TSVMamba}(b) = b + \gamma \cdot \mathrm{FFN}\left(\mathrm{Conv}_{1\times1}^{fuse}([\,\mathrm{SS2D}(b) \odot (1 + G(b)) \;\|\; S(b)\,])\right) \quad (3.12)$$

3.3.6. Ví dụ truy vết kích thước tensor

Bảng 3.2 theo dõi một ảnh (lô 1) đi qua khối C2TSVMamba. Bảng giúp đối chiếu giữa các công thức và kích thước tensor trong mã.

Bảng 3.2: Kích thước tensor qua khối C2TSVMamba (một ảnh 640×640)

Bước	**Phép toán**	**Kích thước ra**
Vào tầng 10	x	512×20×20
cv1, chia đôi	Conv 1×1, split	a: 256×20×20; b: 256×20×20
SS2D, bước 1–2	in_proj, DWConv 3×3	U, Z: 256×20×20 mỗi phần
SS2D, bước 3	Trải phẳng	4 chuỗi, mỗi chuỗi 256×400
SS2D, bước 4	x_proj, dt_proj	Δ: 256×400; B, C: 16×400 (mỗi hướng)
SS2D, bước 5	$\bar{A}$, ΔBu, quét	256×16×400 (đệm lên 512 khi quét)
SS2D, bước 5–7	Đọc ra, gộp 4 hướng, cổng Z, out_proj	F_M: 256×20×20
Nhánh hình dạng	(3.7)–(3.9)	S: 256×20×20
Cổng, điều biến	(3.10)	G, F'_M: 256×20×20
fuse, FFN, nối tắt	(3.11)	b': 256×20×20
Ghép, cv2	Concat, Conv 1×1	y: 512×20×20

3.3.7. Số tham số

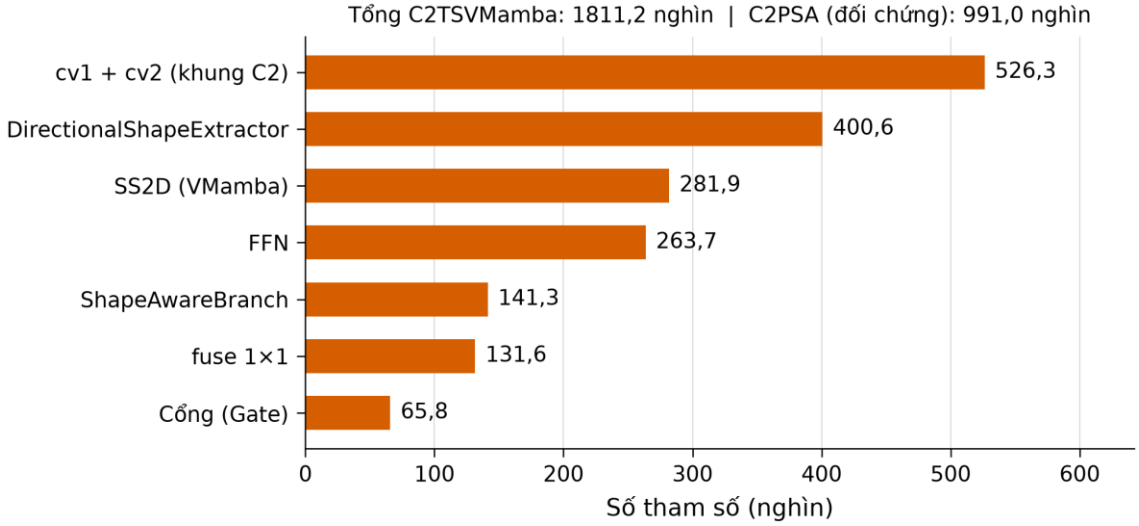
Bảng 3.3 liệt kê số tham số của từng phần trong tầng 10. Các số này được đếm khi dựng lại mô hình, rồi tính lại bằng tay theo kích thước từng lớp; hai cách cho cùng kết quả.

Bảng 3.3: Số tham số của tầng 10 (quy mô s)

Thành phần	**Số tham số**	**Tỉ lệ trong tầng 10**
cv1 + cv2 (khung C2)	526.336	29,1%
SS2D	281.856	15,6%
ShapeAwareBranch	141.312	7,8%
DirectionalShapeExtractor	400.640	22,1%
Cổng	65.792	3,6%
fuse	131.584	7,3%
FFN	263.680	14,6%

Thành phần	Số tham số	Tỉ lệ trong tầng 10
γ	1	~0%
Tổng C2TSVMamba	**1.811.201**	100%
C2PSA của mô hình đối chứng	990.976	

Nhánh hình dạng (ShapeAwareBranch và DirectionalShapeExtractor) cộng với cổng chiếm khoảng 33,5% số tham số của tầng, nhiều hơn chính nhánh SS2D. Phần lớn tham số của DirectionalShapeExtractor nằm ở tích chập 1×1 trộn 1.024 kênh (262.656 tham số). Như vậy, phần tham số tăng thêm so với C2PSA (820.225) không chỉ đến từ SS2D. Hình 3.5 thể hiện cơ cấu này.



Hình 3.5: Số tham số của từng thành phần trong khối C2TSVMamba. Tiêu đề hình ghi tổng của C2TSVMamba và của C2PSA để so sánh.

3.4. THUẬT TOÁN QUÉT KẾT HỢP SONG SONG

3.4.1. Đưa phép truy hồi về phép quét

Xét riêng một kênh d và một chiều trạng thái n, phương trình (3.4) có dạng truy hồi tuyến tính bậc một:

$$h_t = a_t\, h_{t-1} + b_t, \qquad h_0 = 0, \qquad t = 1, \dots, L \tag{3.13}$$

với $a_t = \bar{A}_t$ và $b_t = \Delta_t B_t u_t$.

Nếu tính tuần tự, phải thực hiện $L - 1$ bước nối tiếp, bước sau chờ kết quả bước trước. Trên GPU, $4 \times 256 \times 16 = 16.384$ chuỗi độc lập vẫn có thể chạy song song với nhau. Tuy vậy, vòng lặp theo thời gian buộc phải gọi khoảng 400 lượt phép toán nhỏ nối tiếp nhau. Chi phí gọi lặp này làm chậm đáng kể.

Để song song hóa theo chiều thời gian, ta coi mỗi vị trí là một cặp (a_t, b_t), biểu diễn phép biến đổi affine $h \mapsto a_t h + b_t$. Thực hiện liên tiếp hai phép biến đổi affine cho ra một phép biến đổi affine khác, ứng với toán tử:

$$(a_1, b_1) \circ (a_2, b_2) = (a_2 a_1,\ a_2 b_1 + b_2) \tag{3.14}$$

Có thể kiểm tra trực tiếp rằng toán tử này có tính kết hợp:

$$((a_1, b_1) \circ (a_2, b_2)) \circ (a_3, b_3) = (a_3 a_2 a_1,\ a_3 a_2 b_1 + a_3 b_2 + b_3)$$
$$= (a_1, b_1) \circ ((a_2, b_2) \circ (a_3, b_3))$$

Nhờ tính kết hợp, h_t chính là thành phần thứ hai của tích tiền tố $(a_1, b_1) \circ \cdots \circ (a_t, b_t)$. Mọi tích tiền tố có thể tính bằng thuật toán quét song song (parallel prefix scan) [45].

3.4.2. Thuật toán trong mã

Mã của đề tài dùng biến thể quét song song của Hillis và Steele [46]. Thuật toán chạy qua nhiều vòng:

– Ở vòng thứ j, mỗi vị trí t kết hợp kết quả hiện có của nó với kết quả của vị trí $t - 2^j$.

– Sau $\lceil \log_2 L \rceil$ vòng, mỗi vị trí đã gom được toàn bộ các phần tử đứng trước nó.

Thuật toán 3.1 viết lại hàm `parallel_associative_scan` dưới dạng giả mã.

Thuật toán 3.1. Quét kết hợp song song (theo hàm `parallel_associative_scan`)

```
Đầu vào : a[1..L], b[1..L]   (xử lý cùng lúc cho mọi kênh d, chiều
trạng thái n, hướng k)
Đầu ra  : h[1..L] với h[t] = a[t]·h[t-1] + b[t], h[0] = 0

1. K ← ⌈log2 L⌉ ;  P ← 2^K
2. Đệm a đến độ dài P bằng giá trị 1, đệm b đến độ dài P bằng giá trị
0
   (cặp (1, 0) là phần tử đơn vị của phép ∘, nên không làm sai kết
quả)
3. step ← 1
4. while step < P:
5.     với mọi t > step, đồng thời:
6.         b[t] ← a[t]·b[t − step] + b[t]
7.         a[t] ← a[t]·a[t − step]
8.     (các vị trí t ≤ step giữ nguyên)
9.     step ← 2·step
10. return b[1..L]
```

Dòng 6 và dòng 7 đều dùng giá trị của vòng trước ở vế phải. Vì vậy trong mã, hai dòng này được tính cùng lúc bằng phép cắt lát tensor rồi ghép lại (`torch.cat`). Với $L = 400$, mã đệm chuỗi lên $P = 512$ và chạy $K = 9$ vòng.

3.4.3. Ví dụ minh họa

Xét một chuỗi ngắn $L = 4$ với $a = (0{,}5;\ 0{,}8;\ 0{,}5;\ 0{,}9)$ và $b = (1;\ 2;\ 0;\ 1)$.

Tính tuần tự theo (3.13):

- $h_1 = 1$
- $h_2 = 0{,}8 \cdot 1 + 2 = 2{,}8$
- $h_3 = 0{,}5 \cdot 2{,}8 + 0 = 1{,}4$
- $h_4 = 0{,}9 \cdot 1{,}4 + 1 = 2{,}26$

Tính theo Thuật toán 3.1 ($K = 2$, không cần đệm) cho kết quả ở Bảng 3.4.

Bảng 3.4: Các vòng của phép quét song song trên ví dụ

Vòng	**Vị trí 1**	**Vị trí 2**	**Vị trí 3**	**Vị trí 4**
Ban đầu (a, b)	(0,5; 1)	(0,8; 2)	(0,5; 0)	(0,9; 1)
step = 1	(0,5; 1)	(0,4; 0,8·1 + 2 = **2,8**)	(0,4; 0,5·2 + 0 = 1)	(0,45; 0,9·0 + 1 = 1)
step = 2	(0,5; **1**)	(0,4; **2,8**)	(0,2; 0,4·1 + 1 = **1,4**)	(0,18; 0,45·2,8 + 1 = **2,26**)

Sau vòng cuối, thành phần b là $(1;\ 2{,}8;\ 1{,}4;\ 2{,}26)$, trùng với kết quả tính tuần tự.

Ở vòng step = 2, vị trí 4 kết hợp hai cặp:

- cặp của vị trí 2, đã chứa thông tin của vị trí 1 và 2;
- cặp của chính nó, đã chứa thông tin của vị trí 3 và 4.

Nhờ vậy, chỉ sau hai vòng, vị trí cuối đã gom đủ thông tin của cả bốn phần tử.

3.4.4. Độ phức tạp

Độ sâu. Thuật toán có độ sâu $\lceil \log_2 L \rceil$ vòng. Số bước phải chờ nhau giảm từ 399 bước nối tiếp xuống còn 9 vòng.

Tổng khối lượng tính. Ngược lại, tổng khối lượng tính tăng lên. Vòng thứ j xử lý $P - 2^j$ vị trí, nên cộng lại có

$$\sum_{j=0}^{K-1}(P - 2^j) = K \cdot P - (P - 1) \quad (3.15)$$

phép kết hợp. Với $P = 512$ và $K = 9$:

– Mỗi chuỗi cần 4.097 phép kết hợp. Nếu tính tuần tự chỉ cần 399 phép, tức ít hơn khoảng 10 lần.

– Mỗi ảnh có 16.384 chuỗi, nên phép quét tốn khoảng 67 triệu phép kết hợp.

– Mỗi phép kết hợp gồm hai phép nhân và một phép cộng (dòng 6 và 7 của Thuật toán 3.1).

Tổng công việc của biến thể này là $O(L\log L)$. Vì vậy không thể gọi bản cài đặt của đề tài là có độ phức tạp tuyến tính, dù Mamba gốc được giới thiệu là tuyến tính theo độ dài chuỗi. Mamba đạt mức tuyến tính nhờ hai yếu tố [23]:

– dùng biến thể quét hiệu quả về công việc của Blelloch [45], với tổng công việc $O(L)$;

– dùng kernel CUDA gộp các phép toán, và tính lại giá trị trung gian thay vì lưu lại.

Mã của đề tài không dùng các kỹ thuật này. Bù lại, bản cài đặt chỉ cần PyTorch, chạy được trên cả CPU lẫn GPU, và cho kết quả tất định.

Tương quan với attention. Ở $L = 400$, attention của C2PSA với chi phí $O(L^2)$ vẫn còn rẻ. Lợi thế về độ dài chuỗi của SSM chỉ thể hiện khi chuỗi rất dài. Nếu đặt khối ở P4 ($L = 1.600$) hay P3 ($L = 6.400$):

– số vòng quét tăng lên 11 và 13;

– số vị trí mỗi vòng tăng 4 và 16 lần, nếu giữ nguyên số kênh. Trên thực tế số kênh ở các mức này khác với P5.

3.4.5. Ước tính chi phí của tầng 10

Bảng 3.5 so sánh số phép nhân–cộng (MAC) trong phần xử lý chính của hai khối, tính tay theo kích thước từng lớp cho một ảnh 640×640. Đây là ước tính giải tích. Đề tài không báo cáo GFLOPs đo bằng công cụ, vì công cụ đếm tự động không đếm đúng phép quét tự viết.

Bảng 3.5: Ước tính chi phí phần xử lý chính của tầng 10. Không tính cv1, cv2 vì hai khối dùng chung, khoảng 210 triệu MAC.

Khối	**Thành phần**	**Triệu MAC**
PSABlock (C2PSA)	qkv 52,4; $QK^\top$ 20,5; nhân với V 41,0; proj 26,2; mã hóa vị trí 0,9; FFN 104,9	≈ 246
TSVMamba	SS2D: các phép chiếu và tích chập	≈ 112
	Nhánh hình dạng, cổng, fuse, FFN	≈ 399
	Cộng	**≈ 511**

Riêng phép quét song song không tính được bằng MAC. Như đã nêu ở mục 3.4.4, phép quét cần khoảng 67 triệu phép kết hợp, tức khoảng 200 triệu phép toán từng phần tử.

Theo ước tính này, phần xử lý chính của TSVMamba tốn gấp khoảng hai lần PSABlock. Phần lớn chi phí đó nằm ở nhánh hình dạng và FFN, không nằm ở SS2D. Tuy vậy, khoản tăng thêm chỉ khoảng 0,27 tỉ MAC, trong khi toàn mô hình theo ghi chú của Ultralytics cỡ 37 GFLOPs. Như vậy số phép tính của toàn mô hình chỉ tăng khoảng 1–2%.

Độ trễ thì tăng nhiều hơn hẳn. Trên CPU, mỗi ảnh mất khoảng 270 ms với mô hình đối chứng và 577 ms với mô hình đề xuất. Số liệu này đo với:

– chỉ chạy lan truyền xuôi (forward);

– 4 luồng xử lý;

– checkpoint của seed 0.

Chi tiết đo trình bày ở Chương 6. Như vậy, số phép tính không giải thích được mức tăng độ trễ khoảng hai lần. Phần lớn chênh lệch nhiều khả năng đến từ chi phí bộ nhớ của phép quét: mỗi vòng tạo các tensor trung gian kích thước 256×16×512 cho từng hướng, rồi ghép lại bằng `torch.cat`, lặp lại 9 vòng. Giả thuyết này chưa được kiểm chứng bằng công cụ đo hiệu năng (profiler).

3.5. KHỞI TẠO TRỌNG SỐ VÀ HỌC CHUYỂN GIAO

Cả hai mô hình đều xuất phát từ trọng số `yolo26s-seg.pt`, được tiền huấn luyện trên COCO [41].

– **Mô hình đối chứng** nạp thẳng tệp này.

– **Mô hình đề xuất** được dựng từ tệp cấu hình mới rồi mới nạp `yolo26s-seg.pt`. Ultralytics chỉ chép những tensor trùng cả tên lẫn kích thước.

Kết quả nạp trọng số ở mô hình đề xuất:

– **Tầng 0–9 và 11–23:** nhận trọng số tiền huấn luyện giống hệt mô hình đối chứng. Ngoại lệ là các lớp đầu ra phụ thuộc số lớp của đầu Segment26: COCO có 80 lớp còn dữ liệu polyp có 1 lớp, nên các lớp này không chép được ở cả hai mô hình.

– **Tầng 10, phần kế thừa:** 12 trong 58 tensor tham số trùng tên và kích thước với C2PSA, tương ứng 790.016 trong 1.811.201 tham số. Đó là cv1, cv2 và FFN. FFN chép được vì PSABlock của C2PSA cũng có một FFN cùng tên và cùng kích thước.

– **Tầng 10, phần khởi tạo mới:** 1.021.185 tham số, gồm SS2D, nhánh hình dạng, cổng, fuse và γ.

FFN của TSVMamba nhận trọng số từ FFN của PSABlock chỉ vì trùng tên. Trong khối mới, đầu vào của FFN đã khác: đó là đầu ra của fuse, không còn là đầu ra của attention. Vì vậy đây là khởi tạo kế thừa theo tên, không phải tái sử dụng có chủ đích.

Cách khởi tạo các tham số mới:

– $A_{\log}$ đặt bằng $\log(1), \dots, \log(16)$ cho mọi kênh và mọi hướng, tức $A = -(1,2, \dots ,16)$. Đây là cách khởi tạo S4D-Real [44] mà Mamba dùng.

– Các phép chiếu Δ, tích chập và chuẩn hóa lô dùng khởi tạo mặc định.

– $\gamma = 0{,}001$.

Vì γ nhỏ, ở những bước đầu tầng 10 hoạt động gần như $\mathrm{cv2}([a \parallel b])$, với cv1 và cv2 lấy từ C2PSA. Nói cách khác, mô hình đề xuất bắt đầu huấn luyện khi tầng 10 chưa có cơ chế tổng hợp ngữ cảnh nào, rồi mới học dần đóng góp của khối mới.

3.6. HÀM MẤT MÁT VÀ CHIẾN LƯỢC HUẤN LUYỆN

Đề tài giữ nguyên hàm mất mát của YOLO26-seg. Hai mô hình dùng chung hàm mất mát, bộ siêu tham số và danh sách seed. Vì vậy, khác biệt có hệ thống giữa hai mô hình chỉ có thể quy cho tầng 10. Dao động ngẫu nhiên giữa các lượt

huấn luyện (do khởi tạo, thứ tự dữ liệu và phần cứng được cấp) được xử lý bằng cách lặp lại trên 20 seed ghép cặp.

Hai đầu dự đoán và trọng số thay đổi theo epoch. YOLO26 huấn luyện đồng thời hai đầu [6], [20]:

– **Đầu một-nhiều:** mỗi đối tượng thật được gán nhiều điểm neo, giúp mô hình học nhanh.

– **Đầu một-một:** mỗi đối tượng thật chỉ được gán một điểm neo. Đây là đầu dùng khi suy luận.

Tổng mất mát là tổ hợp có trọng số của hai đầu:

$$\mathcal{L} = \alpha(e)\,\mathcal{L}_{\text{o2m}} + (1-\alpha(e))\,\mathcal{L}_{\text{o2o}}, \qquad \alpha(e) = 0{,}1 + 0{,}7 \cdot \max\left(1 - \frac{e}{E-1},\ 0\right) \tag{3.16}$$

Với $E = 100$ epoch, trọng số của đầu một-nhiều giảm tuyến tính từ 0,8 xuống 0,1. Như vậy, càng về cuối, mô hình càng tập trung vào đầu dùng khi suy luận. Đặc trưng đi vào đầu một-một được tách khỏi đồ thị tính gradient, nên gradient của đầu này không chảy ngược về backbone.

Gán nhãn. Điểm neo dương được chọn theo độ đo căn chỉnh nhiệm vụ của TOOD [37]: $t = s^{0,5} \cdot u^{6}$, với s là điểm lớp dự đoán và u là IoU của hộp.

– Đầu một-nhiều lấy 10 điểm neo tốt nhất cho mỗi đối tượng.

– Đầu một-một chọn 7 ứng viên, rồi giữ lại 1 điểm neo tốt nhất.

YOLO26 bổ sung quy tắc STAL [6]: khi chọn ứng viên, đối tượng có cạnh nhỏ hơn 8 pixel được nới cạnh lên 16 pixel, để vật nhỏ không bị bỏ sót.

Các thành phần mất mát. Bảng 3.6 liệt kê các thành phần mất mát của mỗi đầu.

Bảng 3.6: Các thành phần mất mát của YOLO26-seg (theo đúng nhánh mã đã chạy)

Thành phần	**Nội dung**	**Hệ số**
Hộp (box)	$1 - \text{CIoU}$ [38], có trọng số theo điểm căn chỉnh	7,5
L1	Sai số tuyệt đối trên bốn khoảng cách tới biên hộp. Thay cho DFL vì `reg_max = 1` [6]	1,5
Lớp (cls)	Entropy chéo nhị phân, dùng nhãn mềm từ bộ gán nhãn	0,5

Thành phần	Nội dung	Hệ số
Mặt nạ (seg)	Entropy chéo nhị phân theo pixel giữa mặt nạ dự đoán và mặt nạ thật, chỉ tính trong hộp thật	7,5
Ngữ nghĩa (sem)	0,5·BCE + 0,5·Dice trên nhánh phân đoạn ngữ nghĩa phụ của Proto26. Chỉ chạy khi huấn luyện	7,5

Mặt nạ của mỗi đối tượng là tổ hợp tuyến tính của 32 mặt nạ mẫu, dùng 32 hệ số riêng của đối tượng đó: $\widehat{M}_i = \sigma(\sum_{k=1}^{32} c_{ik}\, P_k)$. Sau đó mặt nạ được cắt theo hộp. Vì nhánh ngữ nghĩa phụ chỉ chạy ở chế độ huấn luyện, cột `val/sem_loss` trong nhật ký luôn bằng 0. Đây là hành vi đúng của mã, không phải lỗi.

Bộ tối ưu và độ chính xác số. Bài báo YOLO26 dùng bộ tối ưu MuSGD [6], còn đề tài dùng AdamW [40] cho cả hai mô hình. Cấu hình huấn luyện:

– tốc độ học ban đầu 0,001; hệ số suy giảm trọng số 0,0005;

– huấn luyện hoàn toàn ở độ chính xác FP32 (`amp=False`);

– 100 epoch, lô 8 ảnh;

– bật chế độ tất định, mỗi lượt huấn luyện dùng một seed cố định.

Chi tiết cấu hình và quy trình huấn luyện trình bày ở Chương 5.

Đề tài không thêm hàm mất mát nào về biên hay tô-pô. Nhánh hình dạng chỉ có thể tác động gián tiếp, thông qua các thành phần mất mát chuẩn ở Bảng 3.6.

3.7. QUY TRÌNH SUY LUẬN VÀ HUẤN LUYỆN CỦA MÔ HÌNH ĐỀ XUẤT

Thuật toán 3.2 tóm tắt hai quy trình của mô hình đề xuất: suy luận trên một ảnh, và một bước huấn luyện. Ở mô hình đối chứng, quy trình giống hệt; chỉ khác là bước 3 dùng C2PSA.

Thuật toán 3.2. Suy luận và huấn luyện YOLO26s-seg + C2TSVMamba

```
SUY LUẬN (một ảnh I)
1. Letterbox: co ảnh để cạnh dài bằng 640, giữ tỉ lệ, thêm viền xám
2. Backbone tầng 0–9: trích đặc trưng, ra 512×20×20 tại tầng 9
3. Tầng 10 – C2TSVMamba: cv1 → chia (a, b) → TSVMamba(b) theo (3.12) →
ghép → cv2
4. Neck tầng 11–22: FPN đi xuống, PAN đi lên → P3, P4, P5
5. Segment26, đầu một-một: với mỗi điểm neo dự đoán hộp, điểm lớp, 32
hệ số mặt nạ;
   Proto26 sinh 32 mặt nạ mẫu 160×160
6. Lấy 300 dự đoán có điểm cao nhất, bỏ các dự đoán dưới ngưỡng tin
cậy (không NMS)
7. Với mỗi dự đoán còn lại: tính Σ c_k·P_k, phóng về kích thước ảnh,
```

```
   nhị phân hóa (sigmoid > 0,5), rồi cắt theo hộp
8. Trả về danh sách (hộp, điểm tin cậy, mặt nạ)

MỘT BƯỚC HUẤN LUYỆN (lô ảnh đã tăng cường, epoch e)
1. Lan truyền xuôi như bước 2–5, cho cả hai đầu một-nhiều và một-một
2. Gán nhãn: TAL + STAL, top-10 cho đầu một-nhiều, top-7 rồi giữ 1 cho
đầu một-một
3. Tính mất mát mỗi đầu theo Bảng 3.6; tổng hợp theo (3.16) với α(e)
4. Lan truyền ngược, cập nhật trọng số bằng AdamW; cập nhật bản trung
bình động (EMA) của trọng số
5. Cuối epoch: kiểm định trên tập val bằng trọng số EMA, lưu best.pt
khi chỉ số tốt hơn
```

3.8. LƯU TRỌNG SỐ Ở DẠNG FP32

Sự cố. Trong lần huấn luyện đầu tiên, nhóm gặp lỗi khi lưu checkpoint:

– Mặc định, Ultralytics chuyển mô hình sang FP16 trước khi lưu tệp `best.pt`.

– Ở seed 5, giá trị trung bình chạy (`running_mean`) của lớp chuẩn hóa lô trong tích chập fuse ở tầng 10 lên tới khoảng 1,5 triệu. Con số này vượt giới hạn 65.504 của FP16.

– Khi lưu, giá trị bị tràn nên checkpoint cho kết quả sai khi suy luận, dù quá trình huấn luyện ở FP32 vẫn diễn ra bình thường.

Cách sửa.

– Khi lưu checkpoint (ở hàm lưu mô hình, ở MultiTrainer và ở hàm `strip_optimizer`), mô hình được giữ ở FP32 bằng `.float()` thay cho `.half()`.

– Mã gốc có xử lý tensor không hữu hạn trong bản EMA: thay bằng tensor tương ứng của mô hình đang huấn luyện, rồi gọi `nan_to_num_`. Mã sau khi sửa bỏ cách xử lý này. Nếu bản EMA còn giá trị NaN hoặc vô cùng, hàm lưu mô hình và hàm `strip_optimizer` sẽ báo lỗi ngay.

Kiểm tra lại. Các lượt TSVM được huấn luyện lại bằng mã đã sửa. So sánh tệp `results.csv` của hai lần chạy trên 20 seed (seed 0–19):

– 44.000 ô số liệu giống hệt nhau;

– 2.000 ô khác nhau, tất cả đều thuộc cột thời gian.

Như vậy bản sửa không làm thay đổi quá trình huấn luyện, chỉ làm checkpoint được lưu đúng. Tên “TSVM FP32” trong đề tài chỉ cách lưu checkpoint này, không phải một chế độ huấn luyện riêng.

Nguyên nhân có thể có. Nguyên nhân có thể nằm ở chính thiết kế. SS2D không có lớp chuẩn hóa ở đầu ra, và F_M còn được nhân thêm hệ số $(1 + G)$, tối đa gần 2, trước khi vào fuse. Vì vậy biên độ đặc trưng trước lớp chuẩn hóa lô của fuse không bị chặn. Đây mới là giả thuyết; đề tài chưa đo phân bố giá trị kích hoạt để kiểm chứng.

3.9. ƯU ĐIỂM VÀ HẠN CHẾ CỦA MÔ HÌNH ĐỀ XUẤT

Phần này đánh giá mô hình đề xuất so với mô hình đối chứng. Ưu và nhược điểm chung của YOLO26-seg và của VMamba đã phân tích ở Chương 2.

Ưu điểm

- **Thay đổi tối thiểu.** Chỉ một tầng khác nhau và giao diện vào/ra giữ nguyên. Nhờ vậy, khác biệt có hệ thống giữa hai mô hình quy được cho khối mới, thay vì cho nhiều thay đổi diễn ra cùng lúc.
- **Ngữ cảnh toàn cục có chọn lọc.** SS2D quét theo bốn hướng, nên mỗi vị trí nhận thông tin từ cả bốn phía. Mức ghi nhớ được điều chỉnh theo nội dung qua Δ, B, C.
- **Thông tin hình dạng dẫn dắt đặc trưng toàn cục.** Nhánh tích chập có hướng (1×5, 5×1) cùng cổng $(1 + G)$ là một kênh riêng để thông tin hình dạng tác động lên đặc trưng toàn cục. C2PSA chỉ có tích chập 3×3 làm mã hóa vị trí.
- **Khởi đầu ổn định.** Hệ số $\gamma = 0{,}001$, cùng với việc kế thừa cv1, cv2 và FFN từ C2PSA, giúp mô hình bắt đầu từ trạng thái gần với mô hình gốc.
- **Dễ chạy ở nhiều nơi.** Bản cài đặt chỉ cần PyTorch, không phải biên dịch kernel CUDA, chạy được trên CPU và cho kết quả tất định.

Hạn chế

- **Chi phí tăng.**
  + Số tham số toàn mô hình tăng 7,17%.
  + Tệp checkpoint lớn hơn do lưu ở FP32.
  + Độ trễ trên CPU tăng khoảng 2,1 lần.
  + Thời gian huấn luyện mỗi lượt tăng từ khoảng 2,3 giờ lên 4,7 giờ.
- **Phép quét chưa tối ưu.** Biến thể Hillis–Steele có tổng công việc $O(L\log L)$ và tạo nhiều tensor trung gian.

– **Ở mức P5, lợi thế của SSM chưa thể hiện.** Với $L = 400$, attention vẫn rẻ, nên ưu điểm của SSM với chuỗi dài chưa có dịp phát huy.

– **Khác bản VMamba gốc.** Mã thiếu hạng tử nhảy D, thiếu LayerNorm ở đầu ra, và khởi tạo Δ theo mặc định. Những điểm này có thể liên quan đến sự cố số học ở mục 3.8.

– **Tên gọi chưa có bằng chứng đi kèm.** Chữ "topology" là ý đồ thiết kế. Khi đo lỗi Betti (số mảnh, số lỗ của mặt nạ) trên 20 seed, TSVM chỉ nhích hơn mô hình gốc và khác biệt không có ý nghĩa thống kê (Chương 6).

– **Đóng góp của từng nhánh mới ở mức thăm dò.** Thí nghiệm tách thành phần (mục 3.10, kết quả ở mục 6.6) chỉ có 5 seed và các chế độ khác nhau về số tham số.

3.10. CÁC CHẾ ĐỘ TÁCH THÀNH PHẦN CÓ SẴN TRONG MÃ

Mô-đun TSVMamba có tham số `mode` để bật hoặc tắt từng nhánh. Bảng 3.7 liệt kê bốn chế độ và số tham số tương ứng của tầng 10. Số tham số của ba chế độ rút gọn được tính bằng cách cộng trừ các thành phần ở Bảng 3.3, rồi đếm lại từ checkpoint của các lượt tách thành phần; hai cách cho cùng kết quả.

Bảng 3.7: Bốn chế độ của khối TSVMamba

Chế độ	**Công thức khối**	**Tham số tầng 10**
`topology_shape_vmamba` (đầy đủ, dùng trong đề tài)	(3.12)	1.811.201
`shape_vmamba_no_guidance`	$b + \gamma \cdot$ FFN(fuse([SS2D(b); S])), bỏ cổng	1.745.409
`shape_only`	$b + \gamma \cdot \mathrm{FFN}(S)$, bỏ SS2D	1.331.969
`vmamba_only`	$b + \gamma \cdot \mathrm{FFN}(\mathrm{SS2D}(b))$, bỏ nhánh hình dạng	1.071.873

Mọi lượt huấn luyện chính của TSVM đều dùng chế độ đầy đủ. Ba chế độ còn lại được huấn luyện với seed 0–4, cùng seed với mô hình chính để so sánh ghép cặp; kết quả ở mục 6.6. Vì các chế độ chênh nhau tới khoảng 740 nghìn tham

số, so sánh này chưa tách được phần do cấu trúc với phần do thêm tham số; muốn tách cần thêm một mô hình đối chứng có số tham số tương đương.

3.11. KẾT CHƯƠNG

Chương 3 đã trình bày mô hình YOLO26s-seg + C2TSVMamba. Mô hình giữ nguyên YOLO26s-seg, chỉ thay khối C2PSA ở tầng 10 bằng khối C2TSVMamba có cùng giao diện 512×20×20. Bên trong khối có ba phần chính:

- **Nhánh SS2D:** cài đặt lại theo VMamba, quét đặc trưng theo bốn hướng bằng mô hình không gian trạng thái chọn lọc.
- **Nhánh hình dạng và cổng:** tích chập có hướng sinh đặc trưng hình dạng và một cổng $(1 + G)$ để khuếch đại đầu ra SS2D.
- **Hợp nhất và nối tắt:** hai nhánh được hợp nhất, rồi cộng trở lại vào đặc trưng vào qua hệ số γ khởi tạo rất nhỏ.

Phương trình truy hồi của SSM được giải bằng phép quét kết hợp song song kiểu Hillis–Steele. Phép quét này có độ sâu 9 vòng nhưng tổng công việc là $O(L\log L)$.

So với C2PSA, khối mới thêm 820.225 tham số. Số phép tính ở tầng 10 tăng khoảng gấp đôi, nhưng chỉ chiếm khoảng 1–2% chi phí toàn mô hình. Độ trễ trên CPU thì tăng khoảng hai lần, chủ yếu do cách cài đặt phép quét. Hàm mất mát, dữ liệu và siêu tham số giữ giống hệt mô hình đối chứng.

Các chương tiếp theo:

- Chương 4 trình bày dữ liệu dùng để huấn luyện.
- Chương 5 mô tả phần cài đặt.
- Chương 6 đánh giá xem mức chi phí tăng thêm này đổi lại được bao nhiêu về chất lượng phân đoạn.

CHƯƠNG 4: XÂY DỰNG BỘ DỮ LIỆU VÀ TIỀN XỬ LÝ DỮ LIỆU

Chất lượng dữ liệu đầu vào ảnh hưởng trực tiếp đến khả năng học của mô hình phân đoạn. Chương này trình bày cách đề tài xây dựng bộ dữ liệu BG20 từ hai nguồn ảnh nội soi công khai và các bước đưa ảnh cùng nhãn về dạng mà mô hình YOLO26-seg tích hợp khối TSVM sử dụng được. Quá trình gồm hai giai đoạn. Giai đoạn chuẩn bị chỉ thực hiện một lần trước khi huấn luyện, gồm chọn ảnh, chia tập và chuyển mặt nạ thành nhãn đa giác. Giai đoạn nạp dữ liệu diễn ra mỗi khi ảnh được đưa vào mô hình, gồm đổi kích thước, tăng cường dữ liệu và chuẩn hóa.

4.1. NGUỒN DỮ LIỆU

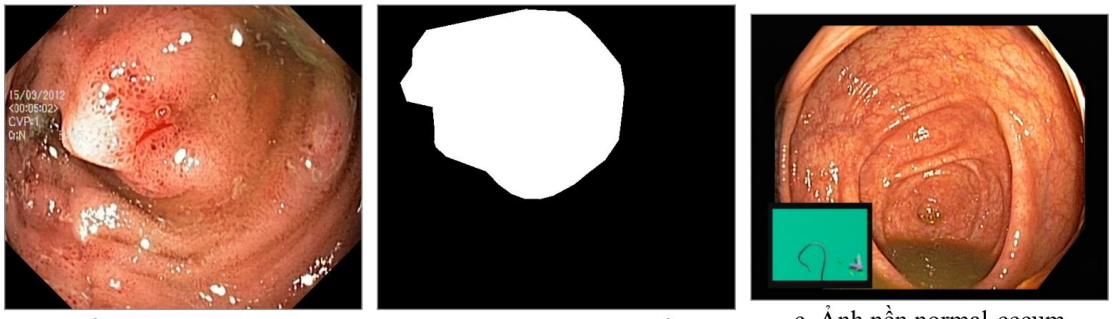
4.1.1. Bộ dữ liệu Kvasir-SEG

Kvasir-SEG là bộ dữ liệu công khai gồm 1.000 ảnh nội soi đường tiêu hóa có polyp, do Jha và cộng sự công bố năm 2020 [5]. Mỗi ảnh đi kèm một mặt nạ do kỹ sư và bác sĩ khoanh vùng, sau đó được bác sĩ chuyên khoa tiêu hóa kiểm tra lại. Trên mặt nạ, pixel thuộc polyp có màu trắng, các pixel còn lại có màu đen.

Dữ liệu có ba đặc điểm cần xử lý. Thứ nhất, kích thước ảnh không đồng nhất, gồm 333 kích thước khác nhau với chiều rộng từ 332 đến 1.920 pixel. Thứ hai, kích thước polyp chênh lệch lớn, vùng nhỏ nhất chỉ chiếm 0,15% diện tích ảnh, vùng lớn nhất chiếm khoảng 81% và gần một nửa số vùng (khoảng 48%) chiếm không quá 10%. Thứ ba, cả ảnh và mặt nạ đều được lưu ở định dạng nén JPEG [5], nên dọc theo biên polyp xuất hiện các pixel có mức xám trung gian và mặt nạ cần được nhị phân hóa lại như trình bày ở mục 4.3.

4.1.2. Ảnh nền normal-cecum

Bộ dữ liệu Kvasir phiên bản 2 gồm 8 lớp ảnh, mỗi lớp 1.000 ảnh [5], [8]. Đề tài dùng lớp normal-cecum, tức ảnh manh tràng bình thường có kích thước 720 × 576 pixel, làm ảnh nền. Ảnh nền là mẫu âm tính, không chứa polyp. Nếu chỉ học trên ảnh có polyp, mô hình dễ cho rằng ảnh nào cũng có polyp và khoanh nhầm các nếp gấp hay vùng phản quang. Việc bổ sung ảnh nền nhằm giúp mô hình học được trường hợp không có đối tượng cần phân đoạn. Hình 4.1 minh họa hai nguồn dữ liệu này.



a. Ảnh nội soi có polyp b. Mặt nạ phân đoạn gốc c. Ảnh nền normal-cecum

Hình 4.1: Ví dụ ảnh có polyp, mặt nạ phân đoạn gốc và ảnh nền trong bộ dữ liệu BG20

Nguồn: ảnh a và b từ bộ dữ liệu Kvasir-SEG [5], ảnh c từ lớp normal-cecum của bộ dữ liệu Kvasir [8], đề tài ghép và đặt chú thích.

4.2. XÂY DỰNG VÀ PHÂN CHIA BỘ DỮ LIỆU BG20

Ảnh polyp được chia theo danh sách có sẵn gồm 880 ảnh huấn luyện và 120 ảnh kiểm định, do nhóm tác giả Kvasir-SEG cung cấp kèm bản phát hành trên Kaggle trong hai tệp train.txt và val.txt [53]. Dùng danh sách có sẵn giúp kết quả của đề tài so sánh được với các nghiên cứu dùng cùng cách chia. Với ảnh nền, chương trình sắp xếp 1.000 tên tệp normal-cecum, cố định seed bằng 42 rồi chọn ngẫu nhiên 200 ảnh, gồm 160 ảnh cho tập huấn luyện và 40 ảnh cho tập kiểm định. Nhờ cố định seed, mỗi lần chạy lại đều cho đúng 200 ảnh như nhau.

Tên BG20 thể hiện số ảnh nền bằng 20% số ảnh polyp. Tính trên toàn bộ 1.200 ảnh, ảnh nền chiếm 16,67%. Phân bố bộ dữ liệu được tổng hợp trong Bảng 4.1.

Bảng 4.1: Phân bố ảnh và nhãn trong bộ dữ liệu BG20

Thành phần	**Huấn luyện**	**Kiểm định**	**Tổng**
Ảnh có polyp	880	120	1.000
Ảnh nền normal-cecum	160	40	200
Tổng số ảnh	1.040	160	1.200
Số vùng polyp được gán nhãn	936	127	1.063

Nguồn: đề tài thống kê từ bộ dữ liệu BG20.

Số vùng polyp lớn hơn số ảnh polyp vì một ảnh có thể chứa nhiều polyp tách rời, mỗi polyp được gán một nhãn riêng. Bộ dữ liệu được tổ chức theo định dạng Ultralytics với hai thư mục images và labels, mỗi ảnh có một tệp nhãn văn bản cùng tên. Dữ liệu chỉ có một lớp là polyp với chỉ số 0. Ảnh nền đi kèm một tệp

nhãn rỗng để cho mô hình biết ảnh không chứa polyp, vì vậy ảnh nền không phải là một lớp thứ hai.

4.3. CHUYỂN MẶT NẠ PHÂN ĐOẠN THÀNH NHÃN ĐA GIÁC

Mặt nạ của Kvasir-SEG là ảnh, trong khi YOLO26-seg yêu cầu nhãn dạng văn bản. Mỗi dòng nhãn gồm chỉ số lớp và tọa độ các đỉnh của đa giác bao quanh đối tượng, chuẩn hóa về đoạn từ 0 đến 1 [42]. Đề tài chuyển mỗi mặt nạ thành nhãn đa giác qua năm bước dưới đây bằng thư viện OpenCV, còn ảnh nội soi được giữ nguyên.

4.3.1. Nhị phân hóa mặt nạ bằng phương pháp Otsu

Mặt nạ được đọc thành ảnh xám có giá trị từ 0 đến 255. Do nén JPEG, mặt nạ có thêm các mức xám trung gian, ví dụ mặt nạ trong Hình 4.2 có 15 mức xám. Nhị phân hóa là chọn một ngưỡng t, pixel lớn hơn t được gán 255 và thuộc polyp, các pixel còn lại được gán 0. Phương pháp Otsu [54] chọn t tự động bằng cách thử mọi ngưỡng và giữ lại ngưỡng làm phương sai giữa nhóm tối và nhóm sáng lớn nhất:

$$\sigma_B^2(t) = \omega_0(t)\,\omega_1(t)\,[\mu_0(t) - \mu_1(t)]^2 \tag{4.1}$$

$$t^* = \underset{0 \le t \le 255}{\operatorname{argmax}}\ \sigma_B^2(t) \tag{4.2}$$

Trong đó, ω_0 và ω_1 là tỷ lệ số pixel của nhóm tối và nhóm sáng, μ_0 và μ_1 là mức xám trung bình của hai nhóm. Trên 1.000 mặt nạ của đề tài, ngưỡng t^* chỉ nằm trong khoảng từ 5 đến 8. Điều này cho thấy mặt nạ gốc gần như đã nhị phân và bước Otsu chủ yếu loại bỏ nhiễu do nén ảnh.

4.3.2. Làm liền vùng polyp bằng phép đóng hình thái học

Sau khi nhị phân hóa, mặt nạ có thể còn lỗ nhỏ bên trong vùng polyp hoặc khe hở mảnh trên đường biên. Đề tài dùng phép đóng (Closing) của hình thái học [52] để làm liền các chỗ này:

$$A \bullet B = (A \oplus B) \ominus B \tag{4.3}$$

Trong đó, A là tập pixel trắng của mặt nạ, B là phần tử cấu trúc hình vuông 3 × 3 pixel. Phép giãn nở $\oplus$ làm vùng trắng phình ra một pixel để lấp lỗ và khe, sau đó phép co $\ominus$ thu vùng trắng lại một pixel để biên trở về gần vị trí cũ. Nhờ vậy, vùng polyp liền mạch hơn nhưng kích thước gần như không đổi.

4.3.3. Tìm đường bao và loại bỏ vùng nhiễu nhỏ

OpenCV tìm đường bao bằng thuật toán dò biên của Suzuki và Abe [55]. Đề tài chỉ lấy đường bao ngoài cùng của mỗi vùng, vì nhãn YOLO mô tả mỗi đối tượng bằng một đa giác khép kín. Các đường bao có diện tích nhỏ hơn 20 pixel vuông bị loại vì thường là đốm nhiễu. Trên 1.000 mặt nạ, chương trình tìm được 1.206 đường bao, loại 143 và còn đúng 1.063 vùng polyp như trong Bảng 4.1.

4.3.4. Đơn giản hóa đường bao

Đường bao bám theo từng pixel trên biên nên trung bình có khoảng 333 điểm cho mỗi polyp. Để nhãn gọn hơn, đề tài dùng thuật toán Douglas–Peucker [56]. Thuật toán chỉ giữ lại những điểm lệch khỏi đoạn thẳng nối hai đầu đường cong nhiều hơn ngưỡng sai số ε, với ε được tính theo chu vi đường bao:

$$\varepsilon = 0{,}002 \cdot L \tag{4.4}$$

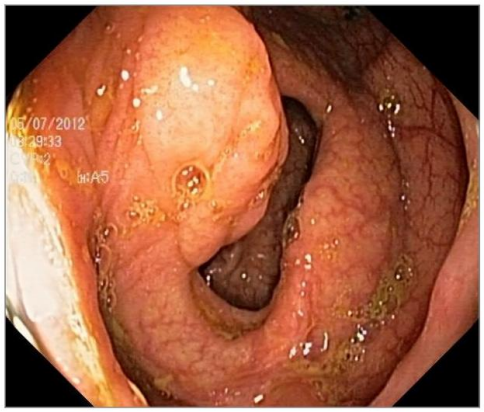
Trong đó, L là chu vi đường bao tính bằng pixel. Ví dụ, polyp có chu vi 1.000 pixel sẽ có ε bằng 2 pixel. Cách tính theo chu vi giúp polyp lớn và nhỏ được rút gọn tương xứng. Kết quả, số điểm trung bình của mỗi đa giác giảm từ 333,2 xuống 24,6, tức giảm 92,6%, và mỗi đa giác có từ 10 đến 80 đỉnh.

4.3.5. Chuẩn hóa tọa độ và ghi tệp nhãn

Cuối cùng, tọa độ mỗi đỉnh được chia cho chiều rộng W và chiều cao H của mặt nạ để đưa về đoạn từ 0 đến 1:

$$\hat{x} = \frac{x}{W}, \qquad \hat{y} = \frac{y}{H} \tag{4.5}$$

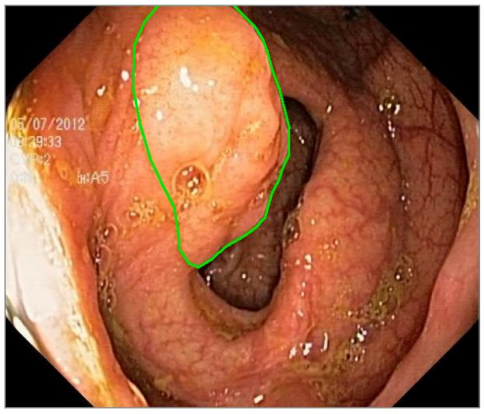
Giá trị sau chuẩn hóa được làm tròn đến 6 chữ số thập phân. Mỗi polyp được ghi thành một dòng gồm chỉ số lớp, tiếp theo là các cặp tọa độ $\hat{x}$ và $\hat{y}$ của từng đỉnh. Nhờ chuẩn hóa, nhãn không phụ thuộc kích thước ảnh, nên khi ảnh được thu phóng ở các bước sau, đa giác vẫn khớp đúng vị trí polyp. Hình 4.2 minh họa toàn bộ công đoạn chuyển đổi trên một ảnh của tập huấn luyện.



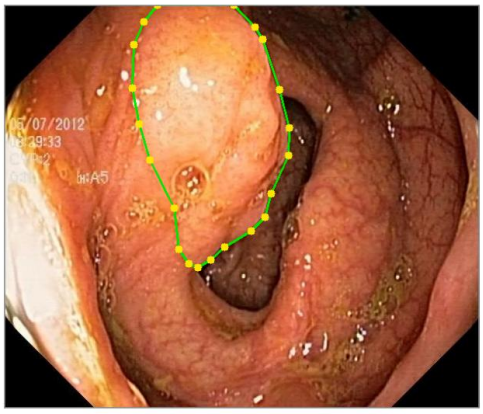
a. Ảnh nội soi gốc



b. Mặt nạ gốc trước xử lý



c. Đường bao ban đầu: 377 điểm



d. Đa giác đơn giản hóa: 21 điểm

Hình 4.2: Trước và sau khi chuyển mặt nạ phân đoạn thành nhãn đa giác

Nguồn: ảnh a và b từ bộ dữ liệu Kvasir-SEG [5], ảnh c và d do đề tài tạo bằng chương trình chuyển đổi mặt nạ của đề tài.

Đường bao ở ảnh c gồm 377 điểm bám sát biên polyp. Sau khi đơn giản hóa, đa giác ở ảnh d chỉ còn 21 đỉnh nhưng vẫn giữ được hình dạng polyp.

4.4. ĐỔI KÍCH THƯỚC ẢNH

Từ mục này trở đi, các bước xử lý diễn ra mỗi khi ảnh được nạp vào mô hình. Mô hình dùng kích thước đầu vào 640 pixel. Nếu kéo giãn trực tiếp ảnh thành hình vuông, polyp sẽ bị méo. Vì vậy, ảnh được thu phóng giữ nguyên tỷ lệ sao cho cạnh dài nhất bằng 640 pixel:

$$r = \frac{640}{\max(W,H)} \tag{4.6}$$

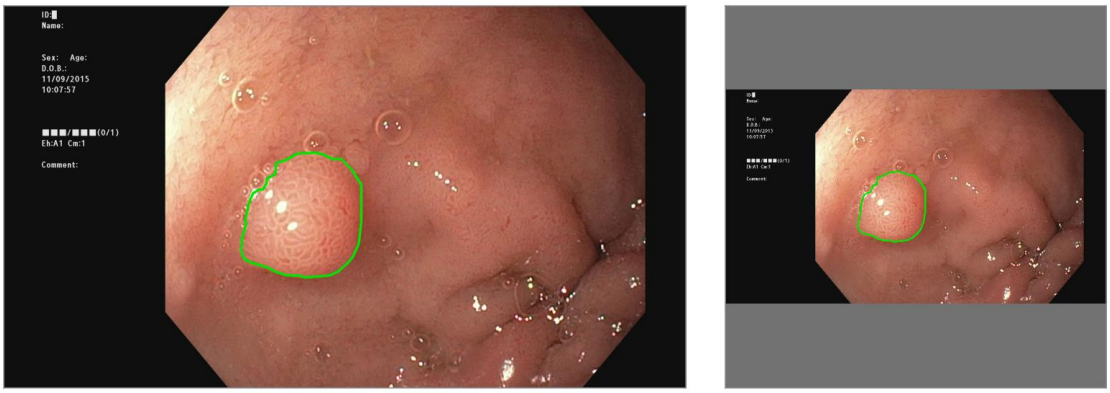
$$W' = \lceil r \cdot W \rceil, \qquad H' = \lceil r \cdot H \rceil \tag{4.7}$$

Trong đó, W và H là kích thước ảnh gốc, r là hệ số thu phóng, W' và H' là kích thước sau khi thu phóng, được làm tròn lên thành số nguyên. Ảnh lúc này có cạnh dài bằng 640 nhưng cạnh ngắn vẫn nhỏ hơn 640.

Phần còn thiếu của cạnh ngắn được bù bằng kỹ thuật LetterBox. Ảnh được đặt vào giữa khung đích, phần trống được tô màu xám có giá trị 114 và chia đều cho hai phía:

$$p_x = \frac{640 - W'}{2}, \qquad p_y = \frac{640 - H'}{2} \tag{4.8}$$

Trong đó, p_x là độ rộng viền thêm vào mỗi bên trái và phải, p_y là độ rộng viền thêm vào mỗi phía trên và dưới. Tọa độ đa giác cũng được nhân với r và cộng thêm p_x, p_y, nhờ đó nhãn luôn trùng với vị trí polyp trên ảnh mới. Ở nhánh huấn luyện, khung đích là 640 × 640. Ở nhánh kiểm định, các ảnh có tỷ lệ khung hình gần nhau được gom thành lô 16 ảnh, khung của mỗi lô được làm tròn lên bội số của 32 và cộng thêm nửa bước, nên có cạnh dài 672 pixel. Hình 4.3 minh họa công đoạn này với khung 640 × 640.



a. Ảnh gốc 1920 × 1072 pixel

b. Sau LetterBox 640 × 640 pixel

Hình 4.3: Trước và sau khi đổi kích thước ảnh bằng thu phóng giữ tỷ lệ và LetterBox

Nguồn: ảnh a từ bộ dữ liệu Kvasir-SEG [5], ảnh b do đề tài tạo theo cách xử lý LetterBox trong mã nguồn Ultralytics.

Ảnh gốc 1.920 × 1.072 pixel được thu nhỏ còn 640 × 358 pixel, sau đó được thêm hai dải xám cao 141 pixel ở phía trên và phía dưới. Polyp không bị méo và đường bao vẫn nằm đúng vị trí.

4.5. TĂNG CƯỜNG DỮ LIỆU

Tập huấn luyện chỉ có 1.040 ảnh, khá ít đối với một mô hình học sâu, nên mô hình dễ ghi nhớ từng ảnh thay vì học đặc điểm chung của polyp. Tăng cường dữ liệu tạo ra một biến thể ngẫu nhiên khác nhau mỗi lần ảnh được nạp và chỉ áp dụng cho tập huấn luyện. Các phép biến đổi chia thành hai nhóm. Nhóm hình học làm thay đổi vị trí pixel nên đa giác nhãn được biến đổi theo. Nhóm màu chỉ đổi giá trị màu nên nhãn giữ nguyên. Thông số của từng phép được lấy từ tệp cấu hình huấn luyện và tổng hợp trong Bảng 4.2.

Mosaic [39] ghép bốn ảnh huấn luyện vào bốn góc quanh một tâm ngẫu nhiên rồi cắt về 640 × 640 pixel. Ảnh thu được chứa polyp ở nhiều vị trí và kích thước, ảnh nền cũng có thể được ghép chung với ảnh polyp. Vì ảnh ghép không giống ảnh nội soi thật, Mosaic được tắt trong 10 epoch cuối [42]. Khi đó, mô hình học trên từng ảnh đơn lẻ nhưng vẫn kèm các phép biến đổi khác.

Phép biến đổi affine dịch tâm ảnh lệch tối đa 10% khung và thu phóng với hệ số từ 0,5 đến 1,5, giúp mô hình nhận ra polyp ở nhiều khoảng cách và vị trí khác nhau. Phần ảnh ra ngoài khung bị cắt, phần trống được tô màu xám 114. Phép biến đổi màu trong không gian HSV gần như giữ nguyên sắc độ để màu niêm mạc không trở nên phi thực tế, nhưng thay đổi mạnh độ bão hòa và độ sáng để mô phỏng các điều kiện chiếu sáng khác nhau. Cuối cùng, mỗi ảnh có xác suất 0,5 bị lật ngang. Các phép lật dọc, xoay, cắt xiên và trộn ảnh không được sử dụng.

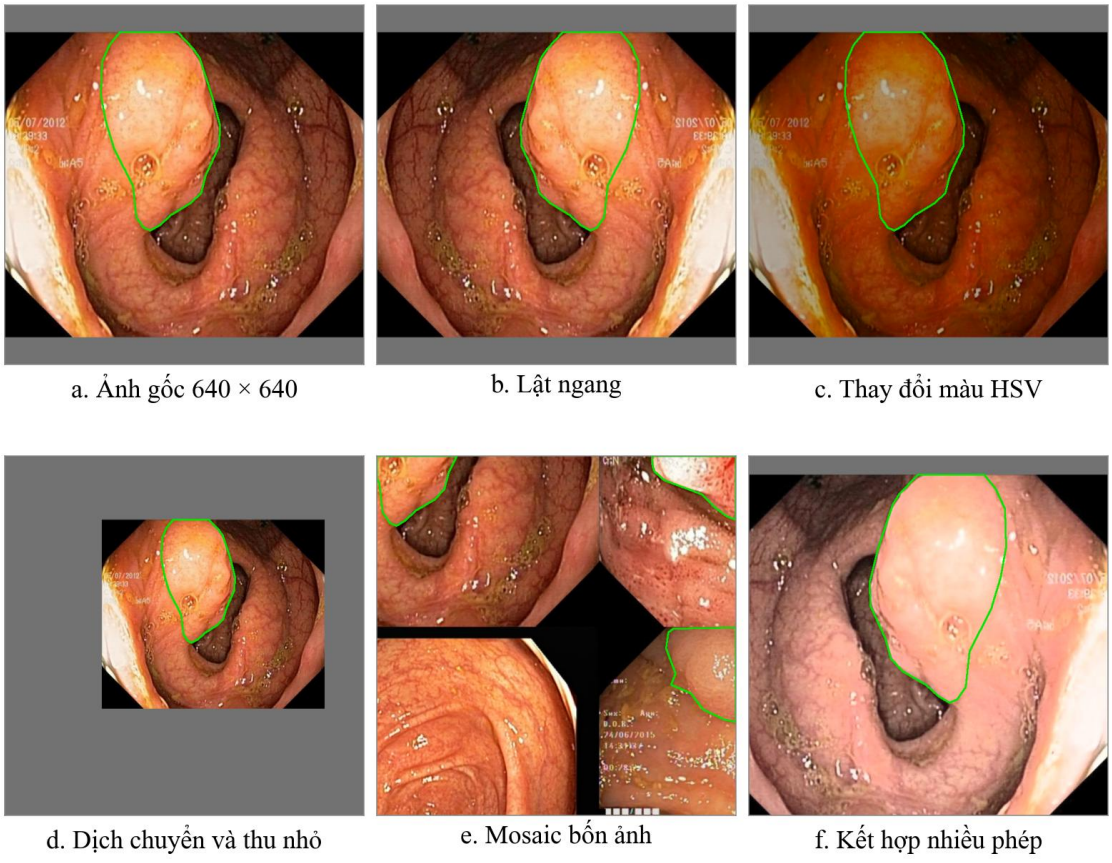
Bảng 4.2: Thông số tăng cường dữ liệu trong cấu hình huấn luyện

Phép biến đổi	**Tham số**	**Giá trị**	**Ý nghĩa**
Mosaic	mosaic	1,0	Luôn ghép bốn ảnh
Tắt Mosaic	close_mosaic	10	Tắt trong 10 epoch cuối
Dịch chuyển	translate	0,1	Lệch tối đa 10% khung
Thu phóng	scale	0,5	Hệ số từ 0,5 đến 1,5
Sắc độ	hsv_h	0,015	Dịch sắc độ rất nhẹ
Độ bão hòa	hsv_s	0,7	Thay đổi tối đa 70%

Phép biến đổi	Tham số	Giá trị	Ý nghĩa
Độ sáng	hsv_v	0,4	Thay đổi tối đa 40%
Lật ngang	fliplr	0,5	Xác suất 50%
Lật dọc, xoay, cắt xiên, phối cảnh	flipud, degrees, shear, perspective	0	Không sử dụng
Trộn ảnh	mixup, cutmix, copy_paste	0	Không sử dụng

Nguồn: tệp cấu hình huấn luyện của đề tài.

Hình 4.4 minh họa tác động của từng phép tăng cường lên cùng một ảnh huấn luyện. Để dễ quan sát, các hệ số trong hình được chọn cố định trong phạm vi của Bảng 4.2.



a. Ảnh gốc 640 × 640 b. Lật ngang c. Thay đổi màu HSV

d. Dịch chuyển và thu nhỏ e. Mosaic bốn ảnh f. Kết hợp nhiều phép

Hình 4.4: Ảnh huấn luyện trước và sau các phép tăng cường dữ liệu

Nguồn: ảnh gốc từ bộ dữ liệu Kvasir-SEG [5] và Kvasir [8], các phép biến đổi do đề tài tạo minh họa theo thông số trong Bảng 4.2.

Ở cả sáu trường hợp, đường bao luôn khớp với polyp, cho thấy nhãn được biến đổi đồng bộ với ảnh.

4.6. ĐỊNH DẠNG VÀ CHUẨN HÓA DỮ LIỆU ĐẦU VÀO

Trước khi đưa vào mạng, ảnh được đổi thứ tự kênh màu từ BGR của OpenCV sang RGB, sắp xếp lại chiều thành số kênh, chiều cao, chiều rộng và ghép thành lô 8 ảnh. Giá trị pixel được chia cho 255 để đưa về đoạn từ 0 đến 1, đề tài không trừ trung bình hay chia độ lệch chuẩn của ImageNet. Đồng thời, đa giác trong tệp nhãn được tô đầy để tạo lại mặt nạ huấn luyện có kích thước 160 × 160, nhỏ hơn ảnh 4 lần để tiết kiệm bộ nhớ. Với ảnh nền, mặt nạ chỉ gồm giá trị 0.

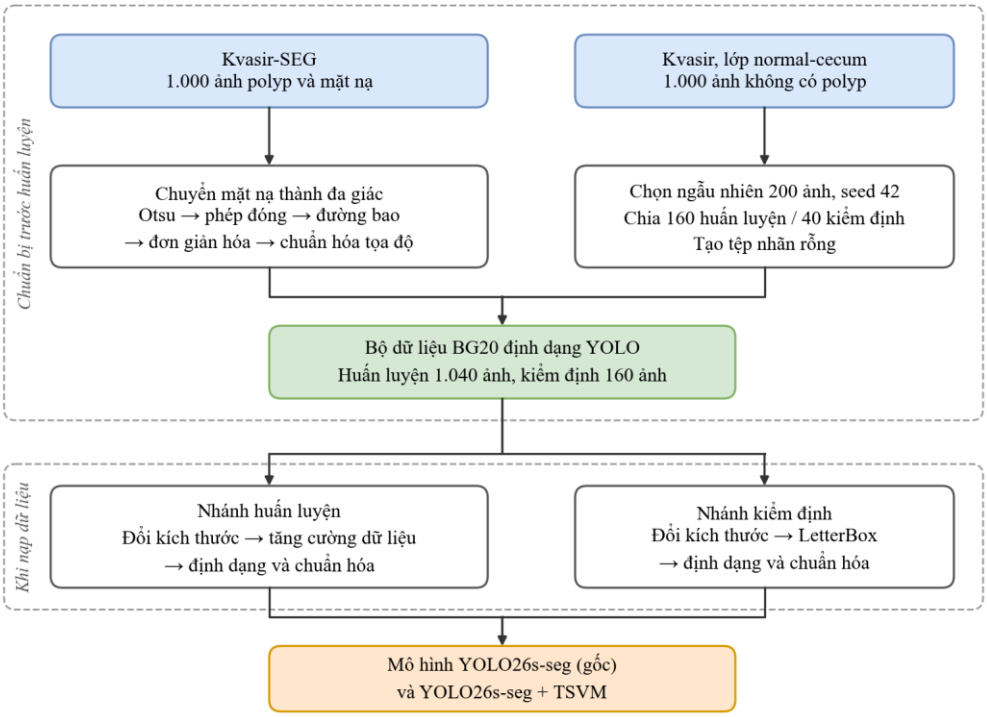
4.7. KIỂM TRA TÍNH TOÀN VẸN CỦA DỮ LIỆU

Chương trình chuyển đổi tự kiểm tra số ảnh và số tệp nhãn của từng tập và dừng lại nếu có sai lệch. Đề tài kiểm tra thêm một lượt độc lập. Kết quả cho thấy mỗi ảnh có đúng một tệp nhãn, toàn bộ 1.063 dòng nhãn đều hợp lệ, và không có ảnh nào trùng tên hoặc trùng nội dung theo mã băm SHA-256 giữa hai tập. Khi chạy lại chương trình bằng OpenCV 5.0.0, cả 1.000 tệp nhãn đều trùng khớp từng ký tự với nhãn đang dùng, và bước chọn ảnh nền với seed 42 cho ra đúng danh sách đã lưu.

Lượt kiểm tra này vẫn có giới hạn. Dữ liệu không có thông tin định danh bệnh nhân nên chưa thể khẳng định hai tập không chứa ảnh của cùng một bệnh nhân. Mã băm chỉ phát hiện ảnh trùng hoàn toàn, chưa phát hiện ảnh gần giống nhau. Ngoài ra, bộ dữ liệu không có tập kiểm thử độc lập, nên kết quả ở các chương sau được đánh giá trên tập kiểm định.

4.8. TỔNG HỢP QUY TRÌNH TIỀN XỬ LÝ

Hình 4.5 tóm tắt toàn bộ quy trình đã trình bày trong chương.



Hình 4.5: Quy trình xây dựng bộ dữ liệu BG20 và tiền xử lý dữ liệu

Nguồn: đề tài tự xây dựng.

Phần trên của sơ đồ là giai đoạn chuẩn bị dữ liệu, chỉ thực hiện một lần. Phần dưới là giai đoạn nạp dữ liệu, gồm nhánh huấn luyện có tăng cường dữ liệu và nhánh kiểm định chỉ đổi kích thước bằng LetterBox. Cả hai nhánh đều kết thúc bằng bước chuẩn hóa trước khi đưa vào mô hình.

Khối TSVM nằm bên trong mô hình và chỉ xử lý đặc trưng sau khi ảnh đã được chuẩn hóa. Mô hình YOLO26-seg gốc và mô hình tích hợp TSVM dùng chung quy trình tiền xử lý, nên dữ liệu không phải là nguồn tạo ra khác biệt giữa hai mô hình. Dao động ngẫu nhiên giữa các lần huấn luyện được kiểm soát bằng 20 seed ghép cặp ở Chương 6.

CHƯƠNG 5: CÀI ĐẶT VÀ ỨNG DỤNG

Ghi chú soạn thảo (xóa khi nộp):

- Khung **[CHỤP HÌNH 5.x]** là ảnh giao diện nhóm cần chụp. Mỗi khung ghi rõ: màn hình nào, thao tác gì trước khi chụp, ảnh mẫu nào, và trên ảnh phải thấy được gì. Danh sách tổng hợp ở cuối tệp (mục "Danh sách hình cần chụp").
- **[CHỜ KẾT QUẢ]** là số liệu chưa đo. **[CẦN KIỂM TRA]** là điểm cần nhóm xác nhận trước khi nộp.
- Thông tin ứng dụng lấy từ mã nguồn và README trong `Ung_Dung/polypweb/`. Thông tin huấn luyện lấy từ `args.yaml` của các lượt chạy và `00_tai_lieu_tong_quan/05_CAU_HINH_HUAN_LUYEN.md`.

Chương này trình bày phần cài đặt của đề tài gồm hai mảng:

- **cài đặt thực nghiệm:** môi trường, cách cài đặt mô hình gốc và mô hình tích hợp, quy trình huấn luyện trên Kaggle, cách lưu và tính lại kết quả;
- **ứng dụng minh họa PolypVision:** kiến trúc hệ thống, dịch vụ suy luận, tổ chức lưu trữ, giao diện Web và Mobile, chức năng so sánh hai mô hình.

5.1. CẤU HÌNH HỆ THỐNG VÀ MÔI TRƯỜNG THỰC NGHIỆM

5.1.1. Môi trường huấn luyện

Huấn luyện chạy trên nền tảng Kaggle, theo đề cương đã duyệt. Kaggle cung cấp GPU miễn phí với hạn mức khoảng 30 giờ mỗi tuần cho mỗi tài khoản. Vì mỗi lượt huấn luyện mô hình tích hợp mất khoảng 4,7 giờ, các lượt được chia cho nhiều tài khoản để chạy song song.

Bảng 5.1: Môi trường huấn luyện

Thành phần	**Giá trị**	**Ghi chú**
Nền tảng	Kaggle Notebook	Chế độ "Save & Run All" chạy nền, không cần giữ trình duyệt
GPU	NVIDIA Tesla T4, 15 GB **[CẦN KIỂM TRA]**	Theo nhật ký Kaggle còn lưu; nhật ký này thuộc lượt chạy thử, chưa có nhật ký gắn với từng lượt chính
Hệ điều hành	Linux (máy ảo Kaggle)	

Thành phần	Giá trị	Ghi chú
Python	3.12 **[CẦN KIỂM TRA]**	Cùng nguồn với dòng GPU
PyTorch	2.10.0 + CUDA 12.8 **[CẦN KIỂM TRA]**	Cùng nguồn với dòng GPU
Ultralytics	8.4.127	Ghi trong mọi tệp `best.pt`
Thời gian một lượt (100 epoch)	Mô hình gốc khoảng 2,3 giờ; mô hình tích hợp khoảng 4,7 giờ	Trung bình 20 seed, phụ thuộc phần cứng Kaggle cấp

5.1.2. Môi trường đánh giá và đo hiệu năng

Sau khi huấn luyện, các tệp kết quả được tải về máy cá nhân để tính lại số liệu, vẽ hình và đo tốc độ.

Bảng 5.2: Môi trường đánh giá cục bộ

Thành phần	Giá trị	Dùng cho
CPU	AMD Ryzen 7 250 (8 lõi, 16 luồng), RAM 15,3 GiB	Đo độ trễ trên CPU, chạy dịch vụ suy luận
GPU	NVIDIA GeForce RTX 5050 Laptop	Đánh giá theo điểm ảnh (Dice, IoU), đo tốc độ trên GPU (mục 6.5)
Hệ điều hành	Windows 11	
Môi trường Python có GPU	Python, PyTorch 2.13.0 + CUDA 13.0	Script đánh giá theo điểm ảnh
Môi trường của dịch vụ suy luận	Python 3.12, PyTorch 2.6.0 (CPU), Ultralytics 8.4.127 theo `requirements.txt` **[CẦN KIỂM TRA: môi trường ảo hiện cài 8.4.103, nên cài lại cho khớp]**	Ứng dụng PolypVision

5.1.3. Ngôn ngữ và công cụ

Đề cương quy định Python cho phần mô hình và API, Java cho phần Web. Nhóm bổ sung Dart/Flutter cho ứng dụng di động. Đề cương có nêu cả PyTorch và TensorFlow; đề tài chỉ dùng PyTorch, vì Ultralytics YOLO26 và các bản cài đặt Mamba đều viết trên PyTorch. Dùng một framework giúp khối C2TSVMamba ghép thẳng vào mô hình gốc mà không phải chuyển đổi trọng số.

Bảng 5.3: Ngôn ngữ, framework và lý do lựa chọn

Hạng mục	**Lựa chọn**	**Lý do**
Học sâu	Python, PyTorch, Ultralytics [42]	YOLO26 được cài đặt chính thức trên Ultralytics; PyTorch cho phép viết khối SS2D tùy biến mà không cần biên dịch kernel riêng
Xử lý dữ liệu, thống kê	NumPy, pandas, OpenCV, SciPy, Matplotlib	Chuyển nhãn, tính chỉ số, kiểm định thống kê, vẽ hình
Dịch vụ suy luận	Python, FastAPI, Uvicorn	Cùng ngôn ngữ với mô hình nên nạp checkpoint trực tiếp; FastAPI nhẹ và tự sinh tài liệu API
Web	Java 17, Spring Boot 4.1, Thymeleaf, Spring Security, Bootstrap 5	Theo yêu cầu đề cương; Spring Boot có sẵn bảo mật, phiên đăng nhập, kết nối MongoDB
Cơ sở dữ liệu	MongoDB	Lưu tài khoản và lịch sử phân tích dạng tài liệu, dễ mở rộng trường
Di động	Flutter, Dart	Một mã nguồn chạy được trên Android và iOS
Quản lý mã	Git	Theo dõi thay đổi mã, cấu hình và tài liệu

5.2. CÀI ĐẶT MÔ HÌNH GỐC YOLO26S-SEG

Mô hình gốc dùng nguyên bản Ultralytics 8.4.127, cài bằng `pip`. Trọng số khởi tạo là `yolo26s-seg.pt`, đã được huấn luyện sẵn trên COCO [41]. Lớp đầu ra được đổi từ 80 lớp của COCO sang 1 lớp `polyp`.

Tệp mô tả dữ liệu `data_bg20.yaml`:

```yaml
path: <thư mục bộ dữ liệu BG20>
train: images/train
val: images/val
nc: 1
names: {0: polyp}
```

Lệnh huấn luyện chính (rút gọn):

```
model = YOLO("yolo26s-seg.pt")
model.train(data="data_bg20.yaml", epochs=100, imgsz=640, batch=8,
            optimizer="AdamW", lr0=0.001, warmup_epochs=5.0,
            amp=False, deterministic=True, seed=SEED, workers=2,
            patience=100, close_mosaic=10)
```

Bảng 5.4: Siêu tham số huấn luyện (giống nhau ở mọi lượt của cả hai mô hình)

Nhóm	**Tham số**
Dữ liệu	Ảnh vào 640×640; không lưu đệm ảnh (`cache=False`)
Tối ưu	AdamW [40]; tốc độ học ban đầu 0,001, cuối 0,01 × ban đầu; momentum 0,937; suy giảm trọng số 0,0005; khởi động 5 epoch
Lịch	100 epoch; lô 8 ảnh; không dừng sớm (`patience=100`); tắt Mosaic ở 10 epoch cuối
Tăng cường	Mosaic [39] 1,0; lật ngang 0,5; co giãn 0,5; dịch 0,1; đổi màu HSV (0,015; 0,7; 0,4); không xoay, không lật dọc, không MixUp
Mặt nạ	Tỉ lệ thu nhỏ mặt nạ 4 (mặt nạ 160×160)
Độ chính xác số	FP32 hoàn toàn (`amp=False`)
Tất định	`deterministic=True`, 2 luồng nạp dữ liệu, seed cố định cho từng lượt

Khóa tất định. Trước khi huấn luyện, notebook đặt seed cho `random`, `numpy`, `torch` (cả CUDA), đặt `PYTHONHASHSEED`, `CUBLAS_WORKSPACE_CONFIG=:4096:8`, tắt `cudnn.benchmark` và bật `torch.use_deterministic_algorithms(True)`. Nhờ vậy, cùng seed thì cùng thứ tự dữ liệu và cùng khởi tạo, nên hai mô hình ở cùng seed so sánh được với nhau.

5.3. CÀI ĐẶT MÔ HÌNH TÍCH HỢP VMAMBA

5.3.1. Tổ chức mã nguồn

Khối C2TSVMamba không có trong Ultralytics gốc, nên nhóm tạo một bản sao (fork) của Ultralytics 8.4.127 và thêm khối mới. Các thay đổi:

Bảng 5.5: Các tệp được thêm hoặc sửa trong fork

Tệp	**Nội dung thay đổi**
`nn/modules/topology_shape_vmamba.py` (mới)	Cài đặt SS2D, ShapeAwareBranch, DirectionalShapeExtractor, TopologyShapeGate, TSVMamba và C2TSVMamba
`nn/modules/__init__.py`	Xuất các lớp mới
`nn/tasks.py`	Đăng ký `C2TSVMamba` để trình đọc YAML dựng được mô hình và tự tính số kênh
`cfg/models/26/yolo26-seg-TopologyShapeVMamba.yaml` (mới)	Cấu hình mô hình: giống `yolo26-seg.yaml`, chỉ đổi tầng 10

Tệp	Nội dung thay đổi
`engine/trainer.py`, `utils/torch_utils.py`	Bản vá lưu checkpoint ở FP32 (mục 5.3.3), mô tả trong `PATCH_FP32.md`
`tests/test_topology_shape_vmamba.py` (mới)	Kiểm thử tự động cho khối mới

Trong tệp YAML, chỉ một dòng khác mô hình gốc:

```
# yolo26-seg.yaml (gốc)
- [-1, 2, C2PSA, [1024]]          # 10
# yolo26-seg-TopologyShapeVMamba.yaml (tích hợp)
- [-1, 2, C2TSVMamba, [1024]]     # 10
```

Chi tiết cấu tạo khối đã trình bày ở Chương 3.

5.3.2. Khởi tạo và huấn luyện

Mô hình tích hợp được dựng từ YAML ở cỡ *s*, sau đó nạp trọng số `yolo26s-seg.pt`:

```
model = YOLO("yolo26s-seg-TopologyShapeVMamba.yaml").load("yolo26s-seg.pt")
model.train(...)   # cùng tham số với mô hình gốc ở Bảng 5.4
```

Hàm `load` chỉ chép những tensor trùng tên và trùng kích thước. Toàn bộ các tầng khác tầng 10 nhận trọng số tiền huấn luyện. Ở tầng 10, phần khung `cv1`, `cv2` và mạng truyền thẳng trùng cấu trúc với C2PSA nên cũng được kế thừa; các nhánh mới (SS2D, nhánh hình dạng, cổng, fuse) được khởi tạo mới.

Lưu ý khi cài đặt: Ultralytics đoán cỡ mô hình từ **tên tệp YAML**, không chỉ từ khóa `scale` trong tệp. Tên tệp phải chứa `yolo26s`; nếu không, mô hình bị dựng ở cỡ *n* và không nạp được trọng số. Notebook của thí nghiệm tách thành phần có thêm bước kiểm tra số tham số tầng 10 trước khi huấn luyện để phát hiện lỗi này sớm.

5.3.3. Bản vá lưu checkpoint ở FP32

Ultralytics mặc định chuyển mô hình sang FP16 khi lưu `best.pt`. Với khối mới, một giá trị thống kê của lớp chuẩn hóa lô trong tầng 10 vượt giới hạn 65.504 của FP16, làm checkpoint sai khi suy luận (Chương 3, mục 3.8). Bản vá đổi ba chỗ:

- hàm lưu mô hình và `MultiTrainer` dùng `.float()` thay cho `.half()`;
- hàm `strip_optimizer` dùng `.float()`;

– nếu gặp giá trị NaN hoặc vô cùng thì báo lỗi ngay, thay vì âm thầm thay bằng 0.

Vì vậy mô hình tích hợp được gọi là **TSVM FP32**. Tệp `best.pt` của mô hình tích hợp lớn hơn của mô hình gốc vì lưu ở FP32.

5.3.4. Thí nghiệm tách thành phần

Mã TSVMamba có tham số `mode` để bật hoặc tắt từng nhánh. Thí nghiệm tách thành phần dùng cùng notebook, chỉ đổi đối số của tầng 10 trong YAML:

```
- [-1, 2, C2TSVMamba, [1024, 0.5, 16, 3, "vmamba_only"]]
```

Bảng 5.6: Các cấu hình tách thành phần

Chế độ	**Giữ lại**	**Bỏ đi**	**Số tham số tầng 10**
Đầy đủ	SS2D, nhánh hình dạng, cổng	–	1.811.201
`vmamba_only`	SS2D	Nhánh hình dạng, cổng, fuse	1.071.873
`shape_only`	Nhánh hình dạng	SS2D, cổng, fuse	1.331.969
`shape_vmamba_no_guidance`	SS2D và nhánh hình dạng	Cổng $(1 + G)$	1.745.409

Mỗi chế độ chạy với seed 0–4, tổng cộng 15 lượt, chia cho 6 tài khoản Kaggle. Kết quả ở mục 6.6.

5.4. QUY TRÌNH HUẤN LUYỆN VÀ TỔ CHỨC LƯU TRỮ KẾT QUẢ THỰC NGHIỆM

5.4.1. Notebook Kaggle

Đề tài dùng ba notebook:

– **Notebook mô hình gốc** (3 ô): tạo `data_bg20.yaml`; cài `ultralytics==8.4.127` bằng pip; khóa tất định và huấn luyện, rồi nén kết quả.

– **Notebook mô hình tích hợp** (4 ô): nạp fork Ultralytics bằng liên kết tượng trưng (symlink) tên `ultralytics`, tạo YAML cỡ s và `data_bg20.yaml`; tải `yolo26s-seg.pt`; khóa tất định và huấn luyện; nén kết quả.

– **Notebook tách thành phần** (6 ô): như notebook mô hình tích hợp, thêm ô chọn chế độ và seed ở đầu, và ô **kiểm tra mô hình** trước khi huấn luyện (dựng

mô hình, kiểm tra chế độ và số tham số tầng 10; sai thì dừng ngay, không tốn giờ GPU).

Lưu ý về bản lưu trữ: bản notebook mô hình tích hợp còn lưu trong kho mã trỏ tới fork cũ (lưu FP16) và tên lượt chạy chưa có hậu tố `_FP32`. Khi chạy các lượt chính, đường dẫn và tên đã được sửa sang fork FP32; bằng chứng là mọi tệp `best.pt` của mô hình tích hợp đều lưu ở FP32 **[NÊN LƯU LẠI BẢN NOTEBOOK ĐÃ SỬA VÀO KHO MÃ]**.

[CHỤP HÌNH 5.1] Notebook Kaggle đang chạy.

- **Màn hình:** notebook tách thành phần trên Kaggle (đang chạy), khung bên phải mở mục *Input* (thấy hai dataset `kvasir-yolo-seg-bg20` và `ultralytics-topology-shape-aware-vmamba-fp32`) và *Settings* (thấy Accelerator là GPU).
- **Cần thấy:** đầu ra của ô kiểm tra mô hình, có dòng "Tham số tầng 10 = … ☑", và vài dòng nhật ký huấn luyện có cột epoch, loss, mAP. Chỉ notebook tách thành phần mới có ô kiểm tra này.
- **Lưu ý:** che tên tài khoản Kaggle nếu không muốn công khai.

5.4.2. Cấu trúc lưu trữ kết quả

Mỗi lượt huấn luyện lưu riêng một thư mục, tên có dạng `Kvasir_BG20_<Mô hình>_s<seed>_w2`:

```
03_ket_qua_huan_luyen/
├── Baseline_YOLO26s_seg/
│   └── Kvasir_BG20_Baseline_YOLO26s_seg_s0_w2/
│       ├── args.yaml        # toàn bộ cấu hình đã dùng
│       ├── results.csv      # chỉ số và loss từng epoch
│       └── weights/best.pt  # trọng số tốt nhất
├── TSVM_FP32/
│   └── Kvasir_BG20_YOLO26s_seg_TSVM_FP32_s0_w2/ …
└── Vmamba_only/, Shape_only/, No_guidance/   # thí nghiệm tách thành phần, seed 0–4 (mục 6.6)
```

Từ các tệp này, script `tinh_lai_so_lieu.py` tính lại mọi số liệu và ghi ra các bảng CSV. Script `danh_gia_pixel.py` suy luận lại trên tập kiểm định để tính Dice và IoU theo điểm ảnh cho bốn mô hình. Script `ve_hinh.py` đọc các bảng để vẽ hình. Mọi con số trong Chương 6 đều truy được về bảng và script tương ứng.

Bảng 5.7: Các tệp kết quả chính

Tệp	Nội dung
`bang/02_ket_qua_tung_seed.csv`	Chỉ số của từng lượt tại epoch đạt mask mAP50–95 cao nhất

Tệp	Nội dung
`bang/04_thong_ke_ghep_cap.csv`	Trung bình, độ lệch chuẩn, chênh lệch, khoảng tin cậy, giá trị p
`bang/05_duong_cong_theo_epoch.csv`	Trung bình và độ lệch chuẩn theo epoch
`bang/06_ma_tran_nham_lan_tung_seed.csv`	TP, FP, FN của từng lượt
`bang/11_pixel_tung_seed.csv`, `12_pixel_thong_ke.csv`	Dice, IoU, Precision, Recall theo điểm ảnh của 4 mô hình

5.5. ỨNG DỤNG POLYPVISION: TỔNG QUAN

5.5.1. Yêu cầu chức năng

Theo đề cương, ứng dụng cần:

1. giao diện cho phép tải ảnh nội soi lên;
2. chức năng nhận diện và phân đoạn polyp, hiển thị mặt nạ, đường biên và độ tin cậy;
3. tổ chức lưu trữ kết quả dự đoán;
4. chức năng so sánh mô hình YOLO26-seg gốc với mô hình tích hợp VMamba.

Ngoài ra, nhóm bổ sung: quản lý tài khoản và phân quyền, lịch sử phân tích, trang quản trị, ứng dụng di động.

Bảng 5.8: Đối chiếu yêu cầu đề cương với chức năng đã cài đặt

Yêu cầu	Web	Mobile	Ghi chú
Tải ảnh nội soi	Có	Có (chọn từ thư viện hoặc chụp)	JPG, JPEG, PNG; tối đa 10 MB
Hiển thị mặt nạ phân đoạn	Có	Có	Ảnh chồng (overlay) mặt nạ lên ảnh gốc

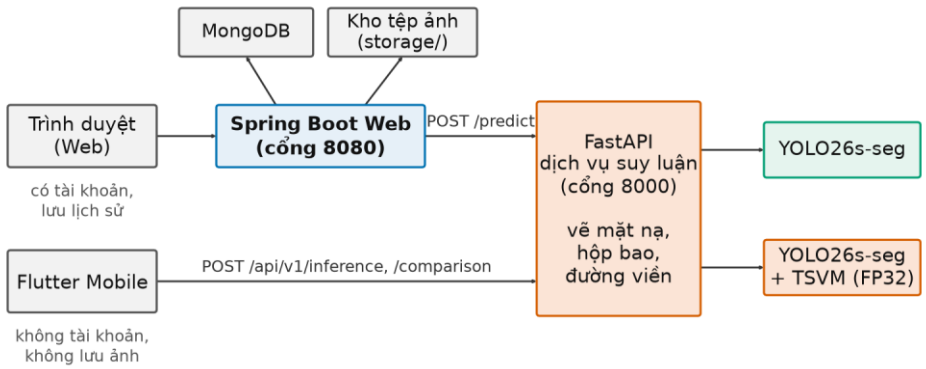
Yêu cầu	Web	Mobile	Ghi chú
Hiển thị đường biên polyp	Có	Có	Sau khi `plot()` của Ultralytics tô mặt nạ và vẽ hộp bao, hàm `draw_polyp_contours` trong dịch vụ suy luận vẽ thêm một đường viền khép kín màu vàng theo đa giác của từng mặt nạ (`masks.xy`), độ dày tỉ lệ với kích thước ảnh. Web và Mobile cùng nhận ảnh này nên không phải sửa phía giao diện
Hiển thị độ tin cậy	Có (thanh tiến trình, %)	Có	Lấy độ tin cậy cao nhất trong các đối tượng phát hiện. Dịch vụ suy luận trả thang 0–1; Website đổi sang thang 0–100 trước khi lưu và hiển thị
Lưu trữ kết quả	Có (MongoDB + tệp ảnh)	Không lưu	Mobile cố ý không lưu ảnh để bảo vệ quyền riêng tư
So sánh hai mô hình	Có	Có	Chạy lần lượt hai mô hình trên cùng ảnh

5.5.2. Kiến trúc hệ thống

Hệ thống gồm ba thành phần. **Dịch vụ suy luận FastAPI** giữ hai mô hình và được dùng chung cho hai giao diện:

– **Nhánh Web:** Trình duyệt → Spring Boot → FastAPI → mô hình. Spring Boot quản lý tài khoản, phiên đăng nhập, lịch sử, MongoDB và tệp ảnh.

– **Nhánh Mobile:** Flutter → FastAPI → mô hình. Ứng dụng di động gọi thẳng dịch vụ suy luận, không có tài khoản và không lưu lịch sử.



Hình 5.2: Kiến trúc hệ thống PolypVision. Website (Spring Boot, cổng 8080) và ứng dụng di động (Flutter) là hai client độc lập của cùng dịch vụ suy luận FastAPI (cổng 8000). Chỉ nhánh Web có MongoDB và kho tệp ảnh.

Tách dịch vụ suy luận ra riêng có ba lợi ích:

– Web và Mobile dùng chung một nơi nạp mô hình, tránh nạp hai lần;
– phần Java không cần biết gì về PyTorch;
– có thể chuyển dịch vụ suy luận sang máy có GPU mà không đổi Web và Mobile.

5.6. DỊCH VỤ SUY LUẬN FASTAPI

5.6.1. Nạp mô hình

Khi khởi động, dịch vụ nạp hai checkpoint lấy từ **seed 0** của thực nghiệm:

Bảng 5.9: Hai mô hình trong dịch vụ suy luận

Khóa mô hình	**Vai trò**	**Tệp checkpoint**
`YOLO26s-seg`	Mô hình gốc	`yolo26s-seg-best.pt`
`YOLO26s-TSVM-FP32-seg`	Mô hình tích hợp	`yolo26s-tsvm-fp32-best.pt`

Một số điểm cài đặt:

– Mã của khối C2TSVMamba được chép vào dịch vụ (`custom_ultralytics/topology_shape_vmamba.py`) và đăng ký đúng đường dẫn mô-đun ghi trong checkpoint trước khi nạp. Nếu không làm vậy, PyTorch không dựng lại được mô hình.
– Mô hình được ép về FP32 và suy luận với `half=False`, tránh lại lỗi tràn số.
– Tệp `models/experiment_manifest.json` lưu nguồn gốc và mã băm SHA-256 của từng checkpoint. Khi chạy, dịch vụ tính mã băm của tệp đang nạp và trả ra ở API `/health`, để đối chiếu thủ công với manifest.
– Hai checkpoint cùng seed 0 để so sánh công bằng trên ứng dụng. `best.pt` được Ultralytics chọn theo chỉ số tổng hợp hộp và mặt nạ, nên nói chung epoch của nó có thể khác epoch có mask mAP50–95 cao nhất dùng ở Chương 6. Ở seed 0, hai epoch này trùng nhau (mô hình gốc: epoch 89; mô hình tích hợp: epoch 88). Cũng cần lưu ý: ở riêng seed 0, mô hình tích hợp có mask mAP50–95 **thấp hơn** mô hình gốc (0,7245 so với 0,7366). Vì vậy các hình so sánh trên ứng dụng chỉ có tính minh họa, không đại diện cho kết quả trung bình.

Các tham số suy luận đọc từ biến môi trường: ngưỡng tin cậy mặc định 0,25, kích thước ảnh 640, thiết bị tự chọn (CPU hoặc GPU). Riêng nhánh Mobile có

thêm giới hạn số yêu cầu suy luận đồng thời và thời gian chờ tối đa; nhánh Web dùng khóa riêng cho từng mô hình, còn Spring Boot tự đặt thời gian chờ khi gọi dịch vụ.

5.6.2. Các API

Bảng 5.10: Các API của dịch vụ suy luận

Phương thức	**Đường dẫn**	**Dùng cho**	**Chức năng**
GET	`/health`	Web	Trạng thái dịch vụ, thiết bị, danh sách mô hình
POST	`/predict`	Web	Nhận ảnh (`image`) và tên mô hình (`modelType`), trả kết quả một mô hình
GET	`/api/v1/health`	Mobile	Trạng thái tổng hợp
GET	`/api/v1/models`	Mobile	Danh sách hai mô hình và trạng thái sẵn sàng
POST	`/api/v1/inference`	Mobile	Nhận ảnh (`image`) và khóa mô hình (`modelKey`), suy luận một mô hình
POST	`/api/v1/comparison`	Mobile	Nhận ảnh (`image`), chạy cả hai mô hình trên cùng ảnh

Kết quả suy luận của API `/predict` (nhánh Web) gồm các trường chính dưới đây; ngoài ra còn `success`, `modelType`, `originalFileName` và `message`.

Bảng 5.11: Các trường chính trong kết quả suy luận

Trường	**Kiểu**	**Ý nghĩa**
`detected`	logic	Có phát hiện polyp hay không
`label`	chuỗi	Tên lớp (`polyp`)
`confidence`	số thực	Độ tin cậy cao nhất trong các đối tượng phát hiện
`maskCount`	số nguyên	Số mặt nạ (số polyp) phát hiện được
`processingTimeMs`	số thực	Thời gian gọi mô hình, tính bằng mili giây
`resultImageBase64`	chuỗi	Ảnh JPEG chồng mặt nạ và hộp bao, mã hóa Base64

Các API cho Mobile dùng cấu trúc riêng: mỗi kết quả có `modelKey`, `modelName`, `modelFamily`, các trường suy luận như trên, và ảnh kết quả đặt trong `overlay.data` (thay cho `resultImageBase64`); kèm theo là `requestId`, kích thước ảnh gốc và một câu cảnh báo nghiên cứu. Kết quả so sánh gồm hai phần `baseline`, `vmamba` và tổng thời gian `totalProcessingTimeMs`. Khi lỗi, các API `/api/v1/*` trả một cấu trúc JSON thống nhất gồm `requestId`, `status`, `code`, `message`,

`fieldErrors`; API `/predict` dùng thông báo lỗi mặc định của FastAPI. Dịch vụ kiểm tra ảnh đầu vào (định dạng, giải mã được, dung lượng) trước khi suy luận và không lưu ảnh lại sau khi xử lý.

Lưu ý: `processingTimeMs` là thời gian đo trên máy chạy dịch vụ, gồm cả bước tiền xử lý và hậu xử lý của Ultralytics. Đây không phải phép đo hiệu năng khoa học; số liệu hiệu năng chính thức ở Chương 6.

[CHỤP HÌNH 5.3] Tài liệu API tự sinh của FastAPI.

– **Thao tác:** chạy dịch vụ, mở `http://127.0.0.1:8000/docs`, mở rộng mục `GET /api/v1/models` rồi bấm *Try it out* → *Execute*.

– **Cần thấy:** danh sách các API như Bảng 5.10, và kết quả trả về có hai mô hình ở trạng thái sẵn sàng.

5.7. TỔ CHỨC LƯU TRỮ KẾT QUẢ

Chỉ nhánh Web lưu kết quả. Dữ liệu chia làm hai nơi: thông tin dạng bản ghi lưu trong MongoDB, còn ảnh lưu thành tệp trên đĩa.

MongoDB có hai tập (collection):

Bảng 5.12: Tập `users`

Trường	**Ý nghĩa**
`id`	Mã tài khoản
`fullName`, `email`	Họ tên và email; email là duy nhất
`passwordHash`	Mật khẩu đã băm bằng BCrypt, không lưu mật khẩu gốc
`role`	`USER` hoặc `ADMIN`
`enabled`	Tài khoản đang hoạt động hay bị khóa
`createdAt`, `updatedAt`	Thời điểm tạo và cập nhật

Bảng 5.13: Tập `diagnosis_history`

Trường	**Ý nghĩa**
`id`, `userId`	Mã bản ghi, tài khoản sở hữu
`fileName`	Tên tệp ảnh người dùng tải lên
`modelType`	Mô hình đã dùng
`detected`, `label`, `confidence`, `maskCount`	Kết quả suy luận; `confidence` lưu theo thang 0–100

Trường	Ý nghĩa
`processingTimeMs`	Thời gian xử lý
`imageWidth`, `imageHeight`	Kích thước ảnh
`originalImageUrl`, `resultImageUrl`	Đường dẫn tới ảnh gốc và ảnh kết quả
`createdAt`	Thời điểm phân tích
`comparisonGroupId`, `comparisonResult`	Gắn hai bản ghi của cùng một lần so sánh với nhau

Kho tệp ảnh nằm trong thư mục `storage/` (đổi được bằng biến môi trường):

```
storage/
├── original/   # ảnh gốc người dùng tải lên
└── results/    # ảnh kết quả có chồng mặt nạ
```

Một lần **so sánh** tạo hai bản ghi trong `diagnosis_history`, một cho mỗi mô hình, có cùng `comparisonGroupId`. Cách làm này cho phép dùng chung một cấu trúc cho cả phân tích đơn lẻ và so sánh, đồng thời lọc riêng lịch sử so sánh khi cần.

Người dùng thường chỉ xem và xóa được dữ liệu của chính mình. Tài khoản quản trị xem được thống kê toàn hệ thống và mở được bản ghi, ảnh của mọi người dùng. Ứng dụng di động không lưu ảnh, không lưu lịch sử, chỉ gửi ảnh tới địa chỉ dịch vụ suy luận đã cấu hình.

[CHỤP HÌNH 5.19] Dữ liệu lưu trữ. *(Minh chứng cho mục "tổ chức lưu trữ kết quả" của đề cương)*

- **Thao tác:** mở MongoDB Compass, chọn tập `diagnosis_history`, mở một bản ghi của lần phân tích đã chụp ở Hình 5.7. Mở thêm thư mục `storage/` bằng trình quản lý tệp.
- **Cần thấy:** các trường như Bảng 5.13 (`modelType`, `confidence`, `originalImageUrl`, `resultImageUrl`, `createdAt`…) và hai thư mục `original/`, `results/` có tệp ảnh tương ứng. Ghép thành một hình hai phần.
- **Lưu ý:** không chụp tập `users` (có email và mã băm mật khẩu).

5.8. GIAO DIỆN VÀ CHỨC NĂNG

5.8.1. Website

Website dùng Spring Boot với giao diện Thymeleaf và Bootstrap 5. Các chức năng chính:

- đăng ký, đăng nhập, đăng xuất; phân quyền `USER` và `ADMIN`;

– phân tích một ảnh bằng một trong hai mô hình;

– so sánh hai mô hình trên cùng ảnh;

– xem lại lịch sử phân tích và lịch sử so sánh, có tìm kiếm, lọc, phân trang;

– tải ảnh kết quả, xuất thông tin kết quả, xóa dữ liệu của mình;

– trang hồ sơ, đổi mật khẩu; trang quản trị tài khoản và thống kê.

Website có các cơ chế bảo mật cơ bản: phiên đăng nhập, chống giả mạo yêu cầu (CSRF), băm mật khẩu bằng BCrypt và các tiêu đề HTTP bảo mật. Ngoài giao diện web, Spring Boot còn cung cấp một bộ REST API riêng (`/api/v1/auth`, `/api/v1/diagnoses`, `/api/v1/comparisons`, `/api/v1/dashboard`, `/api/v1/me`), dùng cho phiên bản ứng dụng di động trước đây có đăng nhập.

[CHỤP HÌNH 5.4] Trang chủ Website.

– **Thao tác:** chạy theo thứ tự MongoDB → FastAPI → Spring Boot, mở `http://localhost:8080`, **chưa đăng nhập**.

– **Cần thấy:** tên PolypVision, phần giới thiệu, nút đăng nhập/đăng ký và dòng cảnh báo "không thay thế chẩn đoán".

[CHỤP HÌNH 5.5] Trang đăng nhập.

– **Thao tác:** bấm *Đăng nhập*. Dùng **tài khoản thử tạo riêng cho việc chụp hình**, không chụp lúc đang gõ mật khẩu.

– **Cần thấy:** biểu mẫu email/mật khẩu, liên kết sang trang đăng ký. Trường mật khẩu để trống hoặc đã che.

[CHỤP HÌNH 5.6] Trang phân tích ảnh, trước khi chạy.

– **Thao tác:** đăng nhập, vào trang *Phân tích* (`/diagnose`). **Không chụp trang `/diagnoses/demo`**: trang này hiển thị kết quả mẫu cố định, không phải kết quả thật. Chọn một ảnh trong tập kiểm định có polyp rõ (gợi ý: một trong ba ảnh đã dùng ở hình `q01` Chương 6, để thống nhất), chọn mô hình *YOLO26s-TSVM-FP32-seg*.

– **Cần thấy:** ô chọn tệp đã có tên ảnh, danh sách chọn mô hình, nút *Phân tích*.

[CHỤP HÌNH 5.7] Kết quả phân tích một ảnh. *(Hình quan trọng nhất của chương)*

– **Thao tác:** bấm *Phân tích* trên ảnh ở Hình 5.6, chờ trang kết quả.

– **Cần thấy cùng lúc:** ảnh gốc và ảnh có mặt nạ chồng lên; **ô độ tin cậy dạng %** có thanh tiến trình; số polyp phát hiện; thời gian xử lý; tên mô hình; dòng cảnh báo y khoa.

– **Nên chụp thêm một bản** với ảnh nền không có polyp, để thấy cách hệ thống báo "không phát hiện".

[CHỤP HÌNH 5.8] Kết quả so sánh hai mô hình.

– **Thao tác:** vào trang *So sánh* (`/compare`), tải **cùng ảnh** ở Hình 5.7, bấm chạy; trang tự chuyển sang `/comparisons/{id}`.

– **Cần thấy:** hai cột kết quả đặt cạnh nhau, mỗi cột có ảnh overlay, độ tin cậy cao nhất, số mặt nạ và thời gian xử lý của mô hình đó.

– **Gợi ý:** nếu có ảnh mà hai mô hình cho mặt nạ khác nhau rõ, chụp thêm ảnh đó. Chú thích hình chỉ mô tả, **không kết luận** mô hình nào tốt hơn từ một ảnh.

[CHỤP HÌNH 5.9] Lịch sử phân tích.

– **Thao tác:** sau khi đã phân tích vài ảnh (cả đơn lẻ và so sánh), vào trang *Lịch sử* (`/history`); chụp thêm trang *Lịch sử so sánh* (`/comparisons`).

– **Cần thấy:** danh sách bản ghi có ảnh thu nhỏ, tên mô hình, độ tin cậy, thời điểm; ô tìm kiếm, bộ lọc theo mô hình và phân trang.

[CHỤP HÌNH 5.10] Bảng điều khiển.

– **Thao tác:** vào `/dashboard` của tài khoản thường; sau đó đăng nhập tài khoản quản trị và mở `/admin/dashboard`.

– **Cần thấy:** số lượt phân tích theo mô hình, theo ngày, hoạt động gần đây. Có thể ghép hai ảnh (người dùng và quản trị) thành một hình hai phần.

[CHỤP HÌNH 5.11] Trang quản trị tài khoản (`/admin/users`, tùy chọn).

[CHỤP HÌNH 5.20] Thông báo lỗi khi tải tệp sai *(tùy chọn)*. Tải một tệp không phải ảnh hoặc lớn hơn 10 MB lên trang `/diagnose`; cần thấy thông báo lỗi rõ ràng, không lộ chi tiết kỹ thuật.

– **Cần thấy:** danh sách tài khoản, vai trò, nút đổi vai trò. **Che email thật** nếu có.

5.8.2. Ứng dụng di động

Ứng dụng Flutter gọi thẳng dịch vụ suy luận, phục vụ trình diễn nhanh trên điện thoại. Chức năng:

– màn hình chính hiển thị trạng thái dịch vụ và hai mô hình;

– suy luận một mô hình và so sánh hai mô hình trên cùng ảnh;

– chọn và kiểm tra ảnh (JPG, JPEG, PNG, tối đa 10 MB);

– hiển thị tiến trình tải lên, cho phép hủy, thử lại, báo lỗi rõ ràng;

– xem ảnh kết quả toàn màn hình, phóng to và kéo;

– giao diện tự co giãn theo điện thoại hoặc máy tính bảng, xoay ngang hoặc dọc; chế độ sáng, tối.

[CHỤP HÌNH 5.12] Màn hình chính ứng dụng di động.

– **Thao tác:** chạy dịch vụ suy luận với `--host 0.0.0.0`, chạy ứng dụng trên máy ảo Android (`AI_BASE_URL=http://10.0.2.2:8000`) hoặc điện thoại thật cùng mạng.

– **Cần thấy:** trạng thái dịch vụ "sẵn sàng" và hai mô hình.

[CHỤP HÌNH 5.13] Kết quả suy luận trên di động.

– **Thao tác:** chọn cùng ảnh đã dùng ở Hình 5.7, chọn mô hình tích hợp, chạy.

– **Cần thấy:** ảnh overlay, độ tin cậy, số mặt nạ, thời gian xử lý.

[CHỤP HÌNH 5.14] So sánh hai mô hình trên di động.

– **Cần thấy:** kết quả của hai mô hình trên cùng màn hình (hoặc hai thẻ kết quả liền nhau).

– **Gợi ý trình bày:** ghép Hình 5.12–5.14 thành một hình ba phần (a), (b), (c), mỗi phần là ảnh chụp màn hình điện thoại để dọc.

[CHỤP HÌNH 5.15] Chế độ tối và xem toàn màn hình *(tùy chọn).*

5.8.3. Kiểm thử phần mềm

Mỗi thành phần có bộ kiểm thử tự động riêng. Theo README của dự án tại lần cập nhật gần nhất:

Bảng 5.14: Kết quả kiểm thử tự động **[CẦN KIỂM TRA: chạy lại các lệnh kiểm thử trước khi nộp và cập nhật số]**

Thành phần	Công cụ	Kết quả ghi nhận
Ứng dụng di động	`flutter test`	498/498 ca đạt
Dịch vụ suy luận	`pytest`	50/50 ca đạt
Website	`mvnw test` (JUnit)	261/261 ca đạt

Ngoài ra, nhóm đã kiểm thử thủ công trên điện thoại Android thật các chức năng: kiểm tra trạng thái, suy luận từng mô hình, so sánh, phóng to toàn màn hình, thử lại khi lỗi, xoay màn hình, chế độ tối và nút quay lại.

[CHỤP HÌNH 5.16] Kết quả chạy kiểm thử *(tùy chọn)*. Chụp phần cuối đầu ra của ba lệnh kiểm thử, thấy dòng tổng kết số ca đạt.

5.9. TRIỂN KHAI TRÊN DỮ LIỆU THỰC TẾ

Để kiểm tra mô hình trên ảnh **khác nguồn** với dữ liệu huấn luyện, nhóm dùng bốn bộ dữ liệu công khai: CVC-ClinicDB (612 ảnh), CVC-ColonDB (380 ảnh), ETIS-Larib (196 ảnh) và phần ảnh đơn của PolypGen (1.537 ảnh từ 6 trung tâm). Các ảnh này không tham gia huấn luyện.

Việc triển khai gồm hai phần:

1. **Đánh giá định lượng:** chạy checkpoint 20 seed của bốn mô hình (hai mô hình YOLO, PraNet, U-Net) trên toàn bộ bốn bộ dữ liệu, tính Dice, IoU, Precision, Recall theo điểm ảnh như trên tập kiểm định. Kết quả ở mục 6.7.
2. **Minh họa trên ứng dụng:** tải một số ảnh của ba bộ dữ liệu lên PolypVision, dùng chức năng so sánh để xem hai mô hình hoạt động trên ảnh lạ.

[CHỤP HÌNH 5.17] So sánh hai mô hình trên ảnh CVC-ClinicDB. **[CHỤP HÌNH 5.18] So sánh hai mô hình trên ảnh ETIS-Larib.**

– **Thao tác:** dùng trang *So sánh* của Website. Mỗi bộ chọn 1–2 ảnh: một ảnh polyp rõ và một ảnh khó (polyp nhỏ hoặc phản quang). Nên chọn ảnh sau khi có kết quả định lượng, để ảnh minh họa đại diện cho xu hướng chung chứ không phải trường hợp hiếm.

– **Cần thấy:** ảnh overlay và độ tin cậy của hai mô hình. Chú thích ghi rõ tên bộ dữ liệu và tên tệp ảnh.

Kết quả trên ứng dụng chỉ có tính minh họa. Kết luận về khả năng tổng quát hóa dựa trên đánh giá định lượng ở Chương 6. Bốn bộ dữ liệu này là ảnh công khai khác nguồn, không phải dữ liệu lâm sàng thu tại bệnh viện.

5.10. HẠN CHẾ CỦA PHẦN CÀI ĐẶT

– **Tốc độ.** Phép quét của SS2D viết bằng PyTorch thuần, không dùng kernel CUDA tối ưu. Trên CPU, mô hình tích hợp chậm khoảng gấp đôi mô hình gốc.

- **Nhật ký môi trường.** Thông tin GPU và phiên bản PyTorch trên Kaggle chỉ có từ nhật ký lượt chạy thử, chưa có cho từng lượt chính.
- **Ứng dụng di động** chưa có tài khoản, lịch sử và chưa chạy được khi không có mạng.
- **Triển khai thực tế** chưa có HTTPS, chưa giới hạn tần suất gọi API; Website lưu ảnh trên đĩa cục bộ.
- **Không phải thiết bị y tế.** Ứng dụng chỉ phục vụ nghiên cứu và minh họa; độ tin cậy hiển thị là điểm của mô hình, không phải xác suất mắc bệnh.

5.11. KẾT CHƯƠNG

Chương 5 đã trình bày:

- môi trường huấn luyện trên Kaggle và môi trường đánh giá cục bộ, cùng lý do chọn ngôn ngữ và thư viện;
- cách cài đặt mô hình gốc và mô hình tích hợp: fork Ultralytics, tệp YAML chỉ khác tầng 10, cách nạp trọng số, bản vá lưu FP32 và các chế độ tách thành phần;
- quy trình huấn luyện có khóa tất định và cấu trúc lưu trữ kết quả, đảm bảo mọi số liệu truy được về tệp gốc;
- ứng dụng PolypVision gồm dịch vụ suy luận FastAPI dùng chung, Website Spring Boot có lưu trữ trong MongoDB và ứng dụng di động Flutter, đáp ứng các yêu cầu tải ảnh, hiển thị mặt nạ và độ tin cậy, lưu trữ và so sánh hai mô hình.

Chương 6 trình bày kết quả thực nghiệm: so sánh hai mô hình qua nhiều seed, thí nghiệm tách thành phần, chi phí tính toán, so sánh với PraNet và U-Net, và kết quả trên dữ liệu thực tế.

CHƯƠNG 6: THỰC NGHIỆM VÀ ĐÁNH GIÁ

Chương 3 đã giới thiệu mô hình YOLO26s-seg tích hợp khối C2TSVMamba, gọi tắt là **mô hình đề xuất** hay **TSVM**. Chương 5 đã mô tả cách huấn luyện. Chương này trả lời câu hỏi: khi thay khối C2PSA ở tầng 10 bằng C2TSVMamba, chất lượng phân đoạn thay đổi thế nào, và chi phí tính toán tăng bao nhiêu.

Nội dung chương gồm:

– thiết lập thực nghiệm và cách kiểm định;

– so sánh hai mô hình YOLO trên 20 seed bằng các chỉ số của Ultralytics;

– đánh giá theo pixel và so sánh với PraNet, U-Net;

– chỉ số topology của mặt nạ;

– chi phí tính toán;

– thí nghiệm tách thành phần;

– kết quả trên bốn bộ dữ liệu ngoài;

– thảo luận và giới hạn.

6.1. THIẾT LẬP THỰC NGHIỆM

6.1.1. Các mô hình và cấu hình huấn luyện

Bảng 6.1: Các mô hình được đánh giá

Mô hình	Vai trò	Số lượt	Ghi chú
YOLO26s-seg	Mô hình gốc (đối chứng)	20	Tầng 10 là C2PSA
YOLO26s-seg + C2TSVMamba (TSVM)	Mô hình đề xuất	20	Chỉ khác tầng 10, lưu trọng số FP32
PraNet (ResNet-50)	So sánh	20	Mô hình phân đoạn polyp chuyên dụng [9]
U-Net	So sánh	20	Mô hình phân đoạn y tế kinh điển [10]

Nguồn: đề tài tổng hợp.

Hai mô hình YOLO dùng chung mọi thiết lập: dữ liệu BG20 (Chương 4), ảnh vào 640 pixel, 100 epoch, lô 8 ảnh, bộ tối ưu AdamW, khởi tạo từ trọng số COCO

và cùng các phép tăng cường dữ liệu. Mỗi mô hình được huấn luyện 20 lần với seed 0 đến 19. Lượt của mô hình gốc và mô hình đề xuất có cùng seed tạo thành một **cặp**. Nhờ ghép cặp, phần dao động do seed gây ra (khởi tạo, thứ tự ảnh, phép tăng cường) được trừ bớt khi so sánh [31].

Trên Kaggle, một lượt 100 epoch của mô hình gốc mất trung bình khoảng 2,3 giờ, của mô hình đề xuất khoảng 4,7 giờ (theo cột `train_time_s` của 40 lượt).

PraNet và U-Net được huấn luyện ở 352 × 352 pixel, không dùng tăng cường dữ liệu, cũng với 20 seed. Hai mô hình này chỉ dùng để đặt kết quả của đề tài vào bối cảnh. Chúng không phải đối tượng cải tiến.

6.1.2. Tập đánh giá

Mọi kết quả trong mục 6.2 đến 6.6 được tính trên **tập kiểm định của BG20**, gồm 160 ảnh: 120 ảnh có polyp với 127 vùng polyp và 40 ảnh nền. Bộ dữ liệu không có tập kiểm thử riêng. Tập kiểm định vừa được dùng để chọn epoch tốt nhất, vừa dùng để báo cáo, nên kết quả tuyệt đối có thể hơi lạc quan. Do hai mô hình chịu cùng thiên lệch này, phép so sánh giữa chúng vẫn công bằng. Mục 6.7 bổ sung đánh giá trên bốn bộ dữ liệu không tham gia huấn luyện.

6.1.3. Chỉ số chính và cách trích số

Chỉ số chính của đề tài là **mask mAP50–95**, tức mAP của mặt nạ lấy trung bình trên 10 ngưỡng IoU từ 0,50 đến 0,95 (mục 2.7.3). Chỉ số này được chọn trước khi xem kết quả vì nó phạt cả việc khoanh thiếu lẫn biên mặt nạ lệch. Các chỉ số còn lại đều là chỉ số phụ, chỉ mang tính thăm dò.

Với mỗi lượt huấn luyện, đề tài lấy **epoch có mask mAP50–95 lớn nhất** trong 100 epoch, rồi lấy mọi chỉ số khác ở cùng epoch đó. Quy tắc này có thiên lệch lựa chọn, nên mục 6.2.3 kiểm tra lại với hai quy tắc khác.

6.1.4. Kiểm định thống kê

Với mỗi chỉ số, đề tài tính hiệu ghép cặp $d_i = x_i^{\text{TSVM}} - x_i^{\text{gốc}}$ cho 20 seed, rồi dùng:

- **kiểm định t ghép cặp** [48] (mục 2.8.2) để kiểm tra trung bình của d_i có khác 0 không, kèm khoảng tin cậy 95% (CI 95%);
- **kiểm định Wilcoxon dấu–hạng** [49] (mục 2.8.3) để đối chiếu, vì kiểm định này không cần giả định phân phối chuẩn;
- **cỡ hiệu ứng Cohen** d_z [51] để biết chênh lệch lớn hay nhỏ so với dao động;

– số seed mà TSVM tốt hơn.

Mức ý nghĩa là $\alpha = 0{,}05$. Đề tài kiểm định nhiều chỉ số nhưng không hiệu chỉnh đa kiểm định, nên chỉ kết luận chắc chắn trên chỉ số chính. Các chỉ số phụ chỉ dùng để mô tả xu hướng [50].

Để so sánh độ dao động giữa các seed của hai mô hình, đề tài dùng **kiểm định Levene dạng trung vị** (còn gọi là kiểm định Brown–Forsythe) [57]. Kiểm định này coi hai nhóm 20 seed là độc lập, không tận dụng cấu trúc ghép cặp, nên kết quả chỉ mang tính thăm dò.

Các phép tính dùng thư viện SciPy.

6.1.5. Đánh giá theo pixel

PraNet và U-Net là mô hình phân đoạn ngữ nghĩa, không cho ra từng đối tượng kèm điểm tin cậy, nên không tính được mAP theo cách của Ultralytics. Để so sánh bốn mô hình trên cùng một thước đo, đề tài chạy lại suy luận và tính chỉ số **theo pixel** (Dice, IoU, Precision, Recall, mục 2.7.2):

– **Nhãn:** mặt nạ vẽ lại từ nhãn đa giác ở độ phân giải gốc của ảnh, tức đúng nhãn mà các mô hình được huấn luyện.

– **YOLO:** ảnh vào 640 pixel, ngưỡng tin cậy 0,25, ngưỡng IoU 0,7. Mọi mặt nạ của một ảnh được gộp thành một mặt nạ. Dùng tệp `best.pt` của từng lượt.

– **PraNet, U-Net:** ảnh vào 352 × 352, ngưỡng xác suất 0,5, rồi phóng mặt nạ về kích thước gốc.

– **Cách lấy trung bình:** chỉ số được tính trên **từng ảnh** của 120 ảnh có polyp, lấy trung bình trong mỗi seed, rồi tính trung bình và độ lệch chuẩn qua 20 seed. Nếu ảnh có polyp mà mô hình không dự đoán pixel nào, Dice và IoU của ảnh đó bằng 0. Đây là cách lấy trung bình theo ảnh đã nêu ở mục 2.7.2. Dice gộp toàn bộ pixel của 120 ảnh được ghi kèm để đối chiếu.

– **Ảnh nền:** với 40 ảnh nền, đề tài đếm tỉ lệ ảnh có ít nhất một pixel bị dự đoán là polyp, gọi là tỉ lệ **báo nhầm**.

Trong toàn chương, mọi giá trị được ghi dưới dạng **trung bình ± độ lệch chuẩn qua 20 seed**.

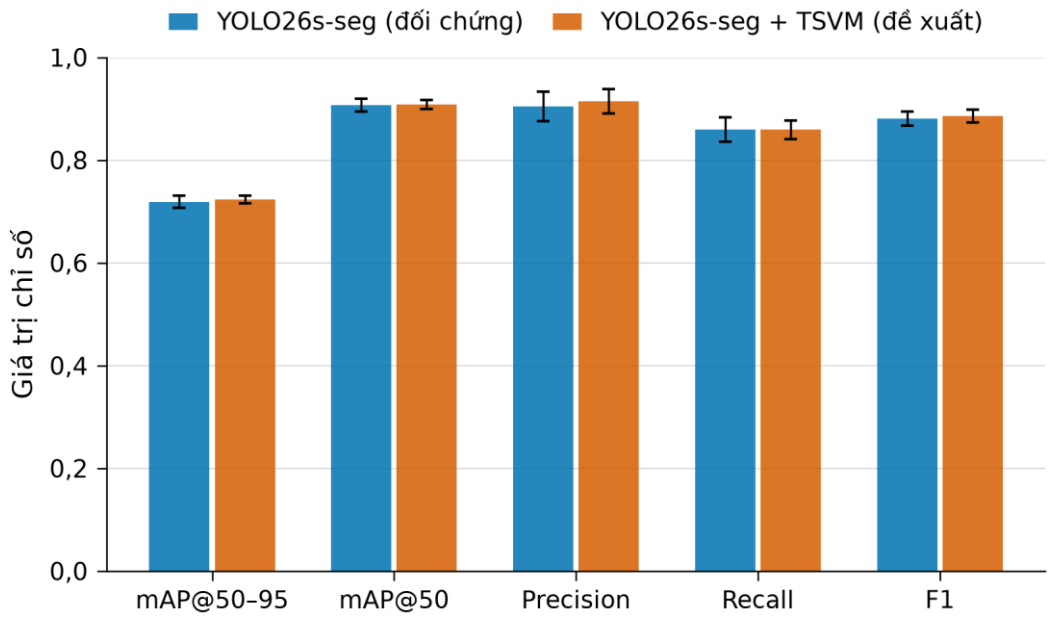
6.2. SO SÁNH MÔ HÌNH GỐC VÀ MÔ HÌNH ĐỀ XUẤT TRÊN 20 SEED

6.2.1. Kết quả tổng hợp

Bảng 6.2: Chỉ số mặt nạ của hai mô hình (20 seed, epoch có mask mAP50–95 lớn nhất)

Chỉ số	YOLO26s-seg	TSVM	Δ (TSVM − gốc)	CI 95% của Δ	p (t)	p (Wilcoxon)	Seed TSVM tốt hơn
Mask mAP50–95	0,7189 ± 0,0118	0,7232 ± 0,0072	**+0,0043**	[−0,0002; 0,0088]	0,061	0,070	13/20
Mask mAP50	0,9069 ± 0,0125	0,9083 ± 0,0086	+0,0014	[−0,0053; 0,0080]	0,668	0,812	10/20
Mask Precision	0,9041 ± 0,0287	0,9145 ± 0,0233	+0,0104	[−0,0086; 0,0293]	0,266	0,498	11/20
Mask Recall	0,8594 ± 0,0241	0,8590 ± 0,0179	−0,0004	[−0,0126; 0,0119]	0,948	0,926	9/20
Mask F1	0,8806 ± 0,0142	0,8856 ± 0,0129	+0,0049		0,259	0,388	12/20
Box mAP50–95	0,7228 ± 0,0159	0,7273 ± 0,0134	+0,0045	[−0,0031; 0,0121]	0,227	0,105	13/20
Box mAP50	0,9006 ± 0,0116	0,9014 ± 0,0083	+0,0008	[−0,0053; 0,0069]	0,778	1,000	11/20

Nguồn: nhóm thực hiện tính toán từ kết quả thực nghiệm.



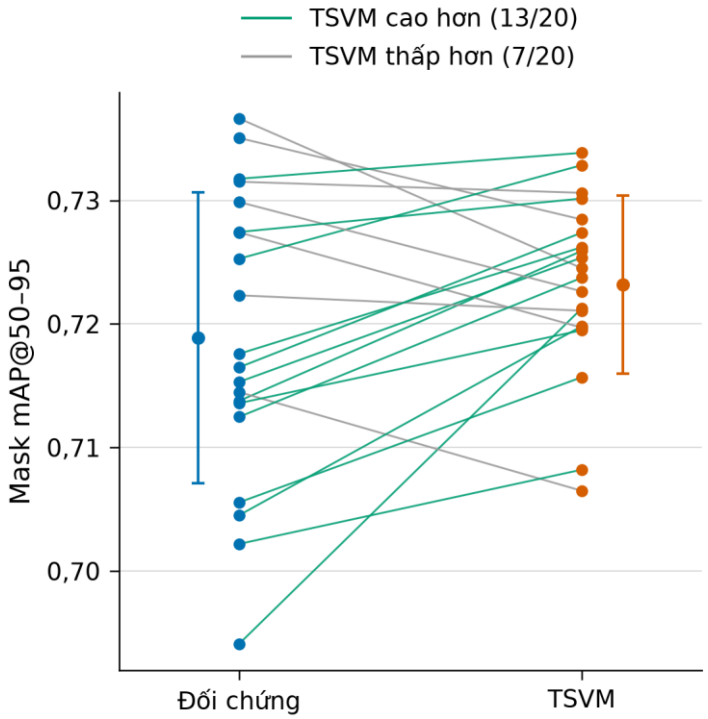
Hình 6.1: Trung bình các chỉ số mặt nạ của hai mô hình trên 20 seed. Thanh lỗi là ±1 độ lệch chuẩn qua các seed.

Trên chỉ số chính, mô hình đề xuất đạt 0,7232, cao hơn mô hình gốc 0,0043, tức khoảng 0,6% tương đối. TSVM cao hơn ở 13/20 seed. Tuy vậy, khoảng tin cậy 95% của hiệu số vẫn chứa 0, kiểm định t cho $p = 0{,}061$ và Wilcoxon cho $p = 0{,}070$. Cỡ hiệu ứng $d_z = 0{,}44$, ở mức nhỏ đến vừa theo quy ước của Cohen (được

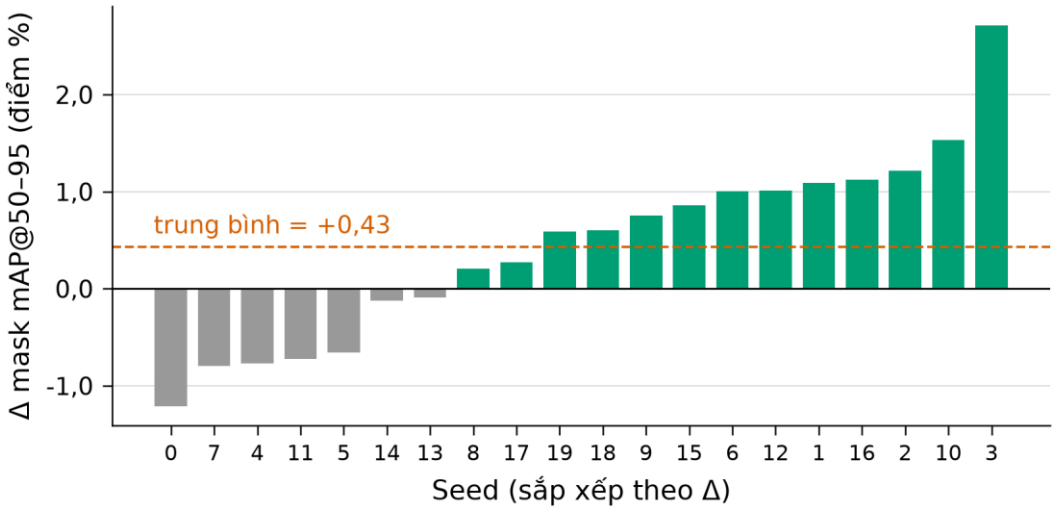
Lakens [51] trình bày lại). **Như vậy, ở mức ý nghĩa 0,05, chưa đủ bằng chứng để khẳng định TSVM tốt hơn mô hình gốc.** Kết quả chỉ cho thấy một xu hướng tăng nhỏ.

Cần lưu ý chiều ngược lại: $p > 0{,}05$ cũng không chứng minh hai mô hình tương đương. Khoảng tin cậy cho thấy hiệu số thật có thể nằm trong khoảng từ gần 0 đến khoảng +0,009.

Các chỉ số phụ đi cùng chiều nhưng yếu hơn. Precision tăng khoảng 1 điểm phần trăm, Recall gần như không đổi, F1 tăng 0,005. Ở mAP50, hai mô hình gần như bằng nhau (+0,0014). Điều này gợi ý rằng phần chênh lệch nhỏ của mAP50–95 nằm ở các ngưỡng IoU cao, tức mức khớp biên mặt nạ, chứ không ở khả năng tìm ra polyp. Không chỉ số phụ nào có ý nghĩa thống kê.



Hình 6.2: Mask mAP50–95 của từng seed. Mỗi đường nối hai lượt cùng seed của hai mô hình.



Hình 6.3: Hiệu số mask mAP50–95 (TSVM − gốc) của từng seed, sắp xếp từ nhỏ đến lớn.

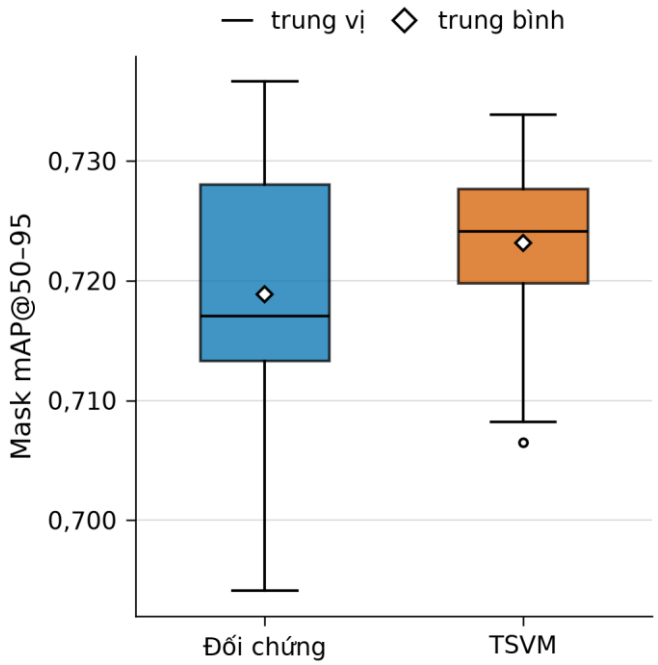
Hình 6.2 và Hình 6.3 cho thấy hiệu số dao động khá rộng, từ −0,0121 (seed 0) đến +0,0272 (seed 3). Lượt tốt nhất của mô hình gốc (0,7366, seed 0) còn cao hơn lượt tốt nhất của TSVM (0,7339, seed 8). Các hiệu số dương lớn nhất rơi vào những seed mà mô hình gốc cho kết quả thấp, ví dụ seed 3 (0,6941 so với 0,7213) và seed 10 (0,7045 so với 0,7198).

6.2.2. Độ ổn định giữa các seed

Bảng 6.3: Mức dao động của mask mAP50–95 qua 20 seed

	YOLO26s-seg	**TSVM**
Độ lệch chuẩn	0,0118	0,0072
Nhỏ nhất	0,6941	0,7065
Lớn nhất	0,7366	0,7339
Khoảng (lớn nhất − nhỏ nhất)	0,0425	0,0274

Nguồn: nhóm thực hiện tính toán từ kết quả thực nghiệm.



Hình 6.4: Phân bố mask mAP50–95 qua 20 seed của hai mô hình.

Độ lệch chuẩn của TSVM nhỏ hơn khoảng 39%. Kiểm định Levene dạng trung vị cho $p = 0{,}026$. Đây là kết quả duy nhất trong phần so sánh trên tập kiểm định có $p < 0{,}05$. Do kiểm định này bỏ qua cấu trúc ghép cặp và không phải chỉ số chính, đề tài chỉ coi đây là dấu hiệu thăm dò: **TSVM cho kết quả ít phụ thuộc vào seed hơn**.

Dấu hiệu này cũng có giới hạn:

– Nó chỉ đúng ở epoch tốt nhất. Ở epoch 100, độ lệch chuẩn của hai mô hình gần bằng nhau (0,0132 và 0,0130).

– Trong 100 epoch, có 34 epoch mà độ lệch chuẩn của TSVM lớn hơn của mô hình gốc.

– Trên dữ liệu ngoài (mục 6.7), độ lệch chuẩn của TSVM không nhỏ hơn.

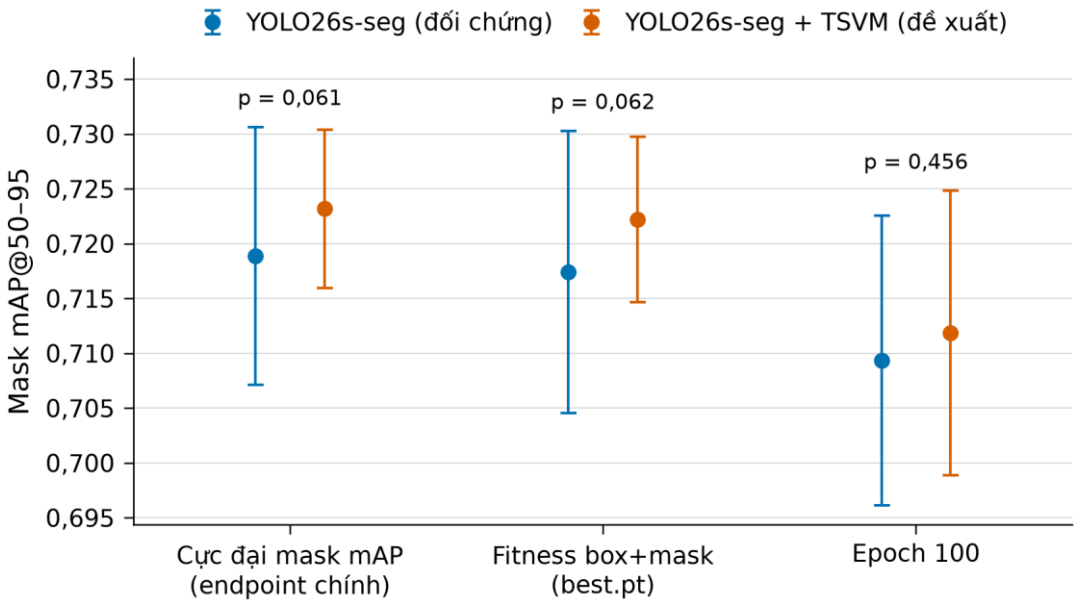
6.2.3. Độ nhạy theo quy tắc chọn epoch

Chọn epoch tốt nhất trên chính tập dùng để báo cáo có thể làm kết quả lạc quan. Bảng 6.4 so sánh ba quy tắc chọn epoch.

Bảng 6.4: Mask mAP50–95 theo ba quy tắc chọn epoch

Quy tắc	YOLO26s-seg	TSVM	Δ	p (t)	p (Wilcoxon)	Seed TSVM tốt hơn
Mask mAP50–95 lớn nhất (chỉ số chính)	0,7189 ± 0,0118	0,7232 ± 0,0072	+0,0043	0,061	0,070	13/20
Epoch của `best.pt` (chỉ số tổng hợp hộp và mặt nạ)	0,7174 ± 0,0129	0,7222 ± 0,0075	+0,0048	0,062	0,076	14/20
Epoch 100 (cuối huấn luyện)	0,7093 ± 0,0132	0,7119 ± 0,0130	+0,0025	0,456	0,349	12/20

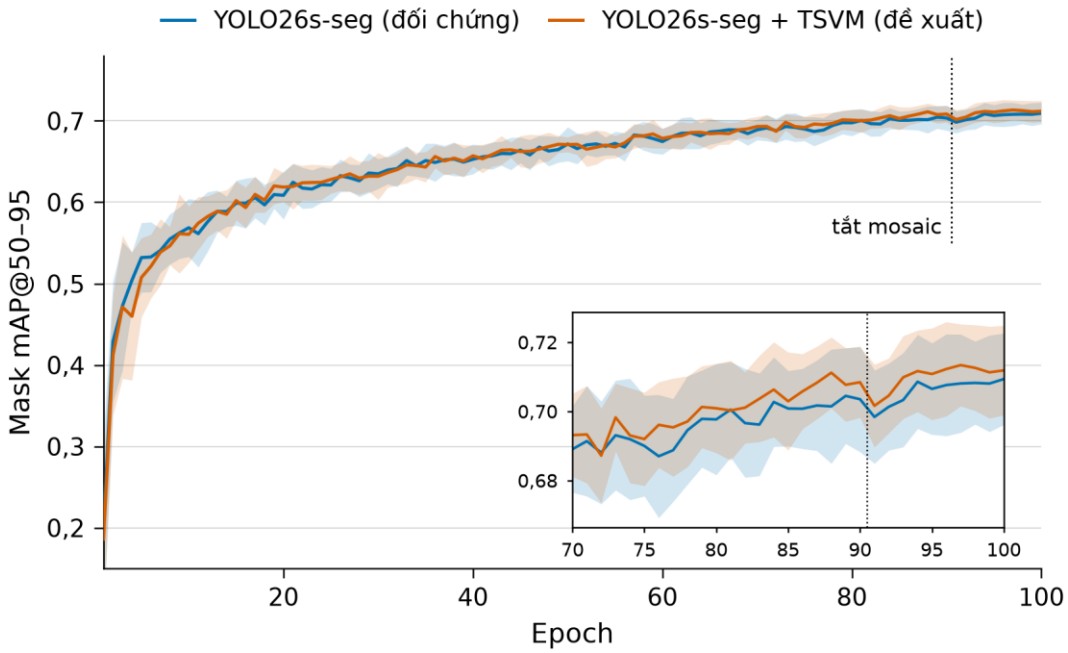
Nguồn: nhóm thực hiện tính toán từ kết quả thực nghiệm.



Hình 6.5: Mask mAP50–95 theo ba quy tắc chọn epoch. Điểm là trung bình, thanh là ±1 độ lệch chuẩn.

Hai quy tắc dựa trên epoch tốt nhất cho kết luận gần như nhau. Nếu lấy epoch cuối, khoảng cách thu hẹp còn +0,0025 và không còn dấu hiệu ổn định hơn. Như vậy kết luận "xu hướng tăng nhỏ, chưa có ý nghĩa thống kê" không phụ thuộc vào cách chọn epoch.

6.2.4. Quá trình hội tụ



Hình 6.6: Mask mAP50–95 trên tập kiểm định theo epoch. Đường là trung bình 20 seed, miền tô là ±1 độ lệch chuẩn. Ô nhỏ phóng to đoạn epoch 70–100.

Bảng 6.5: Mask mAP50–95 tại một số epoch

Epoch	YOLO26s-seg	TSVM
5	0,5320 ± 0,0427	0,5076 ± 0,0485
10	0,5687 ± 0,0355	0,5606 ± 0,0347
30	0,6349 ± 0,0183	0,6320 ± 0,0287
50	0,6716 ± 0,0208	0,6709 ± 0,0091
70	0,6890 ± 0,0125	0,6931 ± 0,0120
90	0,7035 ± 0,0152	0,7084 ± 0,0102
100	0,7093 ± 0,0132	0,7119 ± 0,0130

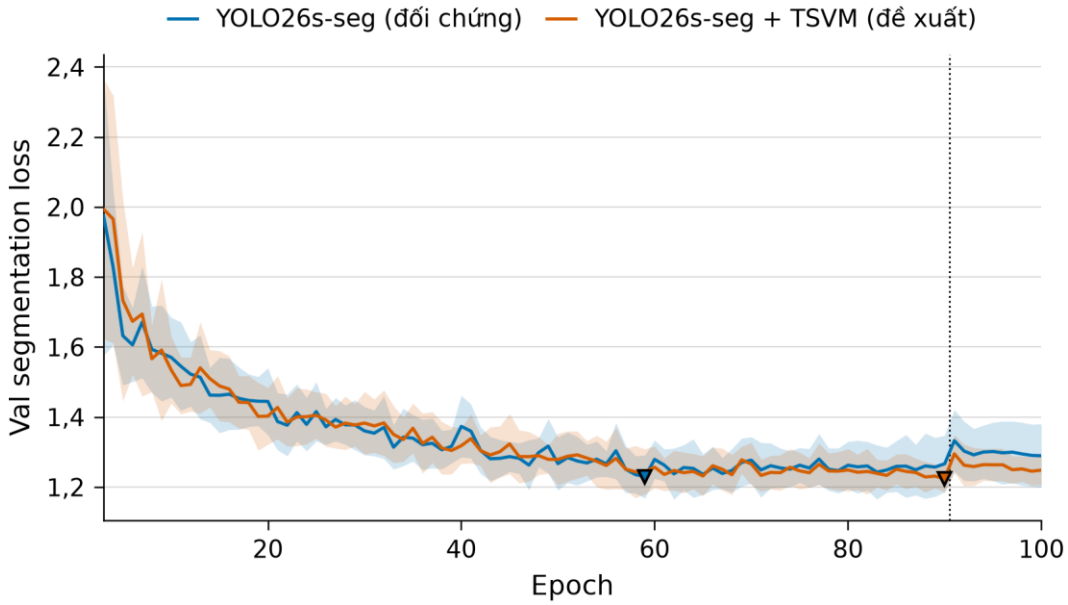
Nguồn: nhóm thực hiện tính toán từ kết quả thực nghiệm.

Hình 6.6 và Bảng 6.5 cho thấy ba giai đoạn:

1. **Khoảng 10 epoch đầu:** TSVM tăng chậm hơn. Điều này phù hợp với cách khởi tạo ở mục 3.5: phần mới của tầng 10 có hơn một triệu tham số khởi tạo ngẫu nhiên, và hệ số $\gamma = 0{,}001$ làm khối mới gần như "tắt" lúc đầu.

2. **Epoch 50 đến 81:** hai đường đan xen nhau.
3. **Epoch 82 đến 100:** trung bình của TSVM luôn cao hơn.

Phần lớn các lượt đạt cực đại muộn: trung vị epoch tốt nhất là 88 với mô hình gốc và 92 với TSVM. Khi tắt Mosaic ở epoch 91, mAP của cả hai mô hình giảm nhẹ một epoch rồi tăng lại.



Hình 6.7: Hàm mất mát phân đoạn trên tập kiểm định theo epoch. Dấu ▼ đánh dấu epoch có giá trị trung bình thấp nhất.

Hàm mất mát phân đoạn trên tập kiểm định của mô hình gốc thấp nhất ở epoch 59 (1,2294) rồi tăng lên 1,2893 ở epoch 100. Của TSVM thấp nhất ở epoch 90 (1,2236). Mô hình gốc có dấu hiệu quá khớp sớm hơn trên hàm mất mát, dù mAP của nó vẫn tăng tới cuối. Đây là quan sát mô tả, chưa kiểm định.

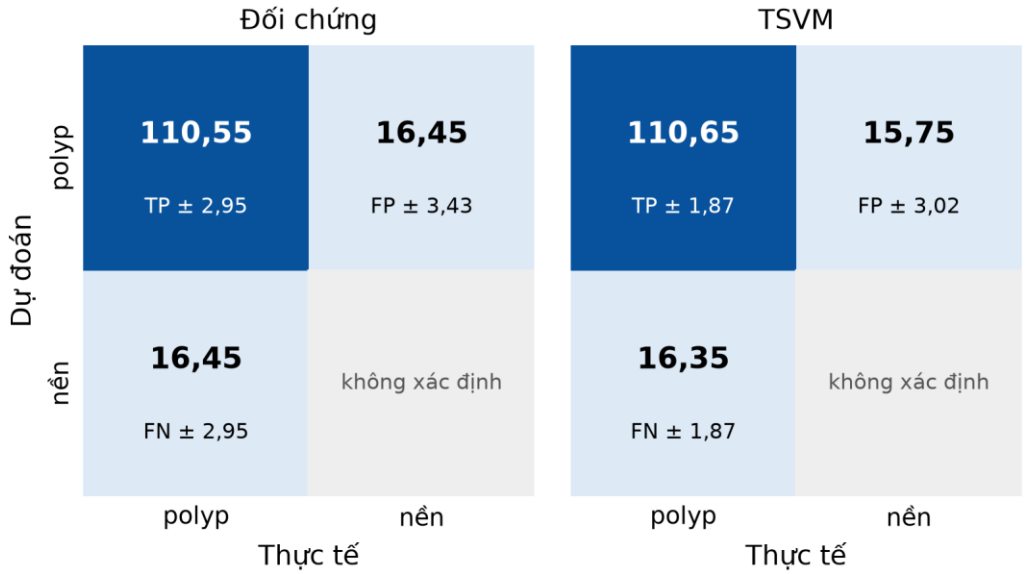
6.2.5. Ma trận nhầm lẫn theo đối tượng

Ma trận nhầm lẫn do Ultralytics tính khi đánh giá `best.pt` ở cuối mỗi lượt. Đơn vị là **vùng polyp**, không phải ảnh.

Bảng 6.6: Số vùng polyp phát hiện đúng, bỏ sót và dự đoán thừa (trên 127 vùng polyp)

	YOLO26s-seg	TSVM	Δ	p (Wilcoxon)
Phát hiện đúng (TP)	110,55 ± 2,95	110,65 ± 1,87	+0,10	1,000
Bỏ sót (FN)	16,45 ± 2,95	16,35 ± 1,87	−0,10	1,000
Dự đoán thừa (FP)	16,45 ± 3,43	15,75 ± 3,02	−0,70	0,671

Nguồn: nhóm thực hiện tính toán từ kết quả thực nghiệm.



Hình 6.8: Ma trận nhầm lẫn trung bình 20 seed. Ô nền–nền để trống vì không xác định được ở mức đối tượng.

Hai mô hình tìm được gần như cùng số polyp, khoảng 110,6/127 vùng (87%). TSVM dự đoán thừa ít hơn trung bình 0,7 vùng, nhưng khác biệt không có ý nghĩa thống kê.

Ô "đúng là nền" (TN) không xác định được: dự đoán thừa có thể nằm trên ảnh nền hoặc ngay trên ảnh có polyp, và một ảnh có thể có nhiều dự đoán thừa. Vì vậy đề tài không tính độ đặc hiệu hay độ chính xác theo ảnh từ bảng này. Tỉ lệ báo nhầm trên ảnh nền được đo riêng ở mục 6.3.

6.3. ĐÁNH GIÁ THEO PIXEL VÀ SO SÁNH VỚI PRANET, U-NET

Bảng 6.7: Chỉ số theo pixel của bốn mô hình trên tập kiểm định (20 seed)

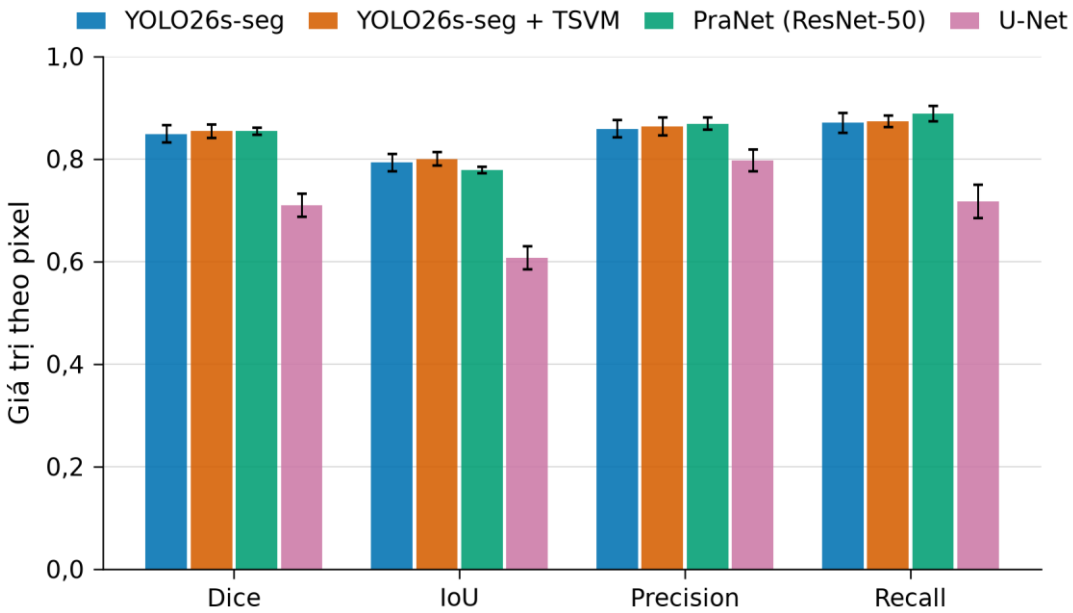
Mô hình	Dice	IoU	Precision	Recall	Dice gộp pixel	Ảnh nền báo nhầm (%)
YOLO26s-seg	0,8486 ± 0,0171	0,7927 ± 0,0170	0,8585 ± 0,0170	0,8703 ± 0,0193	0,8659 ± 0,0313	1,2 ± 1,9
TSVM	0,8540 ± 0,0129	**0,7998 ± 0,0129**	0,8632 ± 0,0173	0,8730 ± 0,0112	**0,8785 ± 0,0243**	1,5 ± 2,1
PraNet	**0,8541 ± 0,0068**	0,7785 ± 0,0063	**0,8687 ± 0,0117**	**0,8881 ± 0,0145**	0,8715 ± 0,0060	**0,8 ± 1,2**
U-Net	0,7097 ± 0,0221	0,6069 ± 0,0221	0,7964 ± 0,0213	0,7167 ± 0,0327	0,7412 ± 0,0185	20,4 ± 6,4

Nguồn: nhóm thực hiện tính toán từ kết quả thực nghiệm.

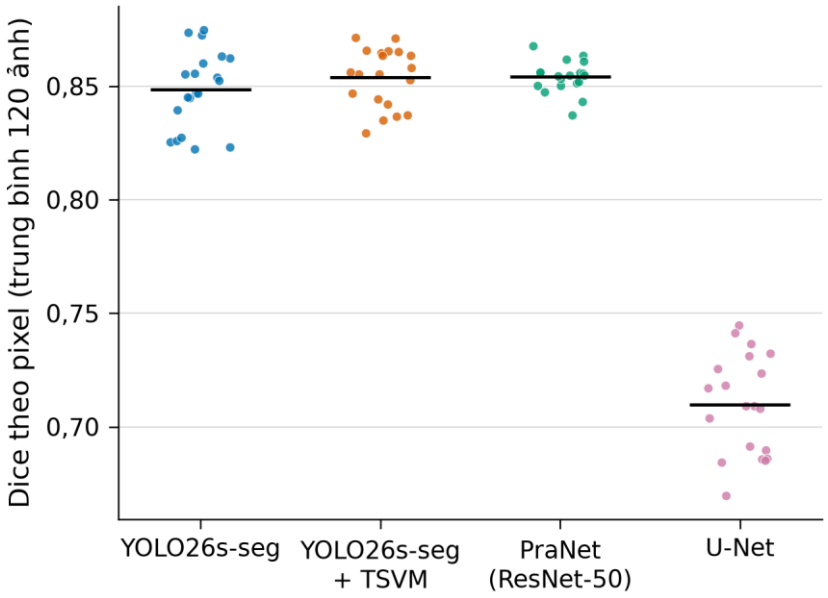
Bảng 6.8: So sánh ghép cặp TSVM với từng mô hình theo pixel (cùng seed)

So với	Chỉ số	Δ (TSVM − mô hình kia)	p (t)	p (Wilcoxon)	Seed TSVM cao hơn
YOLO26s-seg	Dice	+0,0054	0,260	0,246	13/20
	IoU	+0,0071	0,130	0,114	14/20
	Ảnh nền báo nhầm	+0,25 điểm %	0,694	0,793	
PraNet	Dice	−0,0001	0,982	0,927	12/20
	IoU	+0,0213	< 0,001	< 0,001	19/20
	Recall	−0,0151	0,002	0,001	5/20
U-Net	Dice	+0,1443	< 0,001	< 0,001	20/20
	Ảnh nền báo nhầm	−18,9 điểm %	< 0,001	< 0,001	

Nguồn: nhóm thực hiện tính toán từ kết quả thực nghiệm.



Hình 6.9: Dice, IoU, Precision và Recall theo pixel của bốn mô hình. Thanh lỗi là ±1 độ lệch chuẩn qua 20 seed.



Hình 6.10: Dice theo pixel của từng seed.

So với mô hình gốc. Theo pixel, TSVM vẫn có xu hướng tăng nhẹ (Dice +0,005, IoU +0,007, cao hơn ở 13–14/20 seed) nhưng không có ý nghĩa thống kê. Kết luận này khớp với kết luận theo mAP ở mục 6.2.

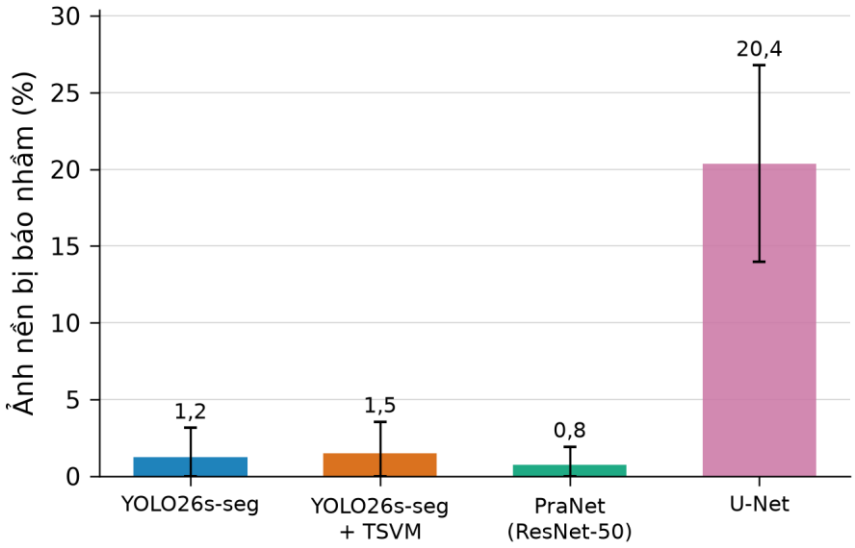
So với PraNet. Dice trung bình của TSVM và PraNet gần như bằng nhau. Hai mô hình mạnh ở hai mặt khác nhau:

– TSVM có **IoU cao hơn** ở 19/20 seed ($p < 0{,}001$). Khi đã khoanh được polyp, mặt nạ của TSVM khớp với nhãn sát hơn.

– PraNet có **Recall cao hơn** ($p = 0{,}002$) và **ổn định hơn nhiều** giữa các seed (độ lệch chuẩn Dice 0,0068 so với 0,0129).

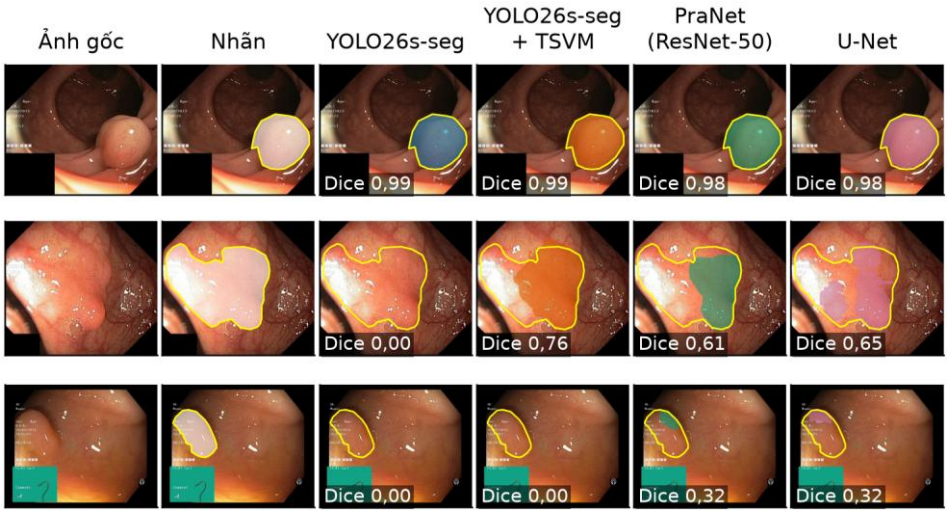
Một nguyên nhân làm Dice trung bình của hai mô hình YOLO bị kéo xuống là các ảnh bị **bỏ sót hoàn toàn**. Ở ngưỡng tin cậy 0,25, YOLO không dự đoán pixel nào trên trung bình khoảng 6,5/120 ảnh có polyp mỗi seed (mô hình gốc 6,45, TSVM 6,65). PraNet chỉ bỏ sót 0,95 ảnh. Mỗi ảnh bỏ sót có Dice bằng 0. Vì vậy, khi gộp pixel cả 120 ảnh (ảnh lớn có trọng số lớn hơn), Dice của TSVM là 0,8785, cao hơn PraNet (0,8715).

So với U-Net. Ba mô hình còn lại đều hơn U-Net rất xa. U-Net còn báo nhầm khoảng 20% ảnh nền.



Hình 6.11: Tỉ lệ ảnh nền bị báo nhầm (có ít nhất một pixel dự đoán là polyp) trên 40 ảnh nền.

Trên 40 ảnh nền, cả hai mô hình YOLO và PraNet đều báo nhầm rất ít (khoảng 1–1,5%, tức dưới một ảnh mỗi seed). Ảnh nền trong tập kiểm định đều lấy từ lớp manh tràng bình thường, khá đồng nhất, nên con số này chưa đại diện cho niêm mạc bình thường nói chung.



Hình 6.12: Ví dụ dự đoán của bốn mô hình (seed 0). Ba ảnh được chọn tự động: ảnh cả bốn mô hình đều tốt, ảnh TSVM hơn mô hình gốc nhiều nhất và ảnh khó nhất.

Hình 6.12 chỉ có tính minh họa. Các ảnh được chọn theo tiêu chí nêu trong chú thích, không phải chọn ngẫu nhiên, nên không dùng hình này để kết luận mô hình nào tốt hơn.

Giới hạn của phép so sánh. Bốn mô hình được huấn luyện với thiết lập khác nhau. YOLO dùng ảnh 640 pixel, có tăng cường dữ liệu và khởi tạo từ COCO.

PraNet và U-Net dùng ảnh 352 pixel, không tăng cường dữ liệu. PraNet trong đề tài dùng xương sống ResNet-50 thay cho Res2Net-50 của bản gốc. Ngưỡng của từng mô hình cũng chưa được tối ưu riêng. Vì vậy kết quả phản ánh các cấu hình cụ thể này, không phải giới hạn tối đa của từng kiến trúc, và không so trực tiếp với số trong bài báo gốc của PraNet.

6.4. CHỈ SỐ TOPOLOGY CỦA MẶT NẠ

6.4.1. Mục đích và cách đo

Tên khối đề xuất có chữ "topology", nhưng mAP và Dice gần như không phạt các lỗi về hình dạng tổng thể của mặt nạ, như mặt nạ bị vỡ thành nhiều mảnh hoặc có lỗ thủng nhỏ. Các lỗi này chỉ chiếm ít pixel. Đề tài đo trực tiếp hai đại lượng topology của mặt nạ:

- b_0: **số mảnh**, tức số thành phần liên thông của vùng polyp (liên thông 8 hướng);
- b_1: **số lỗ**, tức số vùng nền nằm hẳn bên trong vùng polyp (liên thông 4 hướng, không chạm biên ảnh).

Hai đại lượng này là hai số Betti đầu tiên của mặt nạ. Mảnh và lỗ có diện tích dưới 20 pixel không được đếm, trùng với ngưỡng loại đốm nhiễu khi tạo nhãn ở Chương 4. Trên 120 ảnh có polyp, nhãn có 113 ảnh một mảnh, 7 ảnh hai mảnh và không ảnh nào có lỗ.

Một ảnh được coi là **đúng topology** nếu cả b_0 và b_1 của dự đoán đều bằng của nhãn. Giao thức suy luận giống mục 6.1.5.

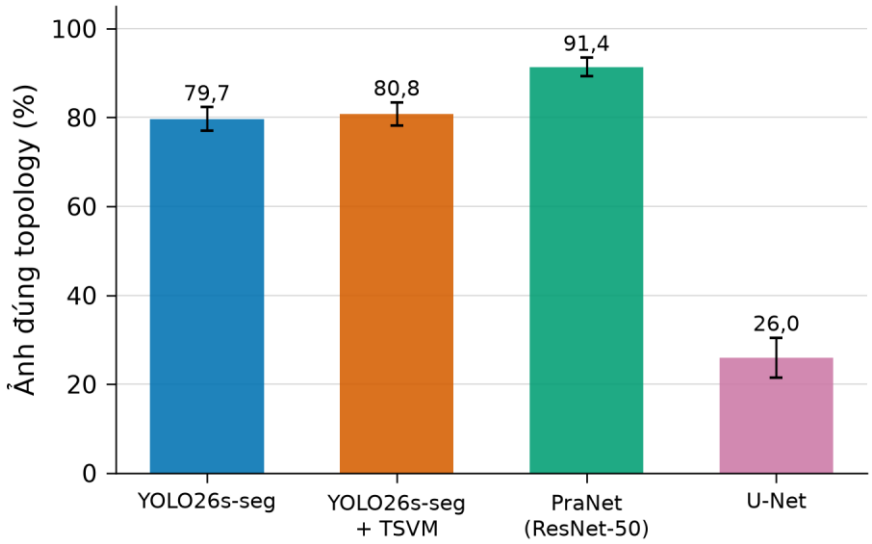
6.4.2. Kết quả

Bảng 6.9: Chỉ số topology của bốn mô hình (ngưỡng 20 pixel, 20 seed)

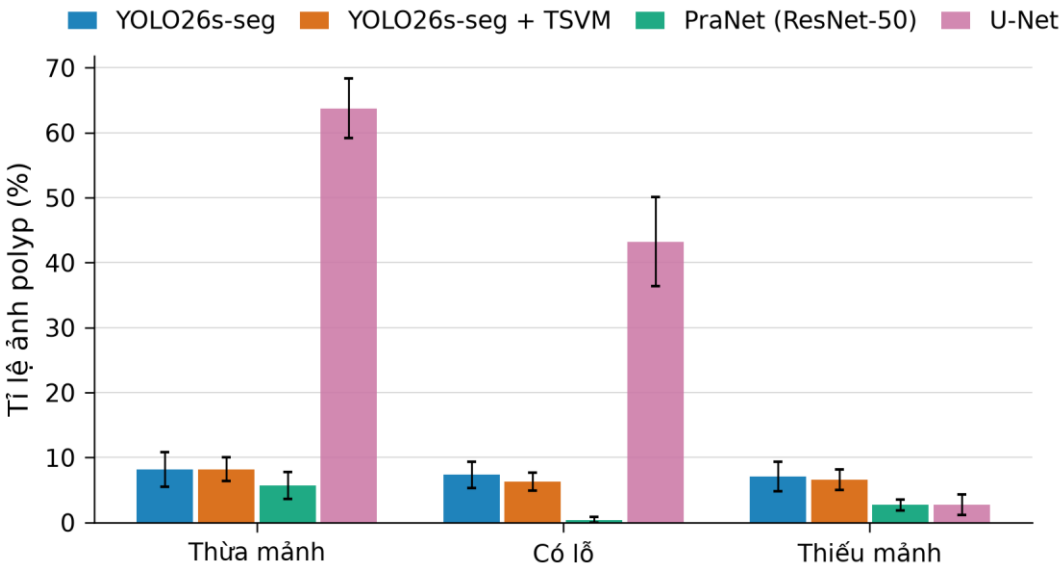
Chỉ số	YOLO26s-seg	TSVM	PraNet	U-Net
Ảnh đúng topology ↑	0,797 ± 0,027	0,808 ± 0,026	**0,914 ± 0,021**	0,260 ± 0,045
Lỗi b_0 trung bình ↓	0,198 ± 0,045	0,191 ± 0,032	**0,086 ± 0,025**	2,745 ± 0,641
Lỗi b_1 trung bình ↓	0,105 ± 0,039	0,094 ± 0,027	**0,004 ± 0,006**	1,215 ± 0,419
Ảnh thừa mảnh ↓	0,082 ± 0,027	0,082 ± 0,018	**0,057 ± 0,021**	0,638 ± 0,046
Ảnh có lỗ ↓	0,073 ± 0,020	0,063 ± 0,014	**0,004 ± 0,005**	0,432 ± 0,069

Chỉ số	YOLO26s-seg	TSVM	PraNet	U-Net
Ảnh thiếu mảnh ↓	0,071 ± 0,023	0,066 ± 0,016	**0,027 ± 0,008**	0,028 ± 0,016

Nguồn: nhóm thực hiện tính toán từ kết quả thực nghiệm.



Hình 6.13: Tỉ lệ ảnh có polyp được dự đoán đúng topology (số mảnh và số lỗ khớp nhãn).



Hình 6.14: Tỉ lệ ảnh mắc từng kiểu lỗi topology: thừa mảnh, có lỗ và thiếu mảnh.

So với mô hình gốc, mọi chỉ số topology của TSVM đều nhích theo chiều có lợi: tỉ lệ ảnh đúng topology tăng 1,1 điểm phần trăm và tỉ lệ ảnh có lỗ giảm 1 điểm phần trăm. Tuy vậy, không chỉ số nào có ý nghĩa thống kê: ảnh đúng topology có $p = 0{,}139$, ảnh có lỗ có $p = 0{,}099$, tỉ lệ ảnh thừa mảnh gần như không đổi. Khi đếm cả mảnh nhỏ hơn 20 pixel, kết luận không đổi (0,758 so với 0,770; $p =$

0,086). **Vì vậy đề tài không khẳng định khối đề xuất cải thiện topology của mặt nạ.**

PraNet có topology tốt nhất rõ rệt: 91% ảnh đúng so với 81% của TSVM, TSVM kém hơn ở cả 20/20 seed ($p < 0{,}001$), và mặt nạ của PraNet hầu như không có lỗ. Một cách giải thích hợp lý là khác biệt ở cách tạo mặt nạ. PraNet xuất một bản đồ xác suất trơn rồi lấy ngưỡng. YOLO ghép mặt nạ từ các mặt nạ mẫu 160 × 160 rồi phóng lên (mục 2.3.4), dễ tạo lỗ nhỏ và mép răng cưa. Giải thích này chưa được kiểm chứng bằng thí nghiệm riêng.

Ở hai mô hình YOLO, lỗi topology chia khá đều giữa ba kiểu (thừa mảnh, có lỗ, thiếu mảnh), mỗi kiểu khoảng 6–8% số ảnh.

6.5. CHI PHÍ TÍNH TOÁN

6.5.1. Số tham số và số phép tính

Bảng 6.10: Số tham số và số phép tính của hai mô hình

	YOLO26s-seg	**TSVM**	**Chênh lệch**
Tham số toàn mô hình	11.434.463	12.254.688	+820.225 (+7,2%)
Tham số tầng 10	990.976 (C2PSA)	1.811.201 (C2TSVMamba)	+82,8%
GFLOPs (`FlopCounterMode` của PyTorch)	34,29	34,82	+1,5%
GFLOPs (cách đếm của Ultralytics)	37,32	37,86	+1,4%
Tệp `best.pt`	22,27 MiB (FP16)	47,37 MiB (FP32)	

Nguồn: nhóm thực hiện tính toán từ kết quả thực nghiệm.

Bộ đếm `FlopCounterMode` của PyTorch chỉ đếm các phép tích chập và nhân ma trận. Các phép theo từng phần tử trong phép quét SS2D không được đếm, nên số GFLOPs của TSVM là **cận dưới**. Bộ đếm của Ultralytics cho số lớn hơn khoảng 3 GFLOPs ở cả hai mô hình do khác cách đếm, nhưng tỉ lệ giữa hai mô hình như nhau. Tệp `best.pt` của TSVM lớn gấp khoảng hai lần chủ yếu vì được lưu ở FP32 (mục 3.8), không phải vì số tham số.

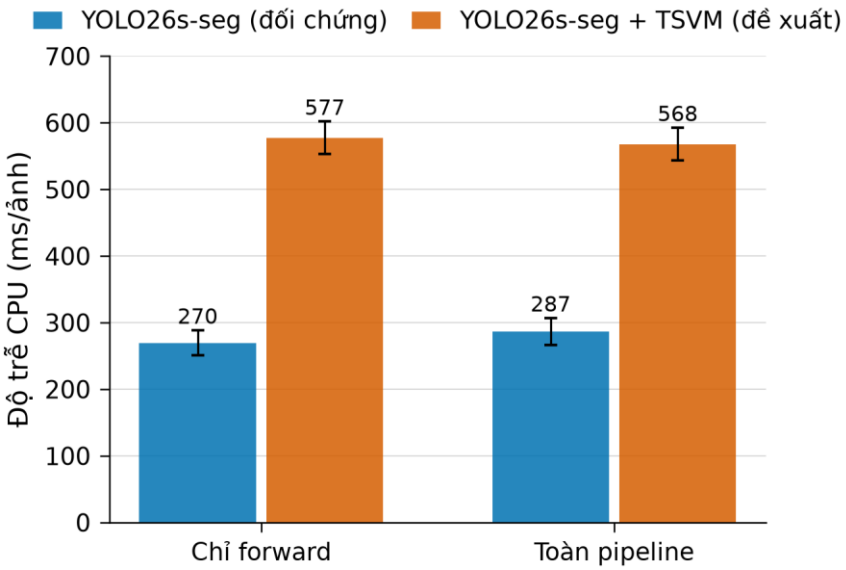
6.5.2. Độ trễ suy luận

Độ trễ được đo trên hai loại phần cứng với checkpoint seed 0, lô 1 ảnh, ảnh vào 640 × 640, FP32.

Bảng 6.11: Độ trễ suy luận của hai mô hình

Phần cứng	Cách đo	YOLO26s-seg (ms)	TSVM (ms)	Tỉ lệ
CPU AMD Ryzen 7 250, 4 luồng	Chỉ lan truyền xuôi, trung bình 100 lần	269,56 ± 18,71	577,36 ± 24,40	2,14 lần
	Cả quy trình dự đoán (tiền xử lý, lan truyền xuôi, hậu xử lý), 20 ảnh	286,56 ± 20,09	567,86 ± 24,45	1,98 lần
GPU RTX 5050 Laptop	Chỉ lan truyền xuôi, trung vị 1.000 lần	26,33	39,86	1,51 lần
	Trung bình ± độ lệch chuẩn	26,99 ± 3,20	41,42 ± 6,82	1,53 lần
	Nhỏ nhất	20,79	29,80	1,43 lần

Nguồn: nhóm thực hiện tính toán từ kết quả thực nghiệm.



Hình 6.15: Độ trễ trên CPU của hai mô hình ở hai chế độ đo. Thanh lỗi là ±1 độ lệch chuẩn qua 100 lần đo.

Trên GPU, mỗi mô hình được đo 5 vòng, mỗi vòng 200 lần sau 50 lần chạy khởi động, xen kẽ hai mô hình và đảo thứ tự mỗi vòng. Trước mỗi khối đo, chương

trình chờ GPU nguội dưới 55 °C và ghi lại nhiệt độ, xung nhịp. Trong cả lần đo, GPU ở 41–51 °C và chạy ở xung nhịp tối đa, nên kết quả không bị ảnh hưởng bởi việc hạ xung do nóng. Tỉ lệ độ trễ giữa hai mô hình ổn định qua các vòng, từ 1,44 đến 1,63 lần.

Một lần đo trước đó trên cùng máy, không kiểm soát nhiệt, cho độ trễ tuyệt đối nhỏ hơn khoảng 1,5–2 lần (mô hình gốc 11,8 ms, TSVM 20,4 ms ở giá trị nhỏ nhất) nhưng tỉ lệ tương tự, khoảng 1,7 lần. Như vậy số mili giây trên máy tính xách tay thay đổi theo trạng thái của máy, còn **tỉ lệ khoảng 1,5–1,7 lần là kết quả đáng tin hơn**.

6.5.3. Nhận xét

Số phép tính chỉ tăng khoảng 1,5%, nhưng độ trễ tăng khoảng 1,5 lần trên GPU và 2,1 lần trên CPU. Như vậy GFLOPs không phản ánh đúng tốc độ của khối Mamba viết bằng PyTorch thuần. Nguyên nhân chính là phép quét song song (mục 3.4): với chuỗi dài 400 vị trí, phép quét chạy 9 vòng nối tiếp, mỗi vòng tạo các tensor trung gian và nhiều phép tính theo từng phần tử. Các phép này bị giới hạn bởi băng thông bộ nhớ và số lần gọi kernel, chứ không bởi số phép nhân. Bản Mamba gốc tránh được chi phí này nhờ kernel CUDA viết riêng [23].

Đặt cạnh kết quả ở mục 6.2, mô hình đề xuất **tốn thêm khoảng 50–110% thời gian suy luận để đổi lấy mức tăng mAP50–95 khoảng 0,004 chưa có ý nghĩa thống kê**. Ở độ trễ khoảng 40 ms trên GPU, mô hình vẫn xử lý được khoảng 25 ảnh mỗi giây khi chỉ tính lan truyền xuôi. Tuy vậy, đây là đo trên một máy, một checkpoint, nên không suy ra tốc độ trên hệ thống nội soi thật.

6.6. THÍ NGHIỆM TÁCH THÀNH PHẦN

6.6.1. Thiết kế

Khối TSVMamba gồm ba thành phần: nhánh SS2D, nhánh hình dạng và cổng $(1 + G)$ (Chương 3). Để biết thành phần nào đóng góp vào kết quả, đề tài huấn luyện thêm ba biến thể, mỗi biến thể bỏ bớt một phần (Bảng 5.6):

- `vmamba_only`: chỉ giữ nhánh SS2D;
- `shape_only`: chỉ giữ nhánh hình dạng;
- `shape_vmamba_no_guidance`: giữ cả hai nhánh nhưng bỏ cổng, hai nhánh chỉ được nối lại rồi trộn bằng tích chập 1 × 1.

Mọi thiết lập khác giữ nguyên như mô hình đầy đủ. Do giới hạn giờ GPU, mỗi biến thể chạy với **5 seed (seed 0–4)**, cùng seed với các lượt chính để so sánh ghép cặp.

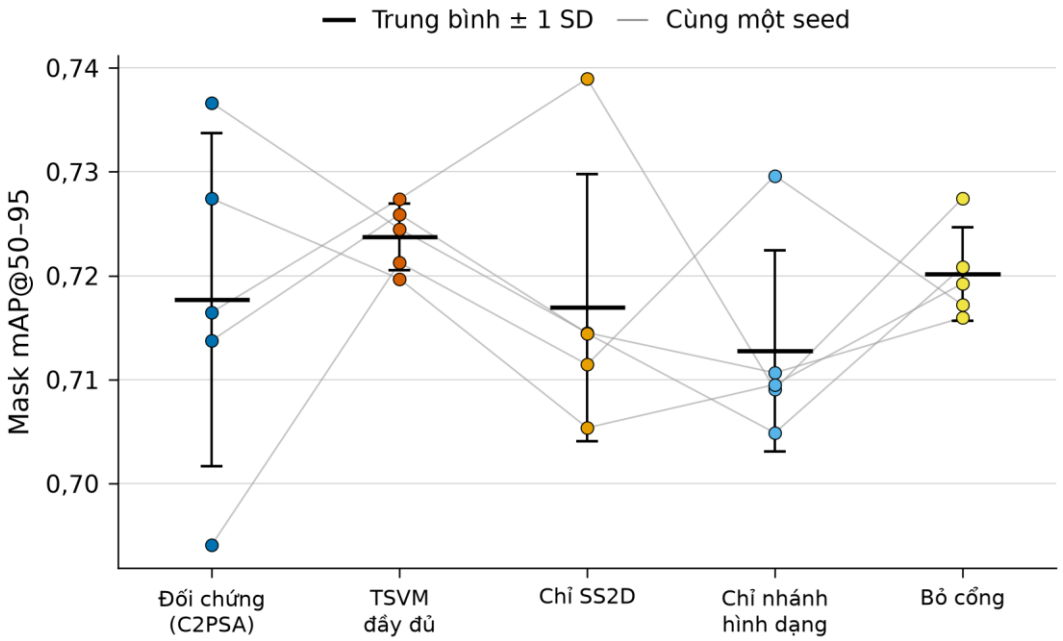
6.6.2. Kết quả

Bảng 6.12: Kết quả thí nghiệm tách thành phần (mask mAP50–95, seed 0–4)

Cấu hình	SS2D	Nhánh hình dạng	Cổng	Tham số tầng 10	Mask mAP50–95	Δ so với gốc	Δ so với đầy đủ	Số seed cao hơn bản đầy đủ
YOLO26s-seg (C2PSA)				990.976	0,7177 ± 0,0160	–	−0,0061	2/5
TSVM đầy đủ	✓	✓	✓	1.811.201	**0,7238 ± 0,0032**	+0,0061	–	–
`vmamba_only`	✓			1.071.873	0,7170 ± 0,0128	−0,0007	−0,0068	1/5
`shape_only`		✓		1.331.969	0,7128 ± 0,0097	−0,0049	−0,0110	1/5
`shape_vmamba_no_guidance`	✓	✓		1.745.409	0,7201 ± 0,0045	+0,0025	−0,0036	1/5

Nguồn: nhóm thực hiện tính toán từ kết quả thực nghiệm.

Trước khi so sánh, đề tài kiểm tra tệp `args.yaml` của 15 lượt tách thành phần. Mọi tham số huấn luyện đều trùng với lượt TSVM chính cùng seed. Chỉ khác tệp cấu hình mô hình và cách ghi đường dẫn đến cùng một tệp trọng số khởi tạo (`bang/18_ablation_cau_hinh_args.csv`).



Hình 6.16: Mask mAP50–95 từng seed (0–4) của năm cấu hình. Đường xám nối kết quả của cùng một seed; vạch ngang đậm là trung bình, thanh dọc là ±1 độ lệch chuẩn qua 5 seed.

So với mô hình đầy đủ, cả ba biến thể đều có mask mAP50–95 trung bình thấp hơn. Mỗi biến thể chỉ cao hơn bản đầy đủ ở 1 trong 5 seed. Trả lời lần lượt ba câu hỏi đặt ra ở mục 6.6.1:

1. **Bỏ nhánh hình dạng (`vmamba_only`):** trung bình giảm 0,0068 so với bản đầy đủ và gần bằng mô hình gốc (−0,0007). Độ lệch chuẩn tăng lên 0,0128, gấp khoảng bốn lần bản đầy đủ. Seed 1 của biến thể này đạt 0,7390, cao nhất trong cả 25 lượt, trong khi bốn seed còn lại đều thấp hơn bản đầy đủ. Như vậy, chỉ thay attention bằng phép quét SS2D chưa cho thấy lợi ích.
2. **Bỏ SS2D (`shape_only`):** đây là cấu hình có trung bình thấp nhất, kém bản đầy đủ 0,0110 và kém cả mô hình gốc 0,0049. Nhánh hình dạng dùng riêng không thay được vai trò của khối ngữ cảnh toàn cục.
3. **Bỏ cổng (`shape_vmamba_no_guidance`):** trung bình giảm ít nhất (−0,0036), độ lệch chuẩn vẫn nhỏ (0,0045). Biến thể này đứng thứ hai, sau bản đầy đủ. Điều này gợi ý rằng phần lớn khác biệt đến từ việc dùng cùng lúc hai nhánh; cổng $(1 + G)$ góp thêm một phần nhỏ, theo cùng chiều ở 4/5 seed.

Ba kết quả trên cho thấy một **xu hướng**: hai nhánh bổ sung cho nhau, và bản đầy đủ vừa có trung bình cao nhất vừa dao động ít nhất giữa các seed. Tuy nhiên, không biến thể nào khác bản đầy đủ có ý nghĩa thống kê. Kiểm định t ghép cặp

cho $p = 0{,}218$ (`vmamba_only`), $p = 0{,}101$ (`shape_only`) và $p = 0{,}085$ (bỏ cổng). Với 5 cặp, kiểm định Wilcoxon hai phía không thể cho p nhỏ hơn 0,0625.

Thời gian huấn luyện cũng tách được theo thành phần. Một lượt 100 epoch của `shape_only` mất khoảng 2,4 giờ, gần bằng mô hình gốc (2,3 giờ). Một lượt `vmamba_only` mất khoảng 4,6 giờ, bằng bản đầy đủ. Như vậy, gần như toàn bộ chi phí thời gian tăng thêm nằm ở SS2D, khớp với nhận xét về phép quét ở mục 6.5.3.

6.6.3. Cách đọc kết quả

Với 5 seed, kiểm định ghép cặp có lực kiểm định thấp. Ở mục 6.2, ngay cả 20 seed cũng chưa đủ để thấy chênh lệch 0,004 có ý nghĩa thống kê. Vì vậy, phần tách thành phần chỉ dùng để **mô tả xu hướng**: biến thể nào gần mô hình đầy đủ, biến thể nào gần mô hình gốc. Nếu khoảng cách giữa các cấu hình nhỏ hơn độ lệch chuẩn giữa các seed (khoảng 0,007–0,012 ở mục 6.2), không nên kết luận thành phần nào quan trọng hơn.

Ngoài ra, các biến thể có số tham số tầng 10 chênh nhau tới khoảng 740 nghìn. Nếu một biến thể tốt hơn, chưa thể tách được phần do cấu trúc và phần do có thêm tham số.

6.7. KẾT QUẢ TRÊN DỮ LIỆU THỰC TẾ

6.7.1. Dữ liệu và giao thức

Để đánh giá khả năng tổng quát hóa, đề tài lấy nguyên các checkpoint đã huấn luyện trên BG20, chạy trên bốn bộ dữ liệu **khác nguồn** mà không huấn luyện lại hay tinh chỉnh.

Bảng 6.13: Các bộ dữ liệu ngoài

Bộ dữ liệu	Số ảnh dùng	Kích thước ảnh	Đặc điểm
CVC-ClinicDB [13]	612	384 × 288	Khung hình từ video nội soi
CVC-ColonDB [14]	380	574 × 500	Khung hình từ video nội soi
ETIS-Larib [15]	196	1.225 × 966	Nhiều polyp nhỏ, khó
PolypGen [12]	1.537 (1.412 có polyp, 125 không polyp)	Tới 1.920 × 1.080	Ảnh đơn từ 6 trung tâm (C1–C6)

Nguồn: đề tài thống kê. Riêng PolypGen chỉ dùng phần ảnh đơn, không dùng phần chuỗi video. CVC-ColonDB dùng bản 380 ảnh phổ biến trong các benchmark

phân đoạn polyp kể từ PraNet [9]; bài công bố gốc [14] mô tả 300 ảnh từ 15 polyp.

Ba bộ CVC-ClinicDB, CVC-ColonDB và ETIS-Larib chỉ có ảnh có polyp. PolypGen có 125 ảnh mà mặt nạ rỗng. Các ảnh này không tính vào Dice và IoU, mà dùng để đo tỉ lệ báo nhầm. Giao thức suy luận và cách tính chỉ số giống hệt mục 6.1.5, với cả bốn mô hình và 20 seed.

Trước khi đánh giá, đề tài kiểm tra PolypGen có trùng ảnh với Kvasir-SEG không bằng ba cách: mã băm MD5 của tệp, mã băm cảm nhận (pHash, dHash) và tương quan ảnh xám thu nhỏ. Không có cặp ảnh nào trùng, nên không loại ảnh nào.

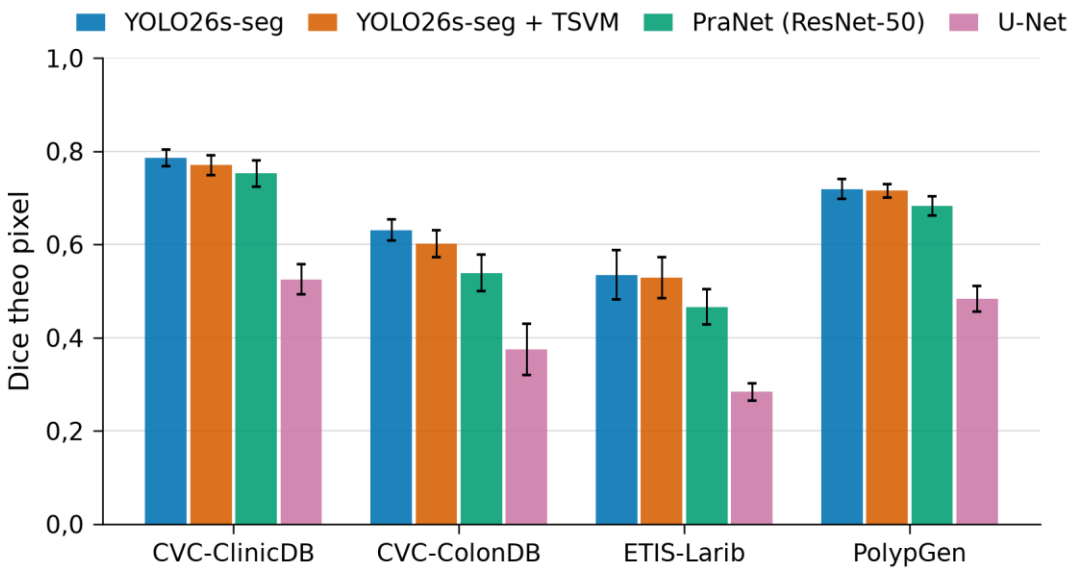
Đây là ảnh công khai thu từ các thiết bị và trung tâm khác Kvasir-SEG, không phải dữ liệu lâm sàng do nhóm thu tại bệnh viện.

6.7.2. Kết quả

Bảng 6.14: Dice và IoU theo pixel trên bốn bộ dữ liệu ngoài (20 seed)

Bộ dữ liệu	Chỉ số	YOLO26s-seg	TSVM	PraNet	U-Net	TSVM − gốc	p (t)	p (Wilcoxon)	Seed TSVM cao hơn
CVC-ClinicDB	Dice	**0,786**	0,770	0,752	0,525	−0,016	0,006	0,006	5/20
	IoU	**0,725**	0,712	0,676	0,425	−0,013	0,010	0,015	6/20
CVC-ColonDB	Dice	**0,631**	0,601	0,539	0,375	−0,030	< 0,001	< 0,001	3/20
	IoU	**0,574**	0,549	0,474	0,304	−0,025	0,001	< 0,001	3/20
ETIS-Larib	Dice	**0,535**	0,529	0,466	0,284	−0,005	0,766	0,498	7/20
	IoU	**0,493**	0,490	0,416	0,225	−0,003	0,851	0,571	8/20
PolypGen (gộp)	Dice	**0,719**	0,715	0,683	0,484	−0,004	0,551	0,246	7/20
	IoU	**0,665**	0,663	0,616	0,402	−0,003	0,605	0,312	7/20

Nguồn: nhóm thực hiện tính toán từ kết quả thực nghiệm.



Hình 6.17: Dice theo pixel của bốn mô hình trên bốn bộ dữ liệu ngoài. Thanh lỗi là ±1 độ lệch chuẩn qua 20 seed.

Bảng 6.15: Tỉ lệ ảnh có polyp bị bỏ sót hoàn toàn (mô hình không dự đoán pixel nào)

Bộ dữ liệu	YOLO26s-seg	TSVM	PraNet	U-Net	TSVM − gốc	p (t)
CVC-ClinicDB	11,3%	13,4%	10,1%	7,9%	+2,1 điểm %	0,003
CVC-ColonDB	28,2%	32,0%	32,0%	27,0%	+3,8 điểm %	< 0,001
ETIS-Larib	37,3%	38,6%	39,4%	20,6%	+1,3 điểm %	0,580
PolypGen (gộp)	18,1%	18,9%	16,3%	11,5%	+0,8 điểm %	0,296

Nguồn: nhóm thực hiện tính toán từ kết quả thực nghiệm.

Từ Bảng 6.14 và Bảng 6.15 rút ra bốn điểm:

1. **Mọi mô hình đều giảm mạnh khi gặp dữ liệu khác nguồn.** Dice của TSVM giảm từ 0,854 trên tập kiểm định xuống còn 0,53–0,77. Tỉ lệ ảnh bị bỏ sót hoàn toàn tăng từ khoảng 5% lên 11–39%. ETIS-Larib khó nhất vì có nhiều polyp nhỏ.
2. **TSVM không tổng quát hóa tốt hơn mô hình gốc.** Trên CVC-ClinicDB và CVC-ColonDB, TSVM **thấp hơn có ý nghĩa thống kê** (Dice −0,016 và −0,030, cả hai kiểm định đều có $p < 0{,}01$) và bỏ sót trọn ảnh nhiều hơn. Trên ETIS-Larib và PolypGen, hai mô hình không khác biệt có ý nghĩa.
3. **Hai mô hình YOLO tốt hơn PraNet và U-Net trên dữ liệu ngoài.** TSVM cao hơn PraNet ở cả bốn bộ ($p < 0{,}05$), dù trên tập kiểm định hai mô hình

này có Dice ngang nhau. Một phần nguyên nhân có thể là YOLO được huấn luyện với tăng cường dữ liệu, còn PraNet thì không.

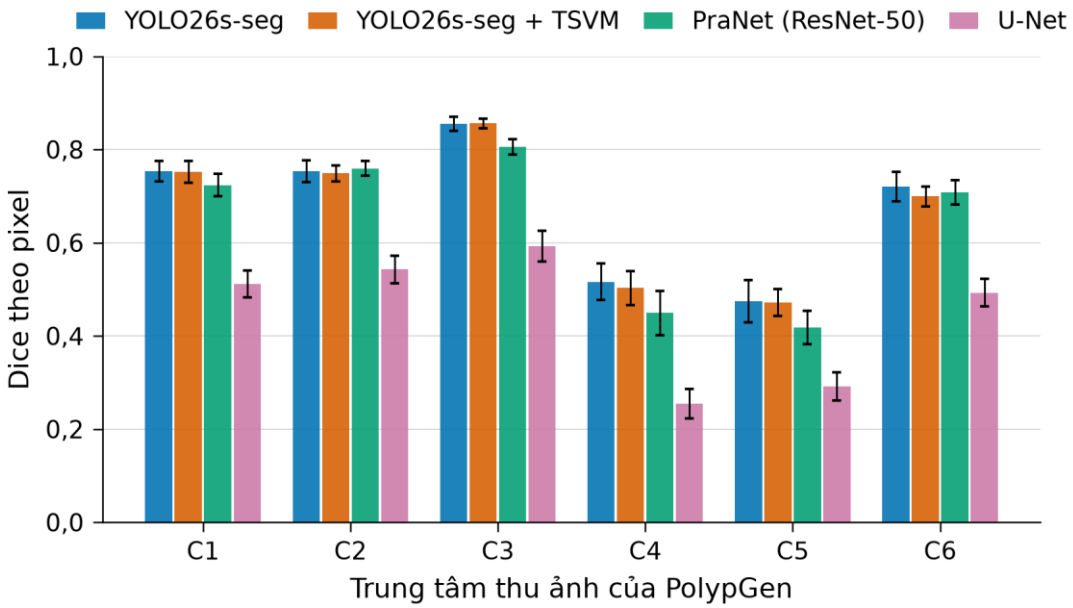
4. **Độ ổn định không còn.** Trên dữ liệu ngoài, độ lệch chuẩn giữa các seed của TSVM không nhỏ hơn mô hình gốc, thậm chí lớn hơn ở CVC-ClinicDB và CVC-ColonDB.

6.7.3. Kết quả theo trung tâm của PolypGen

Bảng 6.16: Dice theo pixel trên từng trung tâm của PolypGen (20 seed)

Trung tâm	Ảnh có polyp	YOLO26s-seg	TSVM	PraNet	U-Net	TSVM − gốc	p (t)	p (Wilcoxon)
C1	251	0,753	0,752	0,724	0,511	−0,001	0,85	0,87
C2	270	0,753	0,749	0,759	0,543	−0,005	0,52	0,50
C3	456	0,855	0,856	0,806	0,593	+0,002	0,69	0,99
C4	146	0,516	0,503	0,449	0,255	−0,013	0,28	0,048
C5	206	0,474	0,472	0,418	0,292	−0,002	0,85	0,45
C6	83	0,720	0,699	0,708	0,492	−0,021	0,032	0,019
Gộp	**1.412**	**0,719**	**0,715**	**0,683**	**0,484**	−0,004	0,55	0,25

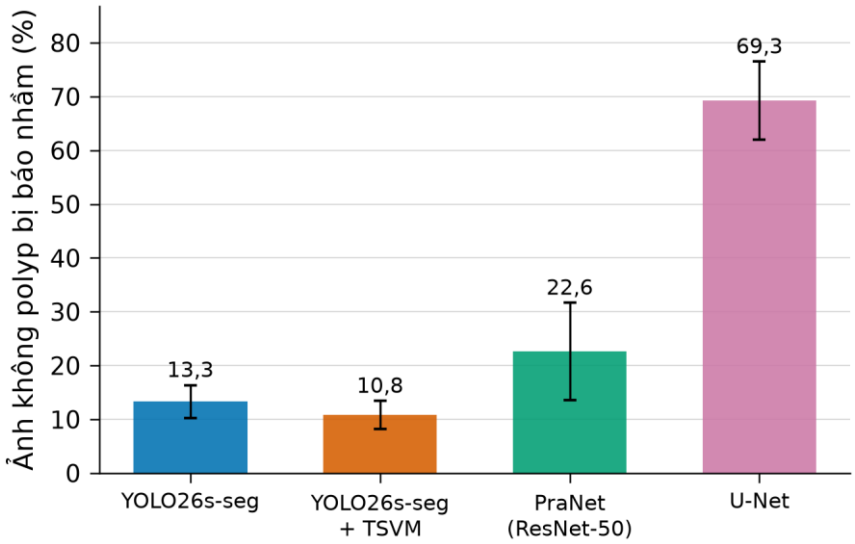
Nguồn: nhóm thực hiện tính toán từ kết quả thực nghiệm.



Hình 6.18: Dice theo pixel trên từng trung tâm của PolypGen.

Chất lượng phân đoạn khác nhau rất nhiều giữa các trung tâm: Dice của TSVM từ 0,47 (C5) đến 0,86 (C3). Điều này cho thấy thiết bị và cách thu ảnh ảnh hưởng mạnh hơn nhiều so với lựa chọn giữa hai mô hình YOLO. Chỉ ở C6 TSVM thấp hơn mô hình gốc với cả hai kiểm định, nhưng C6 chỉ có 83 ảnh và đề tài thực

hiện nhiều phép so sánh, nên cần thận trọng. PraNet chỉ tốt hơn hai mô hình YOLO ở C2.



Hình 6.19: Tỉ lệ ảnh không có polyp của PolypGen (125 ảnh) bị báo nhầm.

Trên 125 ảnh không có polyp của PolypGen, tỉ lệ báo nhầm là 13,3% với mô hình gốc, 10,8% với TSVM, 22,6% với PraNet và 69,3% với U-Net. TSVM báo nhầm ít hơn mô hình gốc 2,5 điểm phần trăm ($p = 0{,}011$; Wilcoxon $p = 0{,}018$), chủ yếu ở C2 và C4. Đây là chỉ số phụ, nên chỉ coi là một dấu hiệu tích cực cần kiểm chứng thêm. Tỉ lệ báo nhầm ở đây cao hơn nhiều so với 40 ảnh nền của tập kiểm định (khoảng 1%), cho thấy ảnh nền của BG20 chưa đủ đa dạng.

6.8. THẢO LUẬN

6.8.1. Trả lời câu hỏi nghiên cứu

Thay C2PSA bằng C2TSVMamba có cải thiện chất lượng phân đoạn không? Trên tập kiểm định của BG20, mô hình đề xuất có mask mAP50–95 cao hơn 0,0043 và cao hơn ở 13/20 seed, nhưng chưa có ý nghĩa thống kê ($p = 0{,}061$). Các đánh giá theo pixel và theo topology cho cùng kết luận: có xu hướng tăng nhỏ, chưa đủ bằng chứng. Trên dữ liệu ngoài, mô hình đề xuất không tốt hơn, và thấp hơn có ý nghĩa ở hai trong bốn bộ.

Mô hình đề xuất có ổn định hơn không? Trên tập kiểm định, độ lệch chuẩn giữa các seed nhỏ hơn khoảng 39% (Levene $p = 0{,}026$, thăm dò). Nhưng dấu hiệu này chỉ có ở epoch tốt nhất và không còn trên dữ liệu ngoài.

Thành phần nào đóng góp? Trên 5 seed, bỏ bất kỳ thành phần nào cũng làm trung bình thấp hơn bản đầy đủ. Bỏ cổng giảm ít nhất (−0,0036), chỉ giữ nhánh

hình dạng giảm nhiều nhất (−0,0110). Như vậy, theo xu hướng, hai nhánh bổ sung cho nhau và cổng góp thêm một phần nhỏ. Không chênh lệch nào có ý nghĩa thống kê.

Chi phí tăng bao nhiêu? Tham số tăng 7,2%, GFLOPs tăng 1,5%, nhưng độ trễ tăng khoảng 1,5 lần trên GPU và 2,1 lần trên CPU, thời gian huấn luyện tăng khoảng 2 lần. Thí nghiệm tách thành phần cho thấy phần thời gian tăng thêm này gần như hoàn toàn do nhánh SS2D.

6.8.2. Vì sao mức cải thiện nhỏ

Đề tài đưa ra một số giải thích. Các giải thích này hợp lý nhưng chưa được kiểm chứng riêng:

- **Phạm vi thay đổi hẹp.** Chỉ tầng 10 thay đổi, nằm ở mức đặc trưng 20 × 20. Khối C2PSA ban đầu đã có cơ chế tự chú ý toàn cục. Ở kích thước 400 vị trí, chi phí bậc hai của attention không lớn, nên lợi thế chi phí tuyến tính của SSM gần như không phát huy.
- **Khởi tạo bất lợi.** Khoảng một triệu tham số mới của tầng 10 được khởi tạo ngẫu nhiên, trong khi C2PSA của mô hình gốc được tiền huấn luyện trên COCO. Với 1.040 ảnh huấn luyện, khối mới có ít dữ liệu để học.
- **Dữ liệu nhỏ và chỉ số gần bão hòa.** mAP50 của cả hai mô hình đã khoảng 0,91, nên dư địa cải thiện chủ yếu nằm ở mức khớp biên.

Giả thuyết khởi tạo cũng có thể giải thích kết quả trên dữ liệu ngoài: khối mới chỉ học từ ảnh Kvasir nên dễ khớp theo đặc điểm riêng của bộ dữ liệu này hơn khối đã tiền huấn luyện trên COCO.

6.8.3. Giới hạn của thực nghiệm

- **Không có tập kiểm thử độc lập.** Tập kiểm định vừa dùng để chọn epoch vừa dùng để báo cáo. Đánh giá trên dữ liệu ngoài bù một phần cho giới hạn này.
- **Một bộ dữ liệu huấn luyện.** Mọi mô hình chỉ học từ Kvasir-SEG cộng 200 ảnh nền của một lớp Kvasir.
- **Thiết lập so sánh khác nhau.** PraNet và U-Net được huấn luyện ở độ phân giải khác và không tăng cường dữ liệu.
- **Nhiều phép so sánh không hiệu chỉnh.** Ngoài chỉ số chính, các kết quả có $p < 0{,}05$ ở chỉ số phụ hoặc ở từng trung tâm của PolypGen chỉ nên coi là gợi ý.

– **Độ trễ đo trên một máy, một checkpoint.** Số mili giây trên máy tính xách tay thay đổi giữa các phiên đo, chỉ tỉ lệ giữa hai mô hình là ổn định. Phép quét chưa được tối ưu bằng kernel CUDA.

– **Tách thành phần chỉ có 5 seed** (mục 6.6).

– **Topology** chỉ đo trên tập kiểm định, nhãn đa giác vốn không có lỗ.

6.9. KẾT CHƯƠNG

Chương 6 đã so sánh mô hình YOLO26s-seg gốc với mô hình tích hợp khối C2TSVMamba qua 20 cặp lượt huấn luyện, đánh giá thêm theo pixel, theo topology, về chi phí tính toán và trên bốn bộ dữ liệu ngoài. Các kết quả chính:

– Mask mAP50–95 tăng 0,0043 (0,7189 lên 0,7232), cao hơn ở 13/20 seed, nhưng **chưa có ý nghĩa thống kê** ($p = 0{,}061$).

– Độ lệch chuẩn giữa các seed nhỏ hơn khoảng 39% (Levene $p = 0{,}026$, thăm dò), chỉ đúng ở epoch tốt nhất.

– Theo pixel, TSVM có Dice ngang PraNet (0,854) và IoU cao hơn PraNet, nhưng topology kém PraNet rõ rệt. Cả ba mô hình đều hơn U-Net rất xa.

– Số phép tính gần như không đổi, nhưng độ trễ tăng khoảng 1,5 lần trên GPU và 2,1 lần trên CPU.

– Trên dữ liệu ngoài, TSVM không tốt hơn mô hình gốc và thấp hơn có ý nghĩa ở CVC-ClinicDB và CVC-ColonDB. Hai mô hình YOLO vẫn tốt hơn PraNet và U-Net.

– Thí nghiệm tách thành phần (5 seed): cả ba biến thể bỏ bớt một thành phần đều có trung bình thấp hơn bản đầy đủ. Bỏ cổng giảm ít nhất, chỉ giữ nhánh hình dạng giảm nhiều nhất. Đây là xu hướng; không chênh lệch nào có ý nghĩa thống kê.

Như vậy, với cách tích hợp và dữ liệu hiện có, khối C2TSVMamba chưa mang lại cải thiện chất lượng đủ rõ để bù cho chi phí suy luận tăng thêm. Phần Kết luận tổng hợp lại các kết quả này và đề xuất hướng phát triển.

KẾT LUẬN

1. KẾT QUẢ ĐẠT ĐƯỢC

Đề tài đã thực hiện đủ chu trình nghiên cứu đặt ra trong đề cương: khảo sát, xây dựng dữ liệu, thiết kế mô hình, thực nghiệm và xây dựng ứng dụng.

Về lý thuyết và khảo sát:

– Hệ thống hóa các hướng phân đoạn polyp: mạng tích chập mã hóa–giải mã, Transformer, họ YOLO và các mô hình không gian trạng thái (Mamba, VMamba).

– Chỉ ra khoảng trống: trong phạm vi khảo sát, chưa thấy công trình nào đánh giá việc đưa khối VMamba vào YOLO26-seg cho phân đoạn polyp bằng nhiều lượt huấn luyện có kiểm định thống kê.

– Trình bày các bước, ưu điểm và nhược điểm của YOLO26-seg và VMamba, kèm ví dụ tính tay cho các khái niệm và độ đo.

Về dữ liệu:

– Xây dựng bộ BG20 gồm 1.200 ảnh: 1.000 ảnh polyp của Kvasir-SEG theo đúng danh sách chia 880/120 của nhóm tác giả, và 200 ảnh manh tràng bình thường làm ảnh nền.

– Chuyển mặt nạ sang nhãn đa giác theo quy trình có kiểm tra toàn vẹn.

– Toàn bộ thực nghiệm dùng chung một bộ dữ liệu và một cách chia.

Về mô hình:

– Thiết kế và cài đặt khối C2TSVMamba thay khối C2PSA ở tầng 10 của YOLO26s-seg. Khối gồm nhánh SS2D quét bốn hướng, nhánh trích đặc trưng hình dạng và một cổng điều biến.

– Bản cài đặt chỉ dùng PyTorch, có ba chế độ tách thành phần dựng sẵn.

– Phát hiện lỗi tràn số khi lưu checkpoint ở FP16 và sửa bằng cách lưu FP32.

Về thực nghiệm:

– Huấn luyện 40 lượt chính: 2 mô hình × 20 seed.

– Huấn luyện 15 lượt tách thành phần: 3 biến thể × 5 seed.

– Huấn luyện PraNet và U-Net, mỗi mô hình 20 seed.

– Đánh giá đủ các độ đo trong đề cương: mask mAP50–95, mAP50, Precision, Recall, F1, Dice, IoU, số tham số, FLOPs và thời gian suy luận trên CPU và GPU. Đề tài đo thêm chỉ số topology của mặt nạ.

– Đánh giá khả năng tổng quát hóa trên bốn bộ dữ liệu ngoài với tổng cộng 2.725 ảnh.

– Mọi con số trong báo cáo đều tính lại được từ tệp kết quả gốc bằng script.

Về ứng dụng: xây dựng PolypVision gồm ba thành phần:

– dịch vụ suy luận FastAPI dùng chung;

– website Spring Boot lưu lịch sử trong MongoDB;

– ứng dụng di động Flutter.

Ứng dụng cho phép tải ảnh, hiển thị mặt nạ, đường biên và độ tin cậy, đồng thời so sánh mô hình gốc với mô hình tích hợp VMamba trên cùng một ảnh.

2. ĐÁNH GIÁ KẾT QUẢ

Đối chiếu với bốn câu hỏi nghiên cứu nêu ở phần Mở đầu:

CH1. Mô hình tích hợp có chính xác hơn mô hình gốc không?

– Trên tập kiểm định của BG20, mask mAP50–95 trung bình tăng từ 0,7189 lên 0,7232 ($\Delta = +0{,}0043$). Mô hình tích hợp cao hơn ở 13/20 seed.

– Chênh lệch này **chưa có ý nghĩa thống kê**: khoảng tin cậy 95% là [−0,0002; 0,0088], $p = 0{,}061$.

– Đánh giá theo pixel (Dice 0,854 so với 0,849) và theo topology cho cùng kết luận: có xu hướng tăng nhỏ, chưa đủ bằng chứng.

– Kết quả ổn định hơn giữa các seed: độ lệch chuẩn nhỏ hơn khoảng 39%, Levene $p = 0{,}026$. Đây là kết quả thăm dò và chỉ đúng ở epoch tốt nhất.

CH2. Mỗi thành phần đóng góp bao nhiêu? Trên 5 seed:

– Cả ba biến thể bỏ bớt một thành phần đều có trung bình thấp hơn bản đầy đủ.

– Bỏ cổng giảm ít nhất (−0,0036). Chỉ giữ nhánh SS2D giảm 0,0068. Chỉ giữ nhánh hình dạng giảm nhiều nhất (−0,0110).

– Như vậy, theo xu hướng, hai nhánh bổ sung cho nhau và cổng góp thêm một phần nhỏ.

– Với 5 seed, không chênh lệch nào có ý nghĩa thống kê. Vì vậy đây là kết quả thăm dò, chưa khẳng định được vai trò của từng thành phần.

CH3. Chi phí tăng bao nhiêu?

– Số tham số tăng 7,2%, số phép tính chỉ tăng khoảng 1,5%.

– Độ trễ suy luận tăng khoảng 1,5 lần trên GPU và 2,1 lần trên CPU. Thời gian huấn luyện tăng khoảng gấp đôi.

– Phần chi phí thời gian này gần như hoàn toàn do phép quét SS2D viết bằng PyTorch thuần.

CH4. Mô hình có chạy được trong ứng dụng và trên ảnh ngoài không?

– Cả hai checkpoint được nạp và chạy trong PolypVision trên Web và di động.

– Trên bốn bộ dữ liệu ngoài, mọi mô hình đều giảm chất lượng rõ rệt: Dice của mô hình tích hợp còn 0,53–0,77.

– Mô hình tích hợp **không tổng quát hóa tốt hơn** mô hình gốc. Nó thấp hơn có ý nghĩa thống kê trên CVC-ClinicDB và CVC-ColonDB, và không khác biệt có ý nghĩa trên ETIS-Larib và PolypGen. Điểm tích cực duy nhất là báo nhầm ít hơn trên ảnh không có polyp của PolypGen (10,8% so với 13,3%).

– Hai mô hình YOLO vẫn tốt hơn PraNet và U-Net trên dữ liệu ngoài.

Tổng hợp.

– **Về quy trình, đề tài đã hoàn thành các mục tiêu đề ra:** khảo sát, dữ liệu, mô hình, thực nghiệm và ứng dụng.

– **Về kết quả, mục tiêu cải thiện chất lượng mặt nạ chưa đạt được ở mức có thể khẳng định.** Với cách tích hợp và dữ liệu hiện có, khối C2TSVMamba chỉ cho xu hướng tăng nhỏ trên tập kiểm định. Mức tăng đó chưa đủ bù cho chi phí suy luận tăng thêm, và không giữ được trên dữ liệu khác nguồn.

– Đề tài báo cáo trung thực kết quả này, thay vì chọn một lần huấn luyện có lợi. Đây là một kết quả có giá trị cho các nghiên cứu tiếp theo theo hướng tích hợp Mamba vào mô hình phát hiện thời gian thực.

3. HẠN CHẾ

Về dữ liệu và thiết kế thực nghiệm:

– Không có tập kiểm thử riêng. Tập kiểm định vừa dùng để chọn epoch vừa dùng để báo cáo, nên số tuyệt đối có thể lạc quan.

– Tập kiểm định nhỏ (120 ảnh polyp, 127 đối tượng).

– Mô hình chỉ học từ một nguồn ảnh. Ảnh nền chỉ lấy từ một lớp manh tràng bình thường nên chưa đa dạng, thể hiện qua tỉ lệ báo nhầm cao trên PolypGen.

– Hai mươi seed chỉ phản ánh dao động do huấn luyện, không phản ánh dao động do dữ liệu. Ước tính cần khoảng 42 cặp để có lực kiểm định 80% với mức chênh lệch quan sát được.

– Thí nghiệm tách thành phần chỉ có 5 seed. Các biến thể có số tham số khác nhau, nên chưa tách được phần do cấu trúc với phần do thêm tham số.

– Nhiều phép so sánh phụ không được hiệu chỉnh, nên các kết quả có $p < 0{,}05$ ngoài chỉ số chính chỉ là gợi ý.

Về mô hình:

– Chỉ thử một vị trí tích hợp (tầng 10), một cỡ mô hình (s) và một độ phân giải đầu vào (640).

– Phần tham số mới của khối tích hợp được khởi tạo ngẫu nhiên, trong khi khối C2PSA bị thay thế đã được tiền huấn luyện trên COCO.

– Phép quét viết bằng PyTorch thuần nên chậm.

Về so sánh: PraNet và U-Net được huấn luyện ở độ phân giải khác và không tăng cường dữ liệu, nên so sánh với hai mô hình này chỉ mang tính tham khảo.

Về ứng dụng:

– Chỉ xử lý ảnh tĩnh, chưa xử lý video.

– Chưa có HTTPS và giới hạn tần suất gọi API.

– Ứng dụng di động chưa có tài khoản và lịch sử.

– Ứng dụng không phải thiết bị y tế.

4. HƯỚNG PHÁT TRIỂN

1. **Đánh giá chặt chẽ hơn:**

+ Thêm tập kiểm thử độc lập, tốt nhất là tách theo bệnh nhân hoặc theo trung tâm.

+ Tăng số seed hoặc dùng kiểm định chéo.
+ Mở rộng thí nghiệm tách thành phần lên 20 seed, kèm biến thể có số tham số tương đương để tách ảnh hưởng của dung lượng mô hình.

2. **Cải thiện khả năng tổng quát hóa:**

+ Huấn luyện trên nhiều nguồn ảnh (Kvasir-SEG, CVC-ClinicDB, PolypGen…).
+ Đa dạng hóa ảnh nền.
+ Thử tiền huấn luyện khối VMamba trên tập ảnh lớn trước khi ghép vào YOLO26-seg.

3. **Thử các vị trí và cách tích hợp khác:** đặt khối VMamba ở vùng kết hợp đặc trưng đa tỉ lệ (neck) hoặc ở nhiều tầng. Đây cũng là hướng mà đề cương gợi ý.
4. **Tăng tốc:** dùng kernel selective-scan viết bằng CUDA, đo lại FLOPs có tính cả phép quét, thử lượng tử hóa để đưa mô hình lên thiết bị biên.
5. **Bổ sung độ đo về hình dạng và biên** (Hausdorff 95, Boundary F1, clDice) để kiểm chứng trực tiếp mục tiêu "mặt nạ bám sát đường biên".
6. **Hoàn thiện ứng dụng:**

+ Xử lý video nội soi theo thời gian thực.
+ Bảo mật đường truyền và giới hạn tần suất gọi API.
+ Đồng bộ tài khoản và lịch sử giữa Web và di động.
+ Thu thập phản hồi của bác sĩ để đánh giá giá trị lâm sàng.

TÀI LIỆU THAM KHẢO

[1] F. Bray, M. Laversanne, H. Sung, J. Ferlay, R. L. Siegel, I. Soerjomataram, and A. Jemal, "Global cancer statistics 2022: GLOBOCAN estimates of incidence and mortality worldwide for 36 cancers in 185 countries," *CA Cancer J. Clin.*, vol. 74, no. 3, pp. 229–263, 2024, doi: 10.3322/caac.21834.

[2] A. G. Zauber *et al.*, "Colonoscopic polypectomy and long-term prevention of colorectal-cancer deaths," *N. Engl. J. Med.*, vol. 366, no. 8, pp. 687–696, Feb. 2012, doi: 10.1056/NEJMoa1100370.

[3] S. Zhao *et al.*, "Magnitude, risk factors, and factors associated with adenoma miss rate of tandem colonoscopy: A systematic review and meta-analysis," *Gastroenterology*, vol. 156, no. 6, pp. 1661–1674.e11, 2019, doi: 10.1053/j.gastro.2019.01.260.

[4] D. A. Corley *et al.*, "Adenoma detection rate and risk of colorectal cancer and death," *N. Engl. J. Med.*, vol. 370, no. 14, pp. 1298–1306, 2014, doi: 10.1056/NEJMoa1309086.

[5] D. Jha *et al.*, "Kvasir-SEG: A segmented polyp dataset," in *MultiMedia Modeling (MMM 2020)* (Lecture Notes in Computer Science, vol. 11962). Cham, Switzerland: Springer, 2020, pp. 451–462, doi: 10.1007/978-3-030-37734-2_37.

[6] G. Jocher, J. Qiu, M. Liu, S. Lyu, F. C. Akyon, and M. E. Kalfaoglu, "Ultralytics YOLO26: Unified real-time end-to-end vision models," 2026, *arXiv:2606.03748*, doi: 10.48550/arXiv.2606.03748.

[7] Y. Liu *et al.*, "VMamba: Visual state space model," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 37, 2024, pp. 103031–103063, doi: 10.52202/079017-3273.

[8] K. Pogorelov *et al.*, "KVASIR: A multi-class image dataset for computer aided gastrointestinal disease detection," in *Proc. 8th ACM Multimedia Syst. Conf. (MMSys)*, Taipei, Taiwan, 2017, pp. 164–169, doi: 10.1145/3083187.3083212.

[9] D.-P. Fan *et al.*, "PraNet: Parallel reverse attention network for polyp segmentation," in *Medical Image Computing and Computer Assisted Intervention – MICCAI 2020* (Lecture Notes in Computer Science,

vol. 12266). Cham, Switzerland: Springer, 2020, pp. 263–273, doi: 10.1007/978-3-030-59725-2_26.

[10] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015* (Lecture Notes in Computer Science, vol. 9351). Cham, Switzerland: Springer, 2015, pp. 234–241, doi: 10.1007/978-3-319-24574-4_28.

[11] J. Mei *et al.*, "A survey on deep learning for polyp segmentation: Techniques, challenges and future trends," *Vis. Intell.*, vol. 3, no. 1, Art. no. 1, 2025, doi: 10.1007/s44267-024-00071-w.

[12] S. Ali *et al.*, "A multi-centre polyp detection and segmentation dataset for generalisability assessment," *Sci. Data*, vol. 10, no. 1, Art. no. 75, 2023, doi: 10.1038/s41597-023-01981-y.

[13] J. Bernal, F. J. Sánchez, G. Fernández-Esparrach, D. Gil, C. Rodríguez, and F. Vilariño, "WM-DOVA maps for accurate polyp highlighting in colonoscopy: Validation vs. saliency maps from physicians," *Comput. Med. Imaging Graph.*, vol. 43, pp. 99–111, Jul. 2015, doi: 10.1016/j.compmedimag.2015.02.007.

[14] N. Tajbakhsh, S. R. Gurudu, and J. Liang, "Automated polyp detection in colonoscopy videos using shape and context information," *IEEE Trans. Med. Imag.*, vol. 35, no. 2, pp. 630–644, Feb. 2016, doi: 10.1109/TMI.2015.2487997.

[15] J. Silva, A. Histace, O. Romain, X. Dray, and B. Granado, "Toward embedded detection of polyps in WCE images for early diagnosis of colorectal cancer," *Int. J. Comput. Assist. Radiol. Surg.*, vol. 9, no. 2, pp. 283–293, 2014, doi: 10.1007/s11548-013-0926-3.

[16] B. Dong, W. Wang, D.-P. Fan, J. Li, H. Fu, and L. Shao, "Polyp-PVT: Polyp segmentation with pyramid vision transformers," *CAAI Artif. Intell. Res.*, vol. 2, Art. no. 9150015, 2023, doi: 10.26599/AIR.2023.9150015.

[17] N. T. Duc, N. T. Oanh, N. T. Thuy, T. M. Triet, and D. V. Sang, "ColonFormer: An efficient transformer based method for colon polyp segmentation," *IEEE Access*, vol. 10, pp. 80575–80586, 2022, doi: 10.1109/ACCESS.2022.3195241.

[18] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, USA, 2016, pp. 779–788, doi: 10.1109/CVPR.2016.91.

[19] D. Bolya, C. Zhou, F. Xiao, and Y. J. Lee, “YOLACT: Real-time instance segmentation,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Seoul, South Korea, 2019, pp. 9156–9165, doi: 10.1109/ICCV.2019.00925.

[20] A. Wang *et al.*, “YOLOv10: Real-time end-to-end object detection,” in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 37, 2024, pp. 107984–108011, doi: 10.52202/079017-3429.

[21] I. Pacal and D. Karaboga, “A robust real-time deep learning based automatic polyp detection system,” *Comput. Biol. Med.*, vol. 134, Art. no. 104519, Jul. 2021, doi: 10.1016/j.compbiomed.2021.104519.

[22] A. Gu, K. Goel, and C. Ré, “Efficiently modeling long sequences with structured state spaces,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2022. [Online]. Available: https://openreview.net/forum?id=uYLFoz1vlAC

[23] A. Gu and T. Dao, “Mamba: Linear-time sequence modeling with selective state spaces,” in *Proc. 1st Conf. Lang. Model. (COLM)*, Philadelphia, PA, USA, 2024. [Online]. Available: https://openreview.net/forum?id=tEYskw1VY2

[24] J. Ma, F. Li, and B. Wang, “U-Mamba: Enhancing long-range dependency for biomedical image segmentation,” 2024, *arXiv:2401.04722*, doi: 10.48550/arXiv.2401.04722.

[25] J. Liu *et al.*, “Swin-UMamba: Mamba-based UNet with ImageNet-based pretraining,” in *Medical Image Computing and Computer Assisted Intervention – MICCAI 2024* (Lecture Notes in Computer Science, vol. 15009). Cham, Switzerland: Springer, 2024, pp. 615–625, doi: 10.1007/978-3-031-72114-4_59.

[26] Z. Xu *et al.*, “Polyp-Mamba: Polyp segmentation with visual Mamba,” in *Medical Image Computing and Computer Assisted Intervention – MICCAI 2024* (Lecture Notes in Computer Science, vol. 15008). Cham, Switzerland: Springer, 2024, pp. 510–521, doi: 10.1007/978-3-031-72111-3_48.

[27] R. Fu, S. Hu, X. Zheng, C. Tang, and X. Liu, “PolyMamba: Spatial-prior guided Mamba for polyp segmentation with high-frequency enhancement,” in *Medical Image Computing and Computer Assisted Intervention – MICCAI 2025* (Lecture Notes in Computer Science, vol. 15970). Cham, Switzerland: Springer, 2025, pp. 455–465, doi: 10.1007/978-3-032-05141-7_44.

[28] D. Adame *et al.*, “Topo-VM-UNetV2: Encoding topology into vision Mamba UNet for polyp segmentation,” in *Proc. IEEE 38th Int. Symp. Comput.-Based Med. Syst. (CBMS)*, 2025, pp. 258–263, doi: 10.1109/CBMS65348.2025.00059.

[29] Z. Wang, C. Li, H. Xu, X. Zhu, and H. Li, “Mamba YOLO: A simple baseline for object detection with state space model,” in *Proc. AAAI Conf. Artif. Intell.*, vol. 39, no. 8, 2025, pp. 8205–8213, doi: 10.1609/aaai.v39i8.32885.

[30] S.-W. Chan *et al.*, “MambaPSA: A Mamba-based replacement for C2PSA in YOLO26,” 2026, *arXiv:2607.12681*.

[31] X. Bouthillier *et al.*, “Accounting for variance in machine learning benchmarks,” in *Proc. Mach. Learn. Syst. (MLSys)*, vol. 3, 2021. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2021/hash/0184b0cd3cfb185989f858a1d9f5c1eb-Abstract.html

[32] P. N. Lan, N. S. An, D. V. Hang, D. V. Long, T. Q. Trung, N. T. Thuy, and D. V. Sang, “NeoUNet: Towards accurate colon polyp segmentation and neoplasm detection,” in *Advances in Visual Computing (ISVC 2021)*, Lecture Notes in Computer Science, Cham, Switzerland: Springer, 2021, pp. 15–28, doi: 10.1007/978-3-030-90436-4_2.

[33] H. D. Viet *et al.*, “Validation of YOLOv8 algorithm in detecting colon polyps in endoscopy videos,” *J. Med. Artif. Intell.*, vol. 8, Art. no. 35, 2025, doi: 10.21037/jmai-24-338.

[34] K. He, G. Gkioxari, P. Dollár, and R. Girshick, “Mask R-CNN,” in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Venice, Italy, 2017, pp. 2980–2988, doi: 10.1109/ICCV.2017.322.

[35] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, “Feature pyramid networks for object detection,” in *Proc. IEEE Conf.*

Comput. Vis. Pattern Recognit. (CVPR), Honolulu, HI, USA, 2017, pp. 936–944, doi: 10.1109/CVPR.2017.106.

[36] S. Liu, L. Qi, H. Qin, J. Shi, and J. Jia, "Path aggregation network for instance segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Salt Lake City, UT, USA, 2018, pp. 8759–8768, doi: 10.1109/CVPR.2018.00913.

[37] C. Feng, Y. Zhong, Y. Gao, M. R. Scott, and W. Huang, "TOOD: Task-aligned one-stage object detection," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Montreal, QC, Canada, 2021, pp. 3490–3499, doi: 10.1109/ICCV48922.2021.00349.

[38] Z. Zheng, P. Wang, W. Liu, J. Li, R. Ye, and D. Ren, "Distance-IoU loss: Faster and better learning for bounding box regression," in *Proc. AAAI Conf. Artif. Intell.*, vol. 34, no. 7, 2020, pp. 12993–13000, doi: 10.1609/aaai.v34i07.6999.

[39] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal speed and accuracy of object detection," 2020, *arXiv:2004.10934*.

[40] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2019. [Online]. Available: https://arxiv.org/abs/1711.05101

[41] T.-Y. Lin *et al.*, "Microsoft COCO: Common objects in context," in *Computer Vision – ECCV 2014* (Lecture Notes in Computer Science, vol. 8693). Cham, Switzerland: Springer, 2014, pp. 740–755, doi: 10.1007/978-3-319-10602-1_48.

[42] G. Jocher, J. Qiu, and A. Chaurasia, *Ultralytics YOLO*, version 8.4.127. Ultralytics, 2026. [Online]. Available: https://github.com/ultralytics/ultralytics

[43] A. Vaswani *et al.*, "Attention is all you need," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017, pp. 5998–6008.

[44] A. Gu, K. Goel, A. Gupta, and C. Ré, "On the parameterization and initialization of diagonal state space models," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, 2022, pp. 35971–35983, doi: 10.52202/068431-2607.

[45] G. E. Blelloch, “Prefix sums and their applications,” School Comput. Sci., Carnegie Mellon Univ., Pittsburgh, PA, USA, Tech. Rep. CMU-CS-90-190, Nov. 1990.

[46] W. D. Hillis and G. L. Steele Jr., “Data parallel algorithms,” *Commun. ACM*, vol. 29, no. 12, pp. 1170–1183, Dec. 1986, doi: 10.1145/7902.7903.

[47] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, “The PASCAL visual object classes (VOC) challenge,” *Int. J. Comput. Vis.*, vol. 88, no. 2, pp. 303–338, 2010, doi: 10.1007/s11263-009-0275-4.

[48] Student, “The probable error of a mean,” *Biometrika*, vol. 6, no. 1, pp. 1–25, 1908, doi: 10.2307/2331554.

[49] F. Wilcoxon, “Individual comparisons by ranking methods,” *Biometrics Bull.*, vol. 1, no. 6, pp. 80–83, 1945, doi: 10.2307/3001968.

[50] J. Demšar, “Statistical comparisons of classifiers over multiple data sets,” *J. Mach. Learn. Res.*, vol. 7, pp. 1–30, 2006. [Online]. Available: https://www.jmlr.org/papers/v7/demsar06a.html

[51] D. Lakens, “Calculating and reporting effect sizes to facilitate cumulative science: A practical primer for t-tests and ANOVAs,” *Front. Psychol.*, vol. 4, Art. no. 863, 2013, doi: 10.3389/fpsyg.2013.00863.

[52] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. New York, NY, USA: Pearson, 2018.

[53] D. Jha, “Kvasir-SEG data (polyp segmentation & detection),” Kaggle. [Online]. Available: https://www.kaggle.com/datasets/debeshjha1/kvasirseg (accessed Oct. 9, 2026).

[54] N. Otsu, “A threshold selection method from gray-level histograms,” *IEEE Trans. Syst., Man, Cybern.*, vol. 9, no. 1, pp. 62–66, 1979, doi: 10.1109/TSMC.1979.4310076.

[55] S. Suzuki and K. Abe, “Topological structural analysis of digitized binary images by border following,” *Comput. Vis., Graph., Image Process.*, vol. 30, no. 1, pp. 32–46, 1985, doi: 10.1016/0734-189X(85)90016-7.

[56] D. H. Douglas and T. K. Peucker, “Algorithms for the reduction of the number of points required to represent a digitized line or its caricature,”

Cartographica, vol. 10, no. 2, pp. 112–122, 1973, doi: 10.3138/FM57-6770-U75U-7727.

[57] M. B. Brown and A. B. Forsythe, “Robust tests for the equality of variances,” *J. Amer. Statist. Assoc.*, vol. 69, no. 346, pp. 364–367, 1974, doi: 10.1080/01621459.1974.10482955.

PHỤ LỤC

PHỤ LỤC A. KẾT QUẢ TỪNG SEED

A.1. Mô hình gốc và mô hình đề xuất (20 seed)

Mỗi lượt lấy tại epoch đầu tiên đạt mask mAP50–95 cao nhất. P, R là Precision và Recall của mặt nạ tại cùng epoch. Cột "epoch" là epoch được chọn.

Bảng PL.1: Chỉ số mặt nạ của từng seed trên tập kiểm định

Seed	Gốc: mAP50–95	Gốc: mAP50	Gốc: P	Gốc: R	Gốc: epoch	TSVM: mAP50–95	TSVM: mAP50	TSVM: P	TSVM: R	TSVM: epoch	Δ mAP50–95
0	0,7366	0,9136	0,9535	0,8189	89	0,7245	0,9040	0,9312	0,8529	88	−0,0121
1	0,7165	0,8952	0,8533	0,8504	98	0,7274	0,9115	0,9437	0,8661	98	+0,0109
2	0,7138	0,9072	0,9139	0,8355	100	0,7259	0,9003	0,9220	0,8377	83	+0,0121
3	0,6941	0,9052	0,8786	0,8545	61	0,7213	0,9164	0,9188	0,8909	89	+0,0272
4	0,7274	0,9062	0,9387	0,8819	99	0,7197	0,9024	0,9018	0,8676	97	−0,0077
5	0,7350	0,9264	0,9318	0,8605	94	0,7285	0,9096	0,9013	0,8819	90	−0,0066
6	0,7153	0,9080	0,8865	0,8611	81	0,7254	0,9006	0,9223	0,8504	95	+0,0101
7	0,7145	0,9154	0,8983	0,8347	71	0,7065	0,8903	0,8595	0,8425	88	−0,0080
8	0,7318	0,9319	0,9114	0,8912	96	0,7339	0,9103	0,8883	0,8768	93	+0,0021
9	0,7253	0,9097	0,8568	0,8949	84	0,7329	0,9166	0,9288	0,8583	96	+0,0076
10	0,7045	0,8811	0,8780	0,8661	82	0,7198	0,9103	0,9542	0,8425	100	+0,0153
11	0,7299	0,9091	0,9253	0,8783	99	0,7226	0,9097	0,9168	0,8661	87	−0,0072
12	0,7056	0,8828	0,9019	0,8347	87	0,7157	0,9031	0,9162	0,8583	89	+0,0101
13	0,7315	0,9109	0,9348	0,8347	97	0,7306	0,9250	0,8926	0,8819	99	−0,0009
14	0,7223	0,9072	0,9014	0,8637	74	0,7211	0,9064	0,9231	0,8500	96	−0,0012
15	0,7176	0,9040	0,9171	0,8713	80	0,7262	0,9100	0,9347	0,8425	84	+0,0086
16	0,7125	0,8965	0,9213	0,8425	96	0,7237	0,9121	0,9027	0,8763	93	+0,0113
17	0,7274	0,9227	0,8615	0,9134	73	0,7301	0,9219	0,9147	0,8740	100	+0,0027
18	0,7022	0,9004	0,8875	0,8583	94	0,7082	0,8935	0,8762	0,8361	47	+0,0060
19	0,7136	0,9045	0,9303	0,8408	71	0,7195	0,9113	0,9403	0,8268	91	+0,0059
TB ± SD	0,7189 ± 0,0118	0,9069	0,9041	0,8594		0,7232 ± 0,0072	0,9083	0,9145	0,8590		+0,0043

Nguồn: nhóm thực hiện tính toán từ kết quả thực nghiệm.

A.2. Dice theo pixel của bốn mô hình (20 seed)

Bảng PL.2: Dice theo pixel trên 120 ảnh có polyp của tập kiểm định

Seed	YOLO26s-seg	TSVM	PraNet	U-Net
0	0,8727	0,8366	0,8547	0,7182
1	0,8624	0,8659	0,8635	0,7172
2	0,8394	0,8554	0,8619	0,6914
3	0,8232	0,8580	0,8475	0,7092
4	0,8736	0,8529	0,8431	0,6857
5	0,8468	0,8712	0,8503	0,6696

Seed	YOLO26s-seg	TSVM	PraNet	U-Net
6	0,8632	0,8349	0,8534	0,6844
7	0,8223	0,8293	0,8515	0,7313
8	0,8748	0,8552	0,8504	0,7080
9	0,8254	0,8636	0,8549	0,7322
10	0,8540	0,8647	0,8372	0,7255
11	0,8601	0,8715	0,8519	0,7446
12	0,8448	0,8656	0,8560	0,6898
13	0,8525	0,8651	0,8559	0,7413
14	0,8453	0,8421	0,8561	0,7092
15	0,8469	0,8372	0,8677	0,7039
16	0,8259	0,8562	0,8555	0,7367
17	0,8557	0,8636	0,8609	0,6861
18	0,8274	0,8443	0,8547	0,7236
19	0,8553	0,8468	0,8544	0,6852
TB ± SD	0,8486 ± 0,0171	0,8540 ± 0,0129	0,8541 ± 0,0068	0,7097 ± 0,0221

Nguồn: nhóm thực hiện tính toán từ kết quả thực nghiệm.

A.3. Thí nghiệm tách thành phần (5 seed)

Bảng PL.3: Mask mAP50–95 từng seed của năm cấu hình

Seed	Gốc (C2PSA)	TSVM đầy đủ	`vmamba_only`	`shape_only`	`shape_vmamba_no_guidance`
0	0,7366	0,7245	0,7145	0,7107	0,7160
1	0,7165	0,7274	0,7389	0,7091	0,7275
2	0,7138	0,7259	0,7145	0,7049	0,7209
3	0,6941	0,7213	0,7115	0,7296	0,7172
4	0,7274	0,7197	0,7054	0,7095	0,7193
TB ± SD	0,7177 ± 0,0160	0,7238 ± 0,0032	0,7170 ± 0,0128	0,7128 ± 0,0097	0,7201 ± 0,0045
Giờ huấn luyện TB	2,32	4,61	4,58	2,41	4,62

Nguồn: nhóm thực hiện tính toán từ kết quả thực nghiệm.

PHỤ LỤC B. CẤU HÌNH HUẤN LUYỆN

Hai mô hình YOLO, cùng ba biến thể tách thành phần, dùng chung toàn bộ siêu tham số dưới đây. Chúng chỉ khác nhau ở tệp cấu hình mô hình (`model`). Mô hình gốc khởi tạo trực tiếp từ `yolo26s-seg.pt`. Mô hình đề xuất và các biến thể được dựng từ tệp YAML rồi nạp trọng số từ cùng tệp này, chỉ ở các tầng trùng tên và trùng kích thước (mục 3.5).

Bảng PL.4: Siêu tham số huấn luyện (theo `args.yaml` của 40 lượt chính)

Nhóm	Tham số	Giá trị
Chung	`epochs`, `batch`, `imgsz`	100; 8; 640

Nhóm	Tham số	Giá trị
	workers, device	2; GPU 0
	patience	100 (không dừng sớm)
	amp	False (huấn luyện FP32)
	deterministic	True
	seed	0–19 (lượt chính), 0–4 (tách thành phần)
Tối ưu	optimizer	AdamW
	lr0, lrf	0,001; 0,01
	momentum, weight_decay	0,937; 0,0005
	warmup_epochs, cos_lr	5; False
Tăng cường dữ liệu	mosaic, close_mosaic	1,0; tắt ở 10 epoch cuối
	mixup	0
	fliplr, flipud, degrees	0,5; 0; 0
	scale, translate	0,5; 0,1
	hsv_h, hsv_s, hsv_v	0,015; 0,7; 0,4
Phân đoạn và đánh giá	overlap_mask, mask_ratio	True; 4
	iou, max_det	0,7; 300
	rect, cache	False; False

Nguồn: nhóm thực hiện tính toán từ kết quả thực nghiệm.

PHỤ LỤC C. TỔ CHỨC MÃ NGUỒN VÀ TỆP KẾT QUẢ

Bảng PL.5: Các thư mục chính của kho mã

Thư mục	Nội dung
01_du_lieu/	Bộ BG20 ở định dạng YOLO, danh sách chia train.txt/val.txt, script tiền xử lý
02_ma_nguon/ultralytics_TSVM_FP32/	Ultralytics 8.4.127 có thêm khối C2TSVMamba (nn/modules/topology_shape_vmamba.py) và bản vá lưu checkpoint FP32
02_ma_nguon/notebook_kaggle/	Notebook huấn luyện trên Kaggle, gồm cả notebook tách thành phần
03_ket_qua_huan_luyen/	40 lượt chính và 15 lượt tách thành phần; mỗi lượt có args.yaml, results.csv, weights/
05_so_sanh_mo_hinh_khac/	Trọng số PraNet và U-Net (20 seed mỗi mô hình)

Thư mục	Nội dung
00_tai_lieu_tong_quan/scripts/	Script tính lại số liệu, đánh giá theo pixel, topology, FLOPs và độ trễ GPU, tách thành phần
00_tai_lieu_tong_quan/bang/	Các bảng CSV kết quả dùng trong báo cáo
08_du_lieu_thuc_te/	Script và kết quả đánh giá trên bốn bộ dữ liệu ngoài
06_bao_cao/hinh/	Hình của báo cáo và script vẽ hình

Bảng PL.6: Script tạo ra số liệu trong báo cáo

Script	Đầu vào	Đầu ra	Dùng ở
tinh_lai_so_lieu.py	results.csv, args.yaml, ảnh dữ liệu	bang/01–09_*.csv	Ch.4, Ch.6 (mục 6.2)
danh_gia_pixel.py	Checkpoint 4 mô hình × 20 seed, 160 ảnh kiểm định	bang/10–12_*.csv	Ch.6 (mục 6.3)
danh_gia_topology.py	Như trên	bang/13–15_*.csv	Ch.6 (mục 6.4)
do_flops_gpu.py	Checkpoint seed 0 của hai mô hình	bang/16*_*.csv	Ch.6 (mục 6.5)
tinh_ablation.py	results.csv, args.yaml, best.pt của 25 lượt	bang/17*_*.csv, bang/18_*.csv	Ch.6 (mục 6.6)
08_du_lieu_thuc_te/scripts/	Checkpoint 4 mô hình × 20 seed, 4 bộ dữ liệu ngoài	08_du_lieu_thuc_te/bang/20seed/*.csv	Ch.6 (mục 6.7)
ve_hinh.py, ve_so_do.py	Các bảng CSV	Hình PNG 300 dpi và PDF	Toàn báo cáo

PHỤ LỤC D. HƯỚNG DẪN CHẠY LẠI

Các lệnh chạy từ thư mục gốc của kho mã. Script chỉ đọc dữ liệu và kết quả huấn luyện, chỉ ghi vào thư mục bang/ và hinh/. Các bước cần GPU dùng môi trường Python có PyTorch hỗ trợ CUDA.

```
# 1. Tính lại số liệu tổng hợp từ 40 lượt chính
python 00_tai_lieu_tong_quan/scripts/tinh_lai_so_lieu.py

# 2. Đánh giá theo pixel và topology của 4 mô hình (cần GPU)
python 00_tai_lieu_tong_quan/scripts/danh_gia_pixel.py
python 00_tai_lieu_tong_quan/scripts/danh_gia_topology.py

# 3. FLOPs và độ trễ GPU (cần GPU)
python 00_tai_lieu_tong_quan/scripts/do_flops_gpu.py
```

```
# 4. Tách thành phần, có đếm tham số từ checkpoint (cần PyTorch)
python 00_tai_lieu_tong_quan/scripts/tinh_ablation.py --torch

# 5. Vẽ lại toàn bộ hình
python 06_bao_cao/hinh/scripts/ve_hinh.py
```

Để huấn luyện lại một lượt, mở notebook trong `02_ma_nguon/notebook_kaggle/` trên Kaggle (GPU), đặt seed và tệp cấu hình mô hình cần chạy, rồi tải thư mục kết quả về `03_ket_qua_huan_luyen/`. Notebook đặt `deterministic=True` và khóa seed cho mọi thư viện. Trong đề tài, lượt `shape_only` seed 3 và seed 4 mỗi lượt có hai bản kết quả với thời gian huấn luyện khác nhau. Hai bản `results.csv` giống hệt nhau ở mọi cột chỉ số qua cả 100 epoch, cho thấy với cùng phiên bản thư viện và cùng loại GPU, kết quả lặp lại được.